DevOps with AWS: Deep Dive

ALPHATOOLZ

Chapter 1: Introduction to AWS DevOps

What is DevOps?

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops). It aims to shorten the development lifecycle and provide continuous delivery with high software quality. DevOps is all about culture, automation, and a commitment to continuous improvement and collaboration.

Key Principles of DevOps:

- Continuous Integration (CI): Integrating code into a shared repository several times a day.
- Continuous Deployment (CD): Automatically deploying code changes to production.
- Infrastructure as Code (IaC): Managing and provisioning computing infrastructure through machine-readable configuration files.
- Monitoring and Logging: Continuous monitoring of applications and infrastructure to identify issues and ensure high availability.
- Collaboration and Communication: Enhanced communication and collaboration between development and operations teams.

Why Choose AWS for DevOps?

AWS provides a suite of DevOps tools and services that allow organizations to implement DevOps practices effectively. It offers scalable infrastructure, flexible services, and powerful tools to automate and streamline the entire DevOps lifecycle.

Key AWS DevOps Services:

- AWS CodeCommit: A fully managed source control service that makes it easy for teams to host secure and scalable Git repositories.
- AWS CodeBuild: A fully managed build service that compiles source code, runs tests, and produces software packages.
- AWS CodeDeploy: A fully managed deployment service that automates software deployments to a variety of compute services such as Amazon EC2, AWS Fargate, and Lambda.
- AWS CodePipeline: A fully managed continuous integration and continuous delivery service for fast and reliable application and infrastructure updates.
- AWS CloudFormation: A service that helps you model and set up your Amazon Web Services resources so that you can spend less time managing those resources and more time focusing on your applications.

Key Features and Benefits

- Scalability: Easily scale your infrastructure and applications to meet demand.
- Automation: Automate repetitive tasks, such as testing and deployment, to improve efficiency and reduce errors.
- Security: Leverage AWS's robust security infrastructure and tools to protect your applications and data.

- Cost-Effective: Pay only for what you use with AWS's flexible pricing models.
- Collaboration: Use integrated tools to enhance collaboration between development and operations teams.
- Innovation: Focus on innovation by automating infrastructure management and deployment tasks.

Real-Life Example: Implementing DevOps for a FinTech Application

Scenario:

A FinTech startup needs to implement a DevOps pipeline to streamline the development, testing, and deployment of its web application. The application is built using a microservices architecture and needs to be highly available and secure.

Solution:

1. Source Control: Use AWS CodeCommit to host Git repositories for version control.
2. Build Automation: Use AWS CodeBuild to automate the building and testing of code.
3. Deployment Automation: Use AWS CodeDeploy to automate the deployment of code to Amazon EC2 instances.
4. Continuous Integration and Delivery: Use AWS CodePipeline to create a CI/CD pipeline that integrates CodeCommit, CodeBuild, and CodeDeploy.
5. Infrastructure as Code: Use AWS CloudFormation to manage infrastructure as code, ensuring consistency and repeatability.
6. Monitoring and Logging: Use AWS CloudWatch for monitoring and logging to ensure high availability and quick troubleshooting of issues.

System Design Diagram

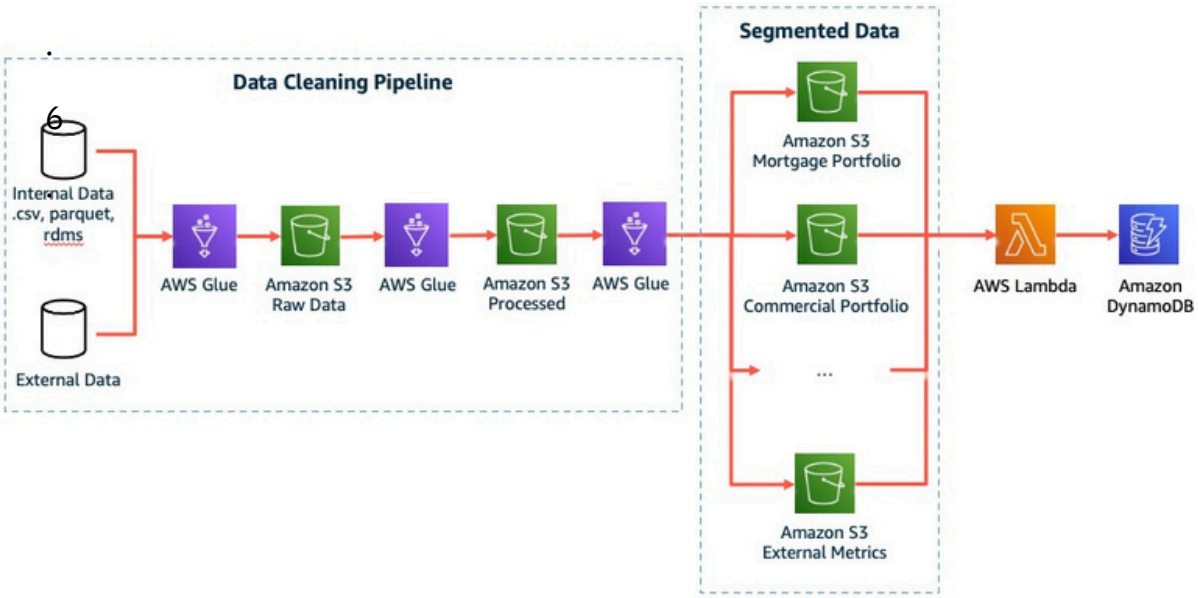


Figure 1: System Design Diagram of the FinTech Application's DevOps Pipeline

Cheat Sheets

AWS CLI Cheat Sheet:

- `aws codecommit create-repository --repository-name MyDemoRepo`
- `aws codebuild create-project --name MyBuildProject --source ...`
- `aws codedeploy create-deployment-group --application-name MyApplication --deployment-group-name MyDeploymentGroup ...`
- `aws codepipeline create-pipeline --pipeline-name MyPipeline --role-arn ...`
- `aws cloudformation create-stack --stack-name MyStack --template-body file://template.json`

Terraform Cheat Sheet for AWS:

- `provider "aws" { ... }`
- `resource "aws_instance" "my_instance" { ... }`
- `resource "aws_s3_bucket" "my_bucket" { ... }`
- `resource "aws_lambda_function" "my_lambda" { ... }`
- `resource "aws_vpc" "my_vpc" { ... }`

Conclusion

AWS provides a comprehensive suite of tools and services that make implementing DevOps practices straightforward and effective. By leveraging AWS's scalable infrastructure, robust security, and powerful automation tools, organizations can achieve faster development cycles, higher software quality, and improved collaboration between development and operations teams.

In the next chapter, we will dive into setting up your AWS environment, including creating an AWS account, configuring the AWS CLI, and setting up IAM roles and policies.

Chapter 2: Getting Started with AWS

In this chapter, we will guide you through the initial setup required to start using AWS for your DevOps needs. This includes creating an AWS account, setting up the AWS CLI, configuring IAM roles and policies, and setting up your first resources. We will use both the AWS Management Console (GUI) and Terraform to demonstrate these steps.

Creating an AWS Account

To start using AWS, you first need to create an AWS account.

1. Sign Up for AWS:
    - o Visit AWS Sign Up and click "Create an AWS Account".
    - o Follow the on-screen instructions to create your account.

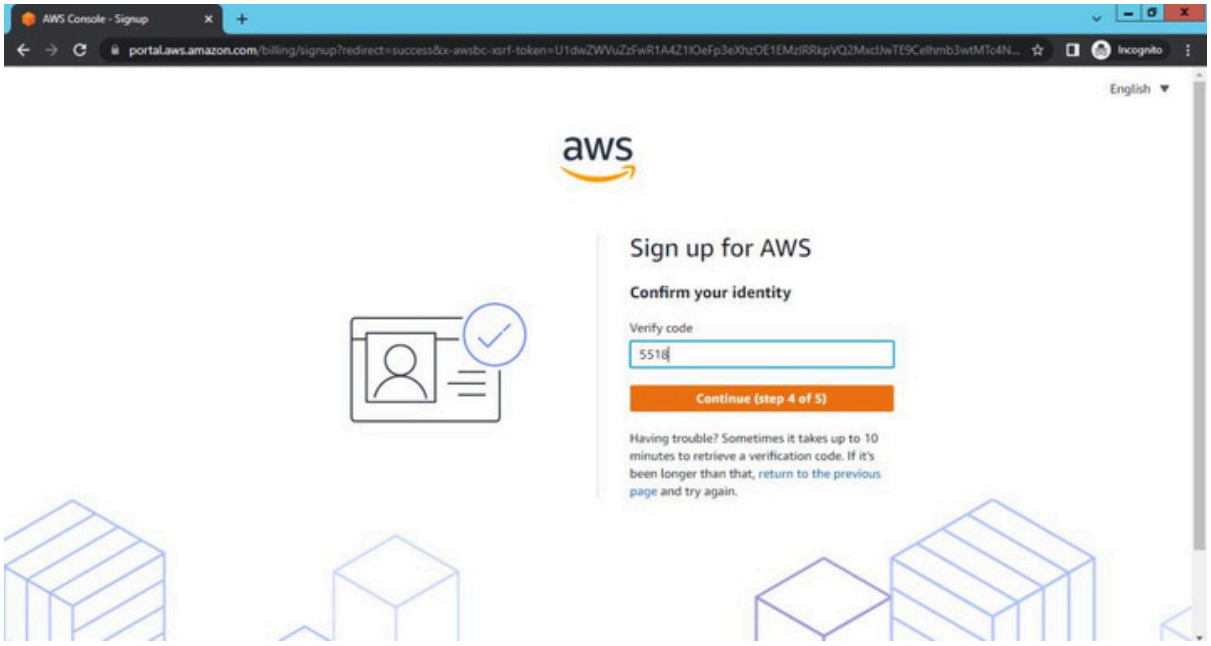


Figure 1: AWS Sign Up Page

2. Accessing the AWS Management Console:
    - After creating your account, log in to the AWS Management Console.

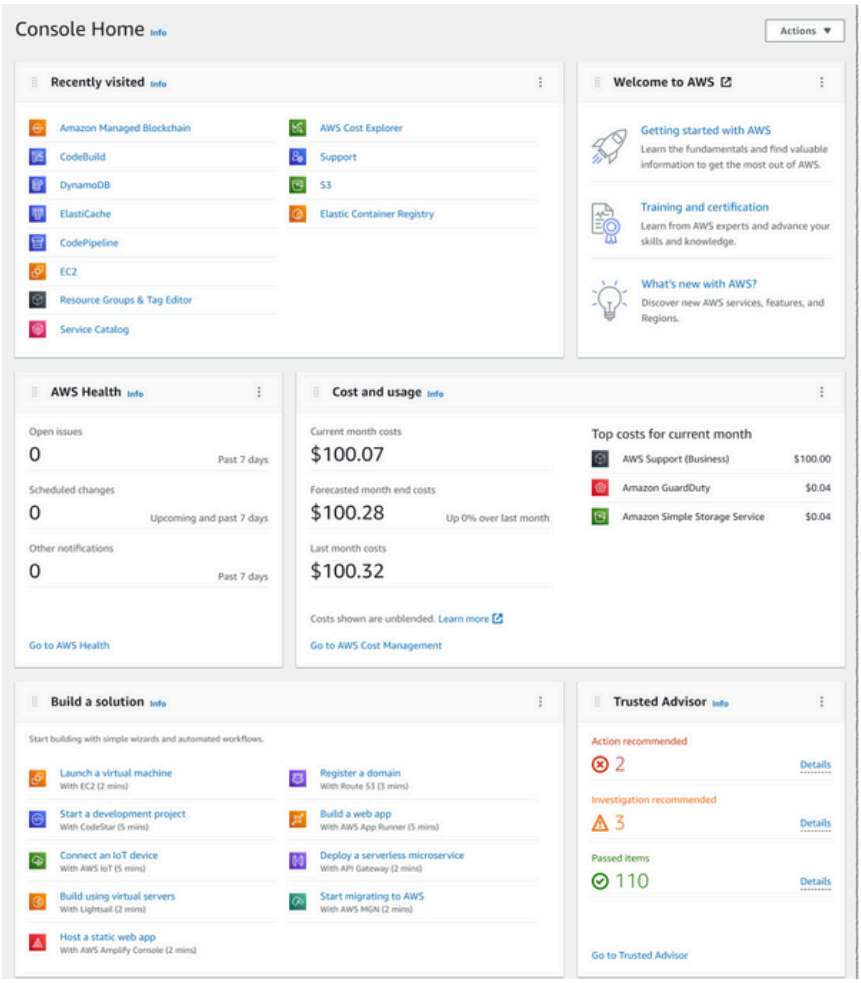


Figure 2: AWS Management Console Dashboard

Setting Up the AWS CLI

The AWS Command Line Interface (CLI) is a unified tool to manage your AWS services. Using the AWS CLI, you can control multiple AWS services from the command line and automate them through scripts.

1. Install the AWS CLI:
    - o Windows:
        - Download the MSI installer from the AWS CLI Installation Page.
        - Run the installer and follow the on-screen instructions.
    - o Linux/Mac:

```
bash

curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o
"AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
```

2. Configure the AWS CLI:

```
bash

aws configure
```

You will be prompted to enter your AWS Access Key ID, Secret Access Key, default region, and output format.

Example:

```
bash
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]:
wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

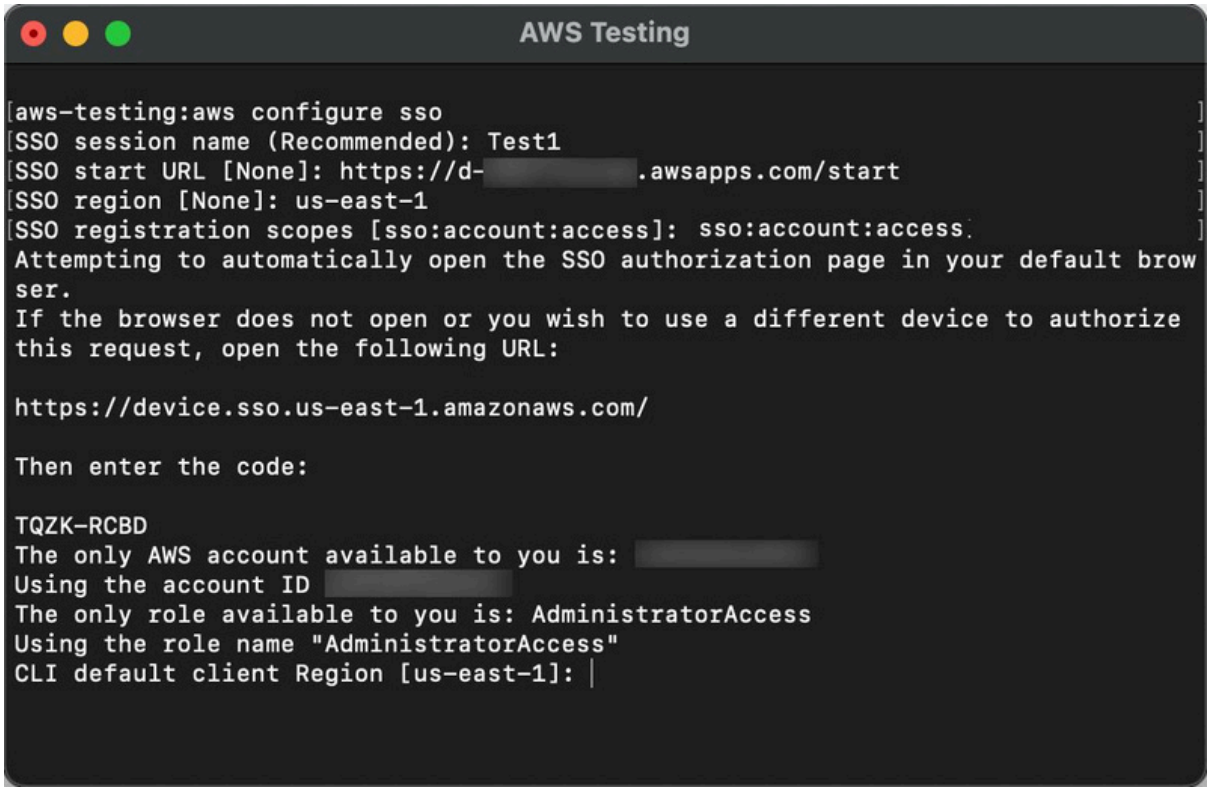


Figure 3: AWS CLI Configuration

Configuring IAM Roles and Policies

IAM (Identity and Access Management) roles and policies are critical for managing permissions in AWS.

1. Creating an IAM Role:
    - Navigate to the IAM dashboard in the AWS Management Console.
    - Click on "Roles" and then "Create role".
    - Select the trusted entity type (e.g., AWS service) and attach policies that define the role's permissions.

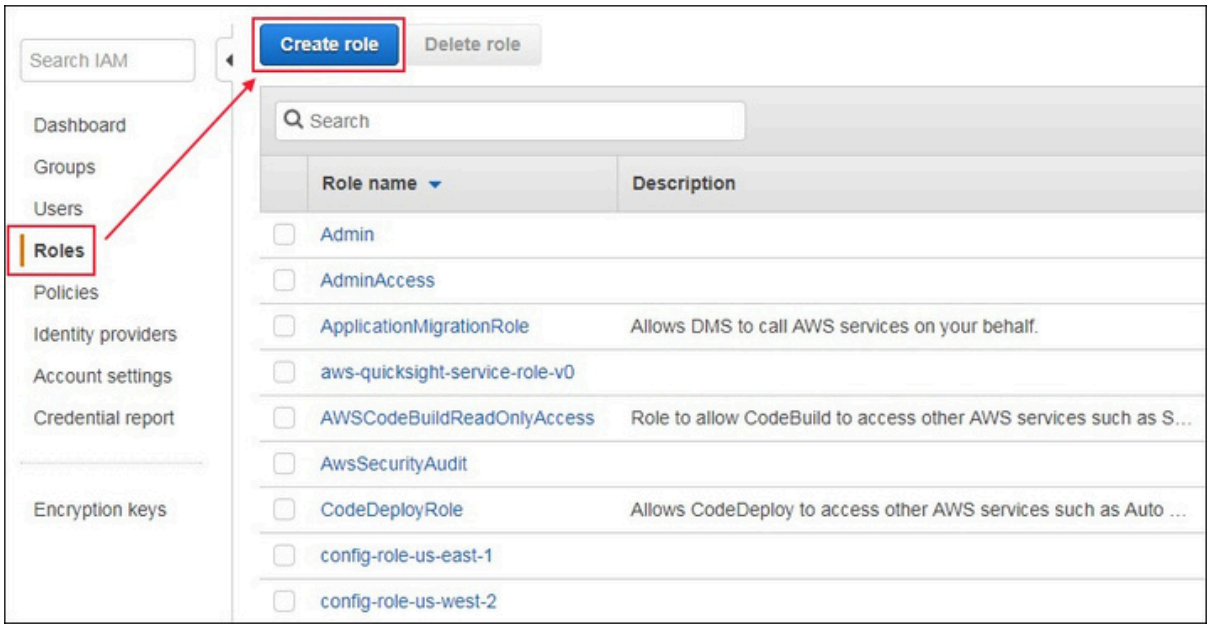


Figure 4: IAM Role Creation

2. Creating an IAM Policy:
    - In the IAM dashboard, click on "Policies" and then "Create policy".
    - Use the policy generator or JSON editor to define the policy.

Example Policy (Full Access to S3):

```
json

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": "*"
    }
  ]
}
```

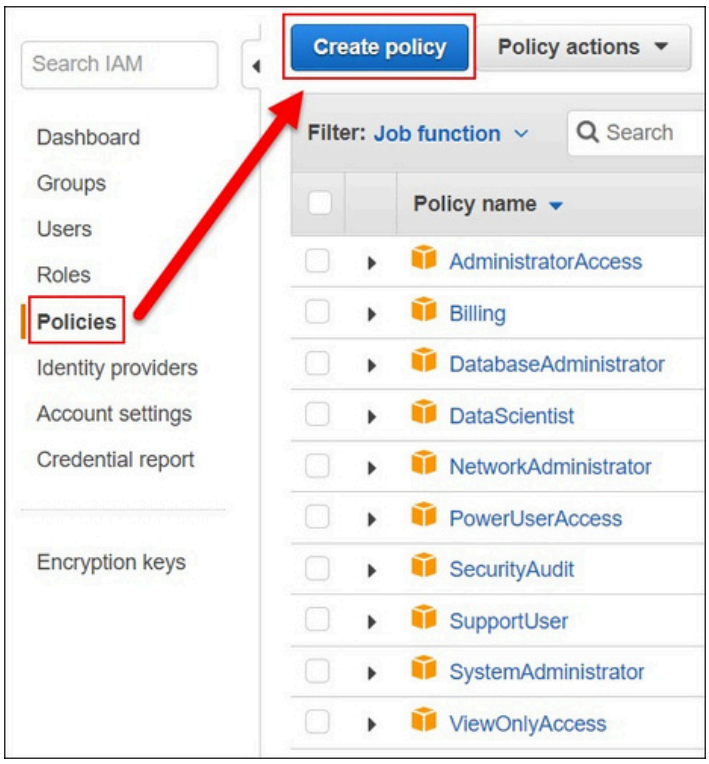


Figure 5: IAM Policy Creation

Setting Up Your First Resources

Using the AWS Management Console

1. Creating an S3 Bucket:
    - Navigate to the S3 service in the AWS Management Console.
    - Click on "Create bucket", enter a unique bucket name, select a region, and follow the remaining steps.

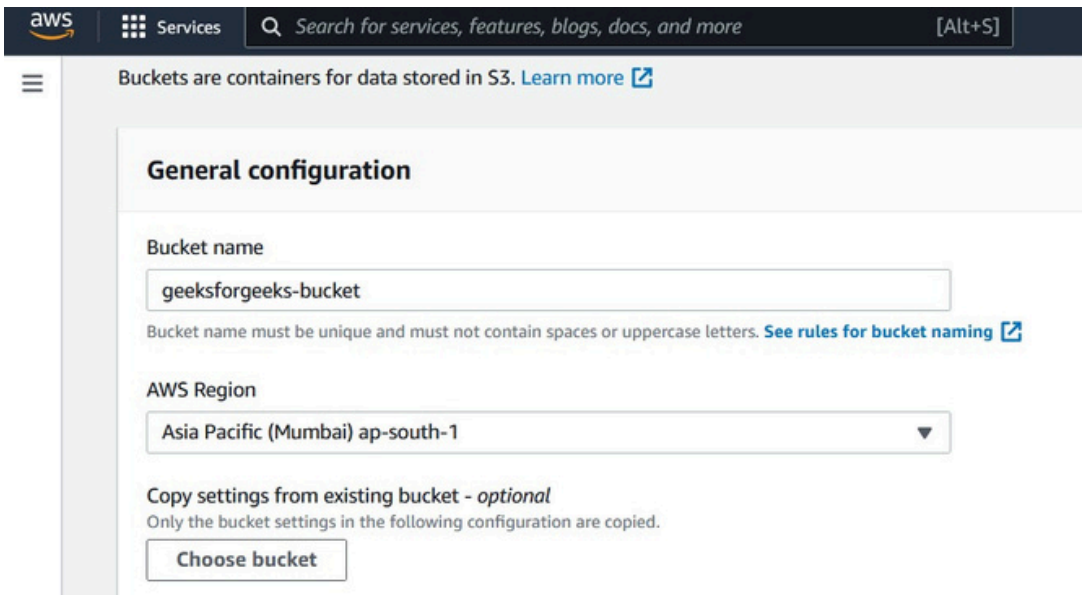


Figure 6: S3 Bucket Creation

Launching an EC2 Instance:

- Navigate to the EC2 service.
- Click on "Launch Instance", select an Amazon Machine Image (AMI), choose an instance type, configure instance details, add storage, and review your instance.

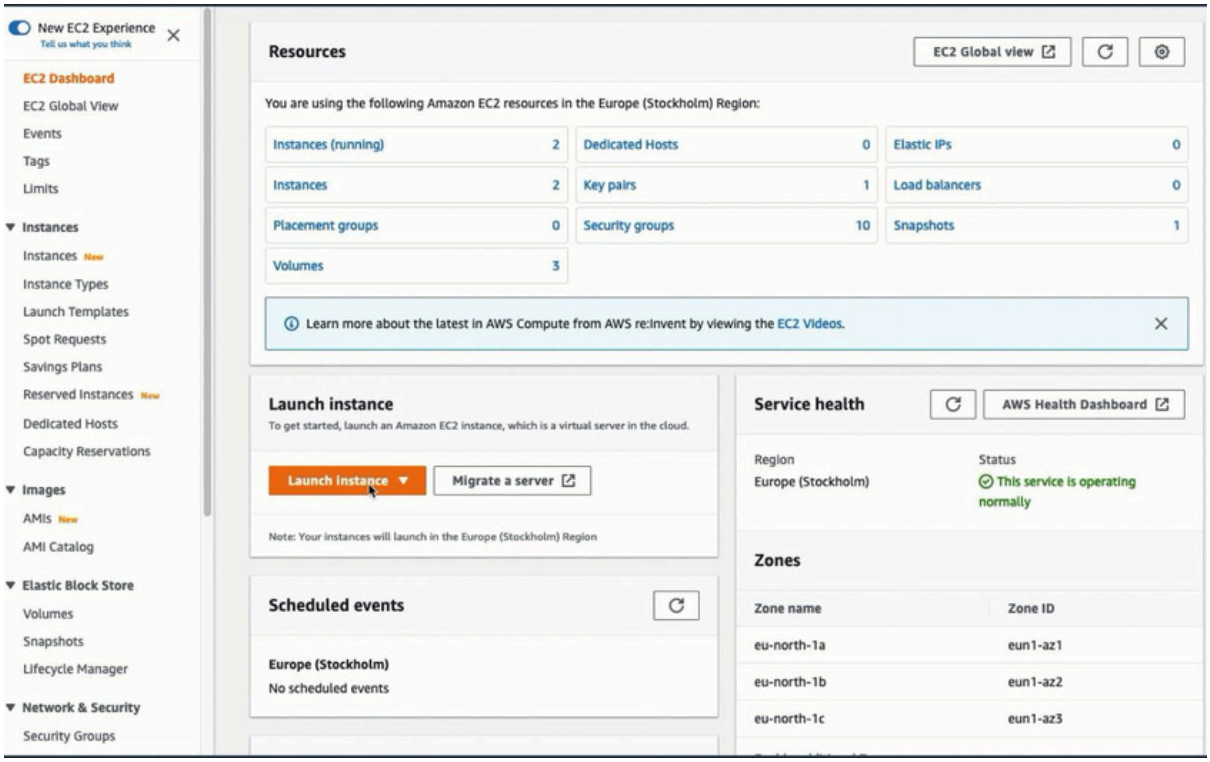


Figure 7: EC2 Instance Launch

Using Terraform

Terraform is an Infrastructure as Code (IaC) tool that allows you to define and provision infrastructure using code.

1. Install Terraform:
   - Follow the instructions on the Terraform Installation Page to install Terraform on your system.
2. Create a Terraform Configuration File:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-unique-bucket-name"
  acl    = "private"
}
resource "aws_instance" "my_instance" {
```

```
 ami             = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "MyInstance"
  }
}
```

3. Initialize Terraform and Apply the Configuration:

```
bash

terraform init
terraform apply
```

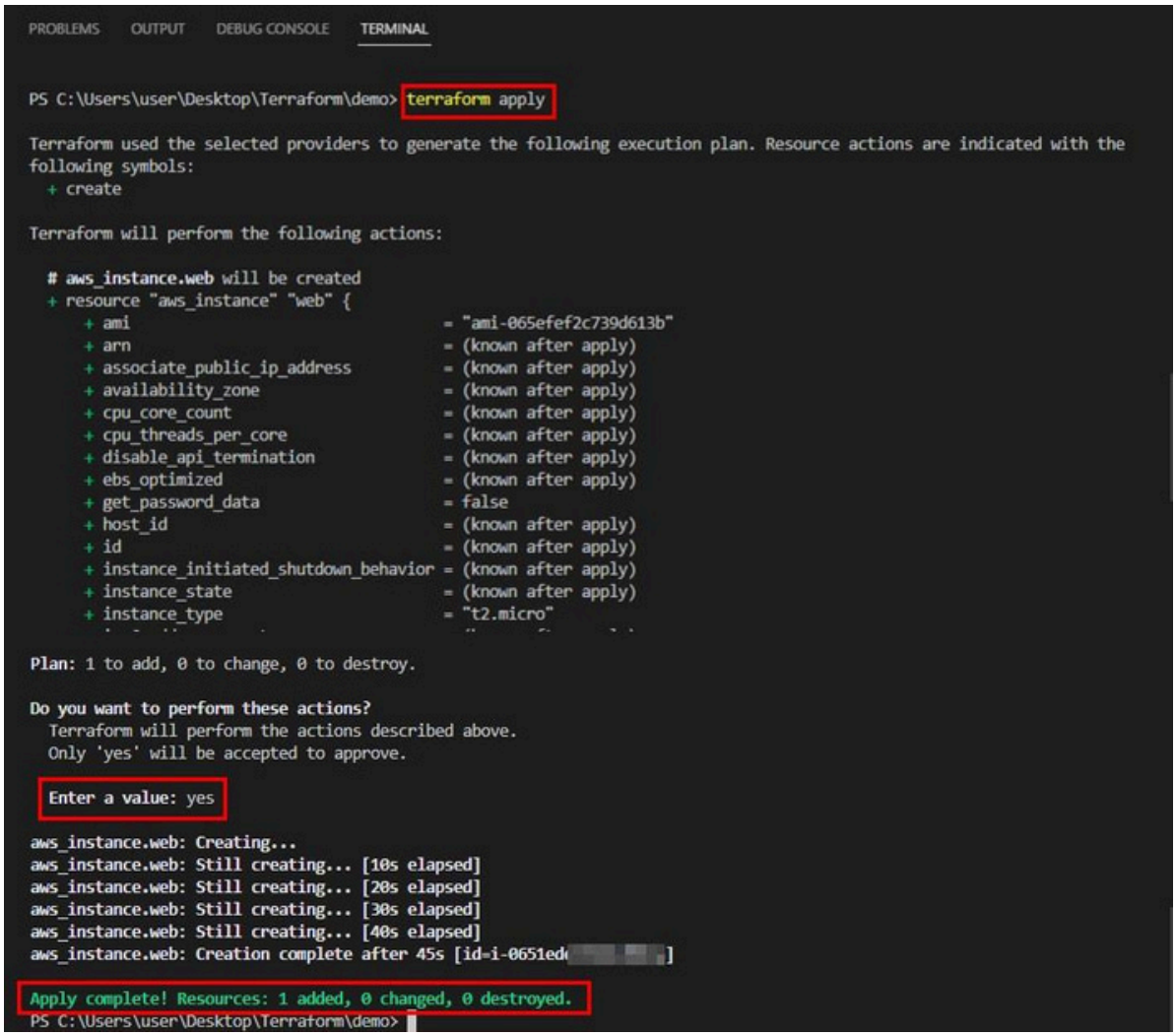


Figure 8: Terraform Apply Command

Cheat Sheets

AWS CLI Cheat Sheet:

- S3:

```
bash
aws s3 mb s3://my-bucket
aws s3 cp myfile.txt s3://my-bucket/
aws s3 ls s3://my-bucket/
```

- EC2:

```
bash aws ec2 run-instances --image-id ami-0c55b159cbfafe1f0 --

instance-
type t2.micro --key-name MyKeyPair
aws ec2 describe-instances --instance-ids i-1234567890abcdef0
aws ec2 terminate-instances --instance-ids i-1234567890abcdef0
```

Terraform Cheat Sheet:

- Initialize:

```
bash

terraform init
```

- Plan and Apply:

```
bash

terraform plan
terraform apply
```

- Destroy:

```
bash

terraform destroy
```

Conclusion
By setting up your AWS account, configuring the AWS CLI, and creating IAM roles and policies, you have laid the foundation for using AWS in your DevOps workflows. Whether you prefer using the AWS Management Console or Terraform for Infrastructure as Code, AWS provides powerful tools and services to streamline and automate your DevOps processes.

In the next chapter, we will dive into source control with Azure Repos, covering essential practices for managing your codebase and collaborating with your team effectively.

Chapter 3: Identity and Access Management (IAM)

In this chapter, we will delve into AWS Identity and Access Management (IAM), a powerful service for managing access to AWS resources securely. We will cover essential concepts, GUI steps, and Terraform configurations to provide a comprehensive understanding of IAM.

Key Concepts of IAM

- Users: Individuals or applications that interact with AWS.
- Groups: Collections of users that share the same permissions.
- Roles: Entities that define a set of permissions that can be assumed by users or services.
- Policies: Documents that define permissions and are attached to users, groups, or roles.

Creating IAM Users and Groups

Using AWS Management Console

1. Creating an IAM User:
    - Navigate to the IAM dashboard.
    - Click on "Users" and then "Add user".
    - Enter the user details and select the access type (programmatic access or AWS Management Console access).

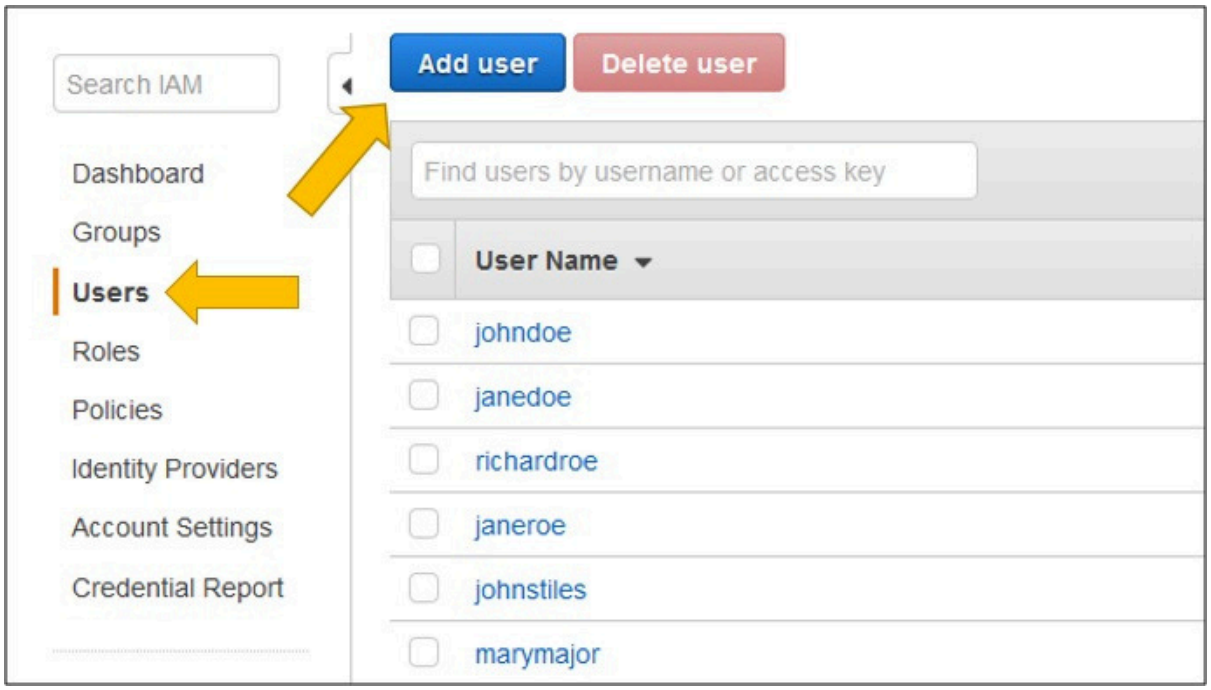


Figure 1: IAM User Creation

Creating an IAM Group:

- In the IAM dashboard, click on "User groups" and then "Create group".
- Enter a group name and attach policies to the group.

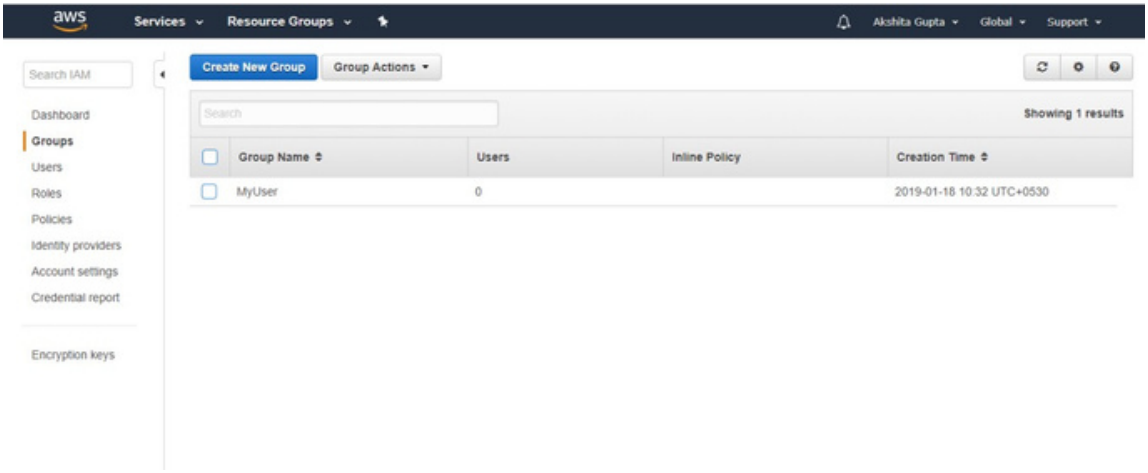


Figure 2: IAM Group Creation

Adding a User to a Group:

- Select the group and click on "Add users to group".
- Choose the users to add and confirm.

Using Terraform

Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_iam_user" "example_user" {
  name = "example_user"
}

resource "aws_iam_group" "example_group" {
  name = "example_group"
}

resource "aws_iam_group_membership" "example_membership" {
  group = aws_iam_group.example_group.name
  users = [aws_iam_user.example_user.name]
}
resource "aws_iam_group_policy_attachment" "example_attach" {
  group      = aws_iam_group.example_group.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
}
```

Commands:

```
bash

terraform init
terraform apply
```

Creating and Managing IAM Roles

Using AWS Management Console

1. Creating an IAM Role:
    - Navigate to the IAM dashboard.
    - Click on "Roles" and then "Create role".
    - Select the trusted entity (e.g., AWS service, another AWS account).
    - Attach policies to the role.

2. Assigning a Role to an EC2 Instance:
    - Navigate to the EC2 dashboard.
    - Select an instance and click on "Actions", then "Instance Settings", and "Attach/Replace IAM role".
    - Choose the role to assign and apply the changes.

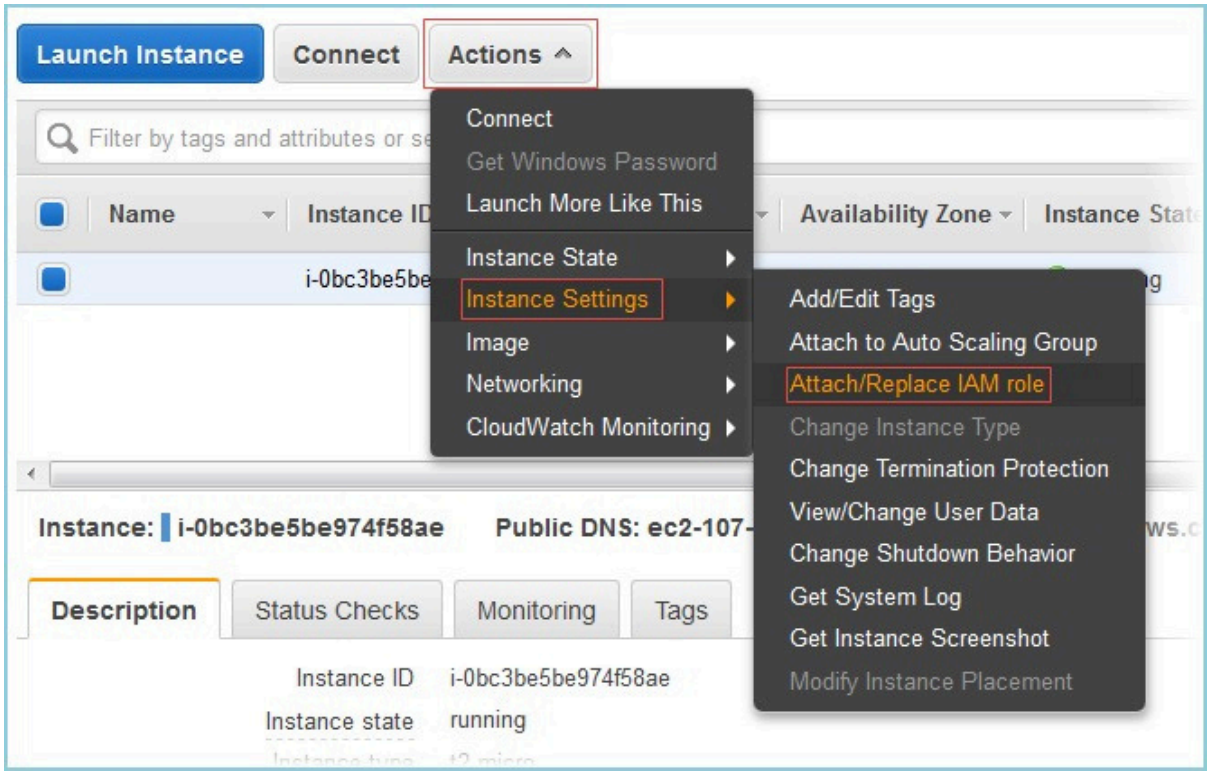


Figure 6: Assign Role to EC2 Instance

Using Terraform

Terraform Configuration:

```
hcl

resource "aws_iam_role" "example_role" {
  name = "example_role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Principal = {
          Service = "ec2.amazonaws.com"
        }
        Effect = "Allow"
        Sid = ""
      }
    ]
  })
}
resource "aws_iam_role_policy" "example_policy" {
  name   = "example_policy"
  role   = aws_iam_role.example_role.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "s3:ListBucket"
        ]
        Resource = "*"
      }
    ]
  })
}
```

Commands:

```
bash

terraform init
terraform apply
```

IAM Policies

IAM policies define permissions and can be attached to users, groups, or roles.

Using AWS Management Console

1. Creating an IAM Policy:
    - Navigate to the IAM dashboard.
    - Click on "Policies" and then "Create policy".
    - Use the visual editor or JSON editor to define the policy.

Example Policy (Read-Only Access to S3):

```
json

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "*"
    }
  ]
}
```

2. Attaching a Policy to a User:
    - Navigate to the user in the IAM dashboard.
    - Click on "Add permissions", then "Attach policies".
    - Select the policy and attach it to the user.

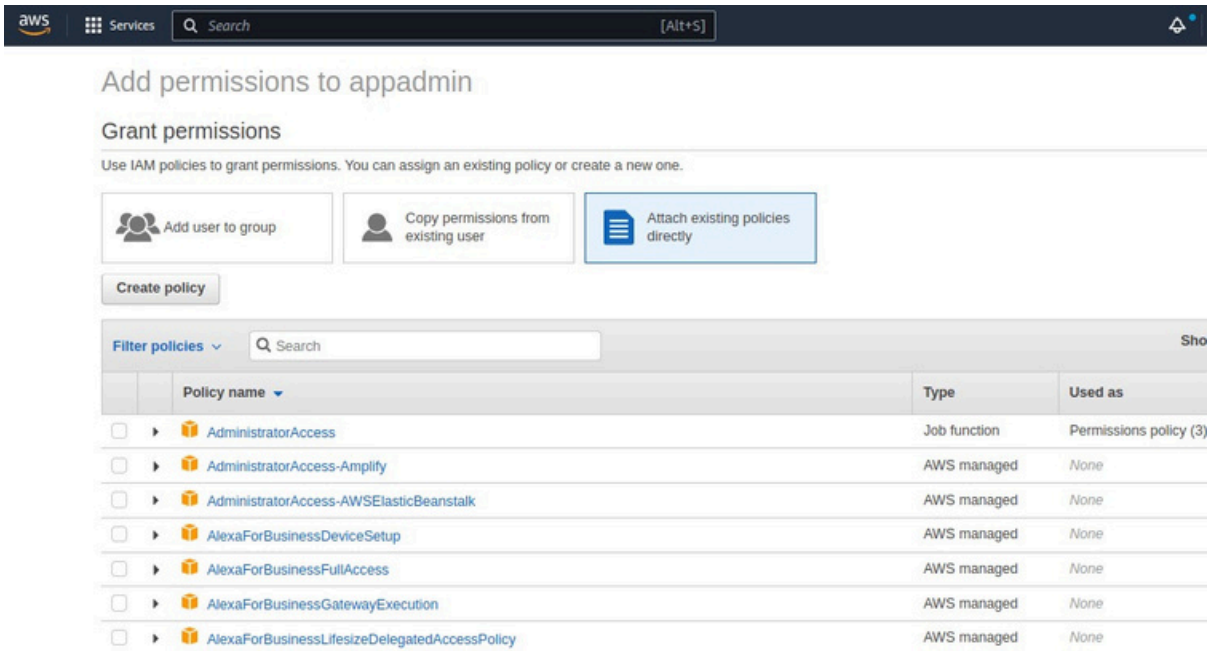


Figure 9: Attach Policy to User

Terraform Configuration:

```
hcl

resource "aws_iam_policy" "example_policy" {
  name        = "example_policy"
  description = "A test policy"
  policy      = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = [
          "ec2:Describe*",
          "s3:List*"
        ]
        Effect   = "Allow"
        Resource = "*"
      }
    ]
  })
}

resource "aws_iam_user_policy_attachment" "example_attach" {
  user       = aws_iam_user.example_user.name
  policy_arn = aws_iam_policy.example_policy.arn
}
```

Commands:

```
bash

terraform init
terraform apply
```

Cheat Sheets

AWS CLI Cheat Sheet for IAM:

- Create a User:

  ```
  bash

  aws iam create-user --user-name example_user
  ```

- Create a Group:

  ```
  bash

  aws iam create-group --group-name example_group
  ```

- Add User to Group:

```
bash

aws iam add-user-to-group --user-name example_user --group-name
example_group
```

- Create a Role:

```
bash

aws iam create-role --role-name example_role --assume-role-policy-
document file://trust-policy.json
```

- Attach Policy to Role:

```
bash

aws iam attach-role-policy --role-name example_role --policy-arn
arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

Terraform Cheat Sheet for IAM:

- Create IAM User:

```
hcl
resource "aws_iam_user" "example_user" {

  name = "example_user"
}
```

- Create IAM Group:

```
hcl

resource "aws_iam_group" "example_group" {
  name = "example_group"
}
```

- Attach Policy to Role:

```
hcl

resource "aws_iam_role_policy_attachment" "example_attach" {
  role       = aws_iam_role.example_role.name
  policy_arn = aws_iam_policy.example_policy.arn
}
```

Real-Life Scenarios

Scenario 1: Enforcing Least Privilege for DevelopersTo ensure developers have only the permissions they need, create IAM groups for different roles (e.g., developers, admins) and attach least privilege policies. This minimizes security risks.

Scenario 2: Automating Role Creation for EC2 Instances Automate the creation of roles and their attachment to EC2 instances using Terraform. This ensures consistent and repeatable infrastructure setup.

Conclusion

IAM is a crucial component for managing access to AWS resources. By understanding and utilizing IAM roles, policies, users, and groups, you can ensure secure and efficient access management within your AWS environment. Whether using the AWS Management Console or Terraform, IAM provides the flexibility and control needed to implement robust security practices.

In the next chapter, we will explore how to manage source control with Azure Repos, integrating best practices and tools to streamline your development workflows.

Chapter 4: Storage Solutions with S3

Amazon S3 (Simple Storage Service) is a highly scalable, durable, and secure object storage service. In this chapter, we will explore S3's capabilities, from basic operations to advanced configurations using both the AWS Management Console and Terraform. We'll provide fully coded examples, outputs, explanations, cheat sheets, architecture diagrams, and real-life scenarios.

Key Concepts of S3

- Buckets: Containers for storing objects.
- Objects: Files stored in buckets.
- Permissions: Access control for buckets and objects.
- Lifecycle Policies: Rules for managing object lifecycle.
- Versioning: Keeping multiple versions of an object.

Creating S3 Buckets

Using AWS Management Console

1. Creating an S3 Bucket:
    - Navigate to the S3 dashboard.
    - Click on "Create bucket".
    - Enter a bucket name and select a region.
    - Configure options such as versioning, logging, and permissions.
    - Click "Create bucket".

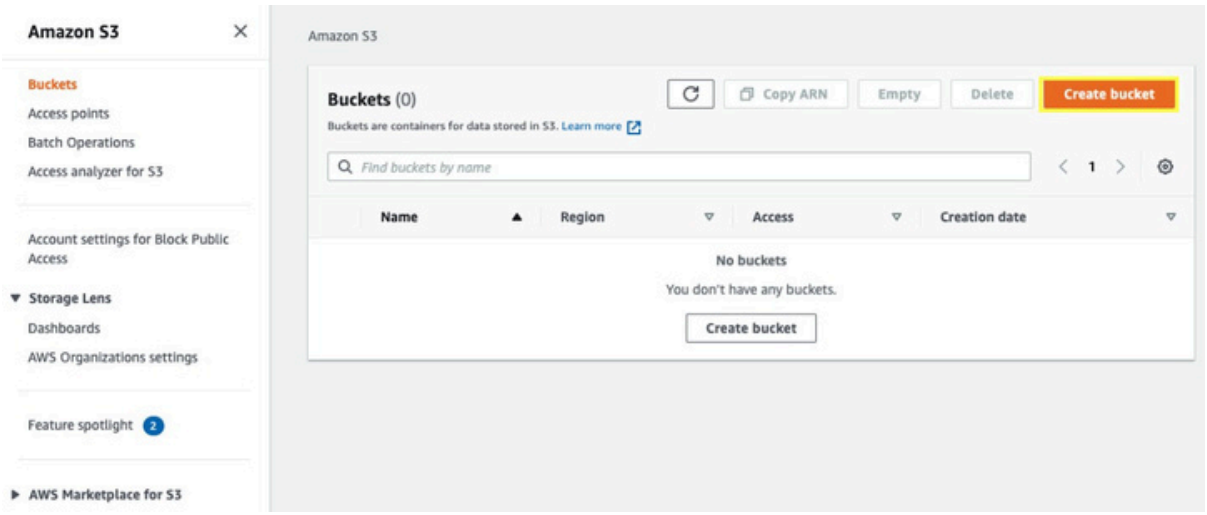


Figure 1: S3 Bucket Creation

Using Terraform

Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_s3_bucket" "example_bucket" {
  bucket = "example-bucket"
  acl    = "private"

  versioning {
    enabled = true
  }

  lifecycle_rule {
    id      = "log"
    enabled = true

    transition {
      days          = 30
      storage_class = "STANDARD_IA"
    }
    expiration {
    days = 365
    }
  }
  tags = {
  Name           = "example-bucket"
    Environment = "Dev"
  }
}
```

Commands:

```
bash

terraform init
terraform apply
```

```
PS C:\playground\aws> terraform apply

Terraform used the selected providers to generate the following execution plan. Resou
following symbols:
  + create

Terraform will perform the following actions:

  # aws_s3_bucket.s3_storage will be created
  + resource "aws_s3_bucket" "s3_storage" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
      + bucket                      = "learning-terraform-cli-first-bucket-2022-04"
      + bucket_domain_name          = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + s3_storage_arn = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

Figure 2: Terraform Apply Command

Uploading and Managing Objects

Using AWS Management Console

1. Uploading an Object:
    - Navigate to the bucket.
    - Click on "Upload".
    - Add files or folders, configure permissions, and upload.

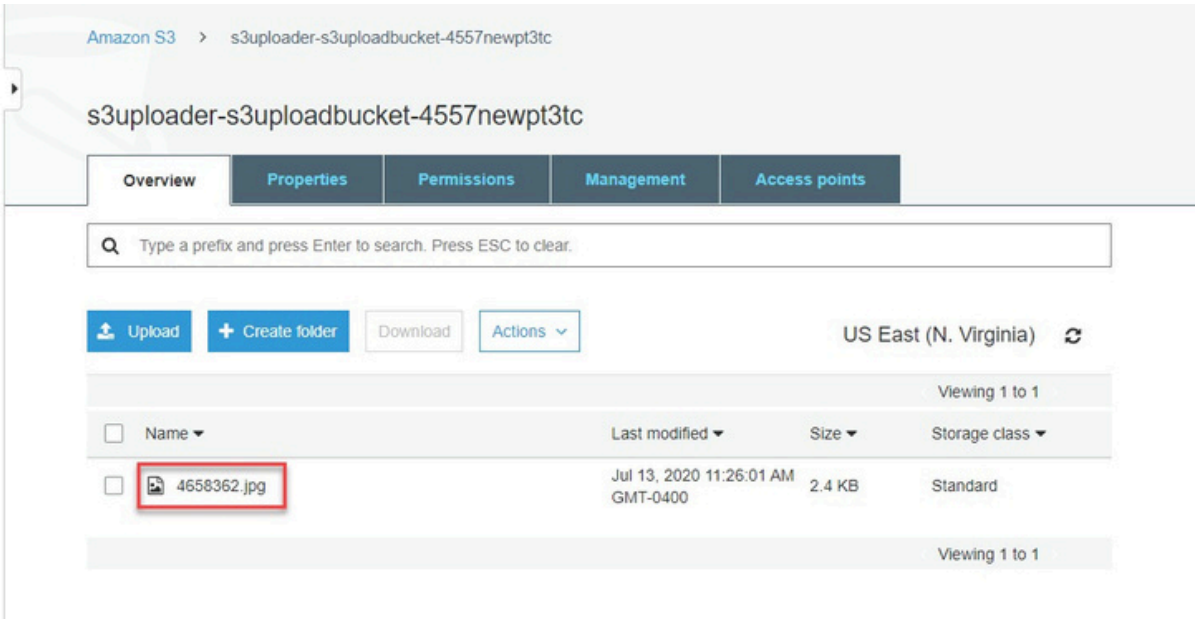


Figure 3: S3 Upload Object

Terraform Configuration for Object:

```
hcl

resource "aws_s3_bucket_object" "example_object" {
  bucket = aws_s3_bucket.example_bucket.bucket
  key    = "example.txt"
  source = "path/to/example.txt"
  acl    = "private"
}
```

AWS CLI Command:

```
bash

aws s3 cp path/to/example.txt s3://example-bucket/
```

Commands:

```
bash

terraform init
terraform apply
```

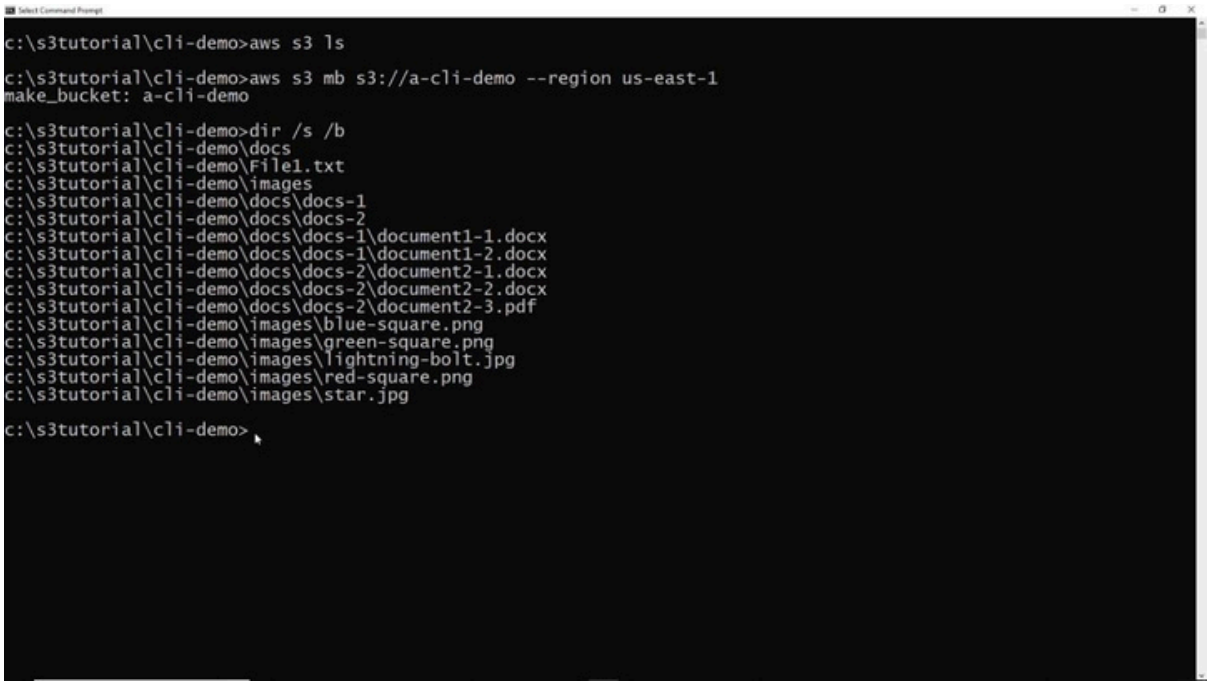


Figure 4: AWS CLI Object Commands example

Managing Permissions

Using AWS Management Console

1. Setting Bucket Permissions:
    - Navigate to the bucket.
    - Click on the "Permissions" tab.
    - Edit bucket policy or access control list (ACL).

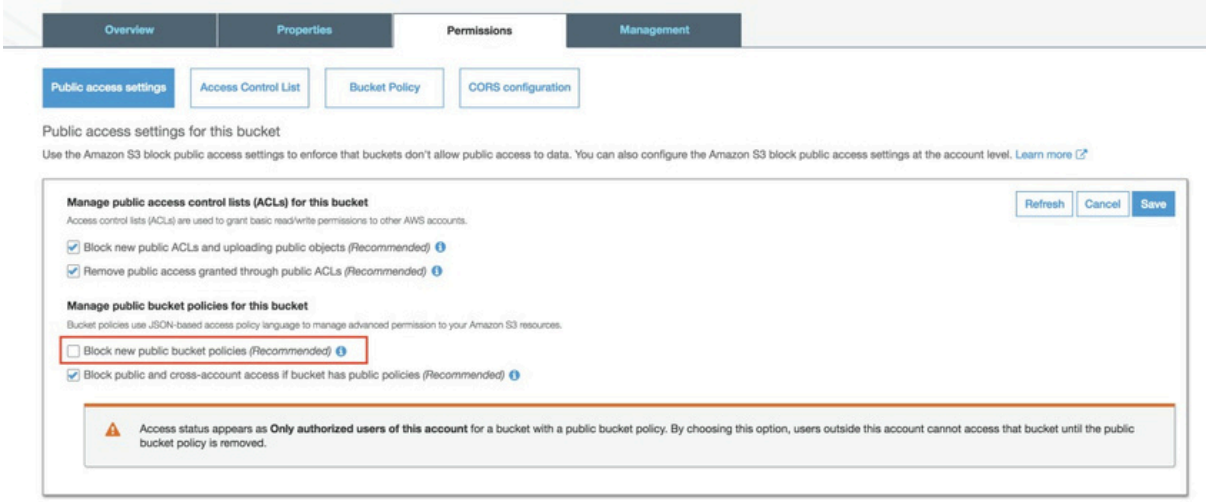


Figure 5: S3 Bucket Permissions

Using Terraform

Terraform Configuration:

```
hcl

resource "aws_s3_bucket_policy" "example_policy" {
  bucket = aws_s3_bucket.example_bucket.bucket
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect = "Allow",
        Principal = "*",
        Action = "s3:GetObject",
        Resource = "arn:aws:s3:::example-bucket/*"
      }
    ]
  })
}
```

Commands:

```
bash

terraform init
terraform apply
```

Versioning and Lifecycle Policies

Using AWS Management Console

1. Enabling Versioning:
    - Navigate to the bucket.
    - Click on the "Properties" tab.
    - Edit the "Versioning" section and enable versioning.

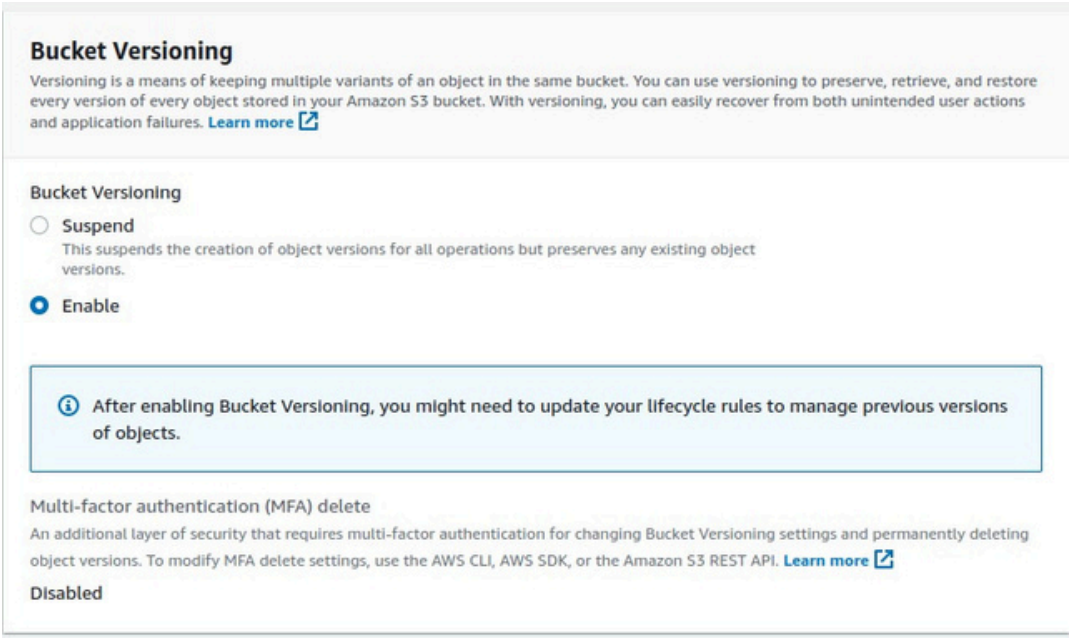


Figure 7: S3 Enable Versioning

2. Creating a Lifecycle Policy:
    - Navigate to the bucket.
    - Click on the "Management" tab.
    - Add a lifecycle rule to transition objects to different storage classes or delete them after a certain period.

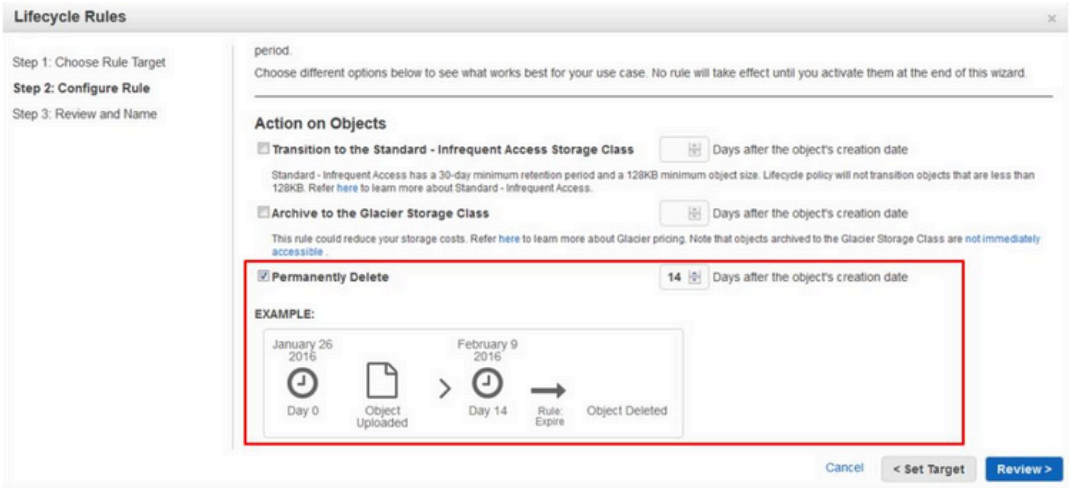


Figure 8: S3 Lifecycle Policy

Terraform Configuration:

```
hcl

resource "aws_s3_bucket" "example_bucket" {
  bucket = "example-bucket"
  acl    = "private"

  versioning {
    enabled = true
  }

  lifecycle_rule {
    id      = "log"
    enabled = true

    transition {
      days          = 30
      storage_class = "STANDARD_IA"
    }
    expiration {
    days = 365
    }
  }
}
```

Commands:

```
bash

terraform init
terraform apply
```

Cheat Sheets

AWS CLI Cheat Sheet for S3:

- Create a Bucket:

  ```
  bash

  aws s3 mb s3://example-bucket
  ```

- List Buckets:

  ```
  bash

  aws s3 ls
  ```

- Upload an Object:

  ```
  bash

  aws s3 cp example.txt s3://example-bucket/
  ```

- Enable Versioning:

```
bash

aws s3api put-bucket-versioning --bucket example-bucket --versioning-configuration Status=Enabled
```

- Set Lifecycle Policy:

```
bash

aws s3api put-bucket-lifecycle-configuration --bucket example-bucket --lifecycle-configuration file://lifecycle.json
```

Terraform Cheat Sheet for S3:

- Create S3 Bucket:

```
hcl

resource "aws_s3_bucket" "example_bucket" {
  bucket = "example-bucket"
  acl    = "private"
}
```

- Upload S3 Object:

```
hcl

resource "aws_s3_bucket_object" "example_object" {
  bucket = aws_s3_bucket.example_bucket.bucket
  key    = "example.txt"
  source = "path/to/example.txt"
  acl    = "private"
}
```

- Set Bucket Policy:

```
hcl

resource "aws_s3_bucket_policy" "example_policy" {
  bucket = aws_s3_bucket.example_bucket.bucket
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect = "Allow",
        Principal = "*",
        Action = "s3:GetObject",
        Resource = "arn:aws:s3:::example-bucket/*"
      }
    ]
  })
}
```

Real-Life Scenarios

Scenario 1: Static Website HostingUse S3 to host a static website by configuring the bucket to serve static content and enabling public access.

Scenario 2: Data Archiving Implement lifecycle policies to transition data to infrequent access or archive storage classes, reducing storage costs for less frequently accessed data.

Scenario 3: Backup and Restore Automate backups of application data to S3 and use versioning to maintain backup history. This ensures data integrity and availability.

Conclusion

Amazon S3 is a versatile and robust storage solution that can handle various storage needs, from hosting static websites to archiving data. By understanding and utilizing its features through both the AWS Management Console and Terraform, you can manage your storage needs efficiently and securely.

In the next chapter, we will explore AWS Lambda, a serverless computing service, and demonstrate how to build and deploy serverless applications.

Chapter 5: Compute Services: EC2 and Lambda

Amazon EC2 (Elastic Compute Cloud) and AWS Lambda are two primary compute services offered by AWS. EC2 provides scalable virtual servers, while Lambda offers serverless computing where you only pay for the compute time you consume. This chapter dives deep into both services, covering basic to advanced concepts, usage scenarios, and automation using Terraform.

Key Concepts

- EC2 Instances: Virtual servers in the cloud.
- AMIs : Amazon Machine Images, which are templates for launching EC2 instances.
- Security Groups: Virtual firewalls for your instances.
- Lambda Functions: Serverless functions that run your code in response to events.

Section 1: Amazon EC2

Launching an EC2 Instance

Using AWS Management Console

1. Step 1: Choose an Amazon Machine Image (AMI):
    - Navigate to the EC2 dashboard.
    - Click on "Launch Instance".
    - Select an AMI (e.g., Amazon Linux 2).

2. Step 2: Choose an Instance Type:
    - Select an instance type (e.g., t2.micro for free tier).

3. Step 3: Configure Instance Details:
    - Configure instance details like number of instances, network settings, etc.

4. Step 4: Add Storage:
    - Add storage to your instance (default is 8 GiB).

5. Step 5: Add Tags:
    - Add tags to your instance for identification.

6. Step 6: Configure Security Group:
    - Create a new security group or select an existing one.

7. Step 7: Review and Launch:
    - Review your configuration and click "Launch".
    - Choose an existing key pair or create a new one.

Using Terraform

Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "example-instance"
  }

  key_name = "my-key-pair"
}
resource "aws_security_group" "example" {

  name"example-sg"
  description = "Example security group"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {

    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Commands:

```
bash

terraform init
terraform apply
```

Outputs:

```
shell

aws_instance.example: Creating...
aws_instance.example: Creation complete after 2m3s [id=i-0123456789abcdef0]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Connecting to Your EC2 Instance

```
bash

ssh -i "my-key-pair.pem" ec2-user@<instance-public-ip>
```

Outputs:

```
shell
Last login: Fri Mar 12 18:15:42 2023 from 203.0.113.0
[ec2-user@ip-172-31-0-123 ~]$
```

Section 2: AWS Lambda

Creating a Lambda Function

Using AWS Management Console

1. Step 1: Create Function:
    - Navigate to the Lambda dashboard.
    - Click "Create function".
    - Choose "Author from scratch", enter function name, and select runtime (e.g., Node.js 14.x).

2. Step 2: Configure Function:
    - Configure function settings such as memory, timeout, and IAM role.

3. Step 3: Write Function Code:
    - Write your function code in the inline editor or upload a ZIP file.

```
javascript
```

```
exports.handler = async (event) => {
    const response = {
        statusCode: 200,
        body: JSON.stringify('Hello from Lambda!'),
    };
    return response;
};
```

4. Step 4: Test Function:
    - Create a test event and run the function to see the output.

Using Terraform

Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_lambda_function" "example" {
  filename         = "lambda_function_payload.zip"
  function_name    = "example_lambda"
  role             = aws_iam_role.lambda_exec.arn
  handler          = "index.handler"
  source_code_hash = filebase64sha256("lambda_function_payload.zip")
  runtime          = "nodejs14.x"

  environment {
    variables = {
      foo = "bar"
    }
  }
}
resource "aws_iam_role" "lambda_exec" {
  name = "lambda_exec_role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Action = "sts:AssumeRole",
        Effect = "Allow",
        Sid    = "",
        Principal = {
          Service = "lambda.amazonaws.com"
        }
      }
    ]
  })
}
```

```
resource "aws_iam_role_policy_attachment" "lambda_policy" {
  role       = aws_iam_role.lambda_exec.name
  policy_arn = "arn:aws:iam::aws:policy/service-
role/AWSLambdaBasicExecutionRole"
}
```

Commands:

```
bash

zip lambda_function_payload.zip index.js
terraform init
terraform apply
```

Outputs:

```
shell
aws_lambda_function.example: Creating...
aws_lambda_function.example: Creation complete after 1m1s
[id=example_lambda]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Invoking Lambda Function

```
bash

aws lambda invoke --function-name example_lambda output.txt
```

Outputs:

```
json

{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}
```

output.txt:

```
json

"Hello from Lambda!"
```

Cheat Sheets

EC2 CLI Commands:

- Launch an EC2 instance:

```
bash

aws ec2 run-instances --image-id ami-0c55b159cbfafe1f0 --instance-
type t2.micro --key-name my-key-pair --security-group-ids sg-
0abc12345 --subnet-id subnet-0abc12345
```

- Stop an EC2 instance:

```
bash

aws ec2 stop-instances --instance-ids i-0123456789abcdef0
```

- Terminate an EC2 instance:

```
bash

aws ec2 terminate-instances --instance-ids i-0123456789abcdef0
```

Lambda CLI Commands:

- Create a Lambda function:

```
bash
aws lambda create-function --function-name example_lambda --zip-file
fileb://lambda_function_payload.zip --handler index.handler --runtime
nodejs14.x --role arn:aws:iam::123456789012:role/lambda_exec_role
```

- Invoke a Lambda function:

```
bash

aws lambda invoke --function-name example_lambda output.txt
```

- Update Lambda function code:

```
bash

aws lambda update-function-code --function-name example_lambda --zip-
file fileb://lambda_function_payload.zip
```

Real-Life Scenarios

Scenario 1: Auto Scaling Web Application Use EC2 to host a web application and set up auto scaling to handle traffic spikes.

Scenario 2: Data Processing with Lambda Create Lambda functions to process data from S3, transforming and storing the results in DynamoDB.

Scenario 3: Serverless Microservices Develop a microservices architecture using Lambda for computation and API Gateway for exposure.

Conclusion

EC2 and Lambda provide flexible and scalable compute resources for various use cases. By leveraging the AWS Management Console for GUI-based management and Terraform for automation, you can efficiently deploy, manage, and scale your applications. Whether you need persistent virtual servers or event-driven serverless functions, AWS has the tools to meet your compute needs.

In the next chapter, we will explore AWS's monitoring and logging services to ensure your applications run smoothly and securely.

Chapter 6: Containers and Orchestration

Containers and orchestration tools are essential components in modern DevOps, enabling consistent and scalable deployment of applications. AWS offers services like Amazon ECS (Elastic Container Service) and Amazon EKS (Elastic Kubernetes Service) for container management and orchestration. This chapter explores these services in detail, providing examples, explanations, cheat sheets, and architecture diagrams.

Key Concepts

- Containers: Lightweight, standalone, and executable packages that include everything needed to run a piece of software.
- Orchestration: Automated configuration, coordination, and management of computer systems, applications, and services.
- Amazon ECS : A fully managed container orchestration service.
- Amazon EKS : A managed Kubernetes service that makes it easy to run Kubernetes on AWS.

Section 1: Amazon Elastic Container Service (ECS)

Creating an ECS Cluster

Using AWS Management Console

1. Step 1: Create a Cluster:
    - Navigate to the ECS dashboard.
    - Click "Clusters" in the left navigation pane.
    - Click the "Create Cluster" button.

2. Step 2: Configure Cluster:
    - Choose the cluster template (e.g., EC2 Linux + Networking).
    - Enter cluster name, instance type, and number of instances.

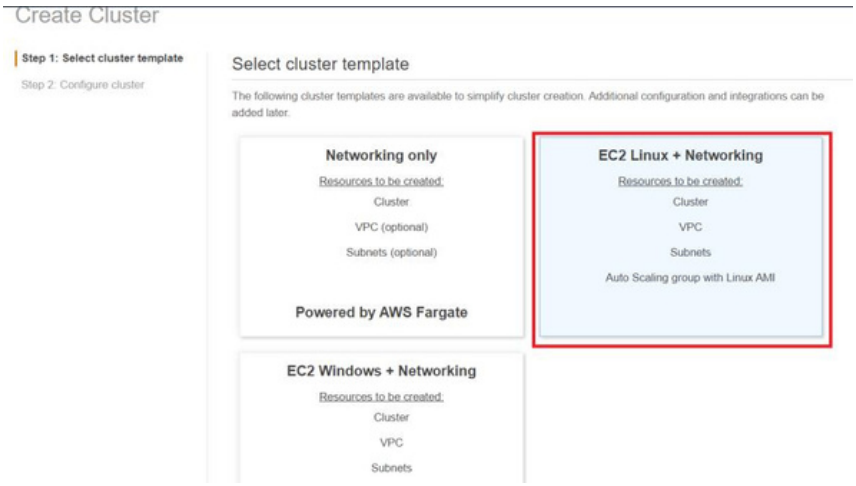


Figure 1: Creating ECS

3. Step 3: Review and Create:
    - Review your settings and click "Create".

Using Terraform

Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_ecs_cluster" "example" {
  name = "example-cluster"
}

resource "aws_ecs_task_definition" "example" {
  family                   = "example-task"
  container_definitions    = jsonencode([
    {
      name      = "nginx",
      image     = "nginx",
      cpu       = 256,
      memory    = 512,
      essential = true,
      portMappings = [
        {
          containerPort = 80,
          hostPort      = 80
        }
      ]
    }
  ])
  requires_compatibilities = ["EC2"]
  network_mode             = "bridge"
}
resource "aws_ecs_service" "example" {

  name            = "example-service"
  cluster         = aws_ecs_cluster.example.id
  task_definition = aws_ecs_task_definition.example.arn
  desired_count   = 2
  launch_type     = "EC2"
}
```

Commands:

```
bash

terraform init
terraform apply
```

Outputs:

```
shell aws_ecs_cluster.example: Creating...
aws_ecs_cluster.example: Creation complete after 2m3s [id=arn:aws:ecs:us-
west-2:123456789012:cluster/example-cluster]
aws_ecs_task_definition.example: Creating...
aws_ecs_task_definition.example: Creation complete after 1m5s [id=example-
task:1]
aws_ecs_service.example: Creating...
aws_ecs_service.example: Creation complete after 1m0s [id=arn:aws:ecs:us-
west-2:123456789012:service/example-service]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Running Tasks in ECS

```
bash

aws ecs run-task --cluster example-cluster --task-definition example-task
```

Outputs:

```
json

{
    "tasks": [
        {
            "taskArn": "arn:aws:ecs:us-west-2:123456789012:task/abc123",
            "clusterArn": "arn:aws:ecs:us-west-
2:123456789012:cluster/example-cluster",
            ...
        }
    ]
}
```

Section 2: Amazon Elastic Kubernetes Service (EKS)

Creating an EKS Cluster

Using AWS Management Console

1. Step 1: Create a Cluster:
    - Navigate to the EKS dashboard.
    - Click "Create cluster".

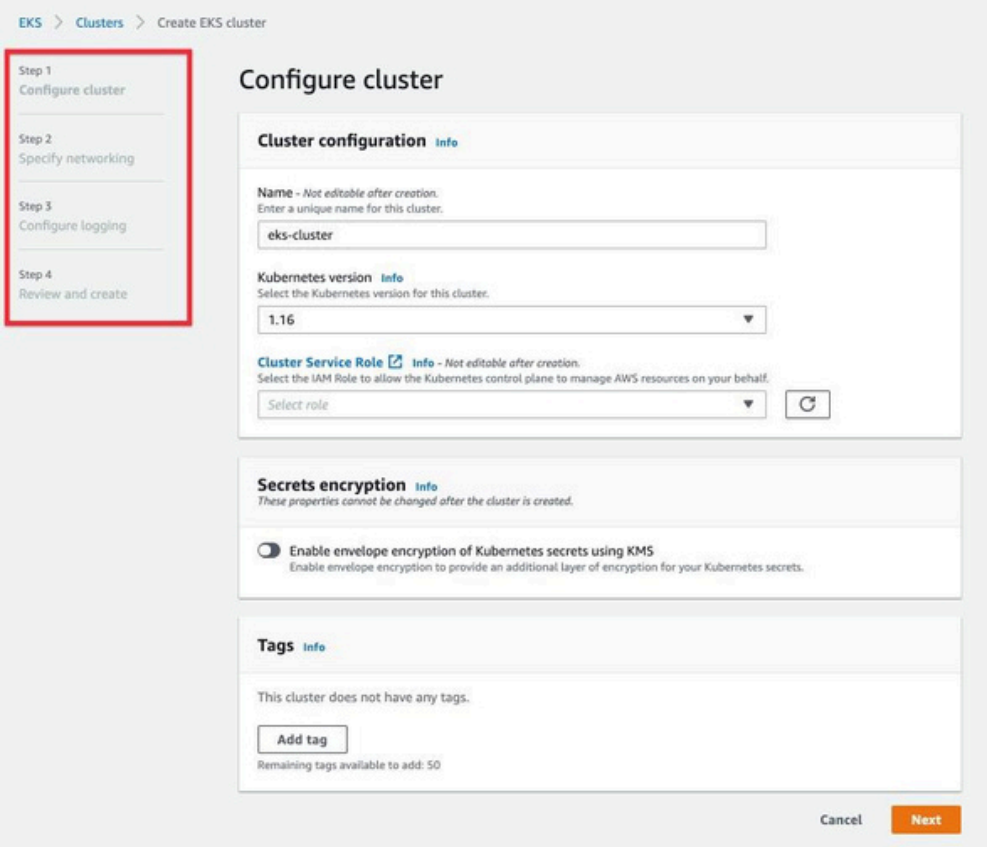


Figure 2: Configuring a cluster

2. Step 2: Configure Cluster:
    - Enter cluster name and select Kubernetes version.
    - Configure networking settings.

3. Step 3: Create Node Group:
    - Configure node group settings such as instance type and desired size.

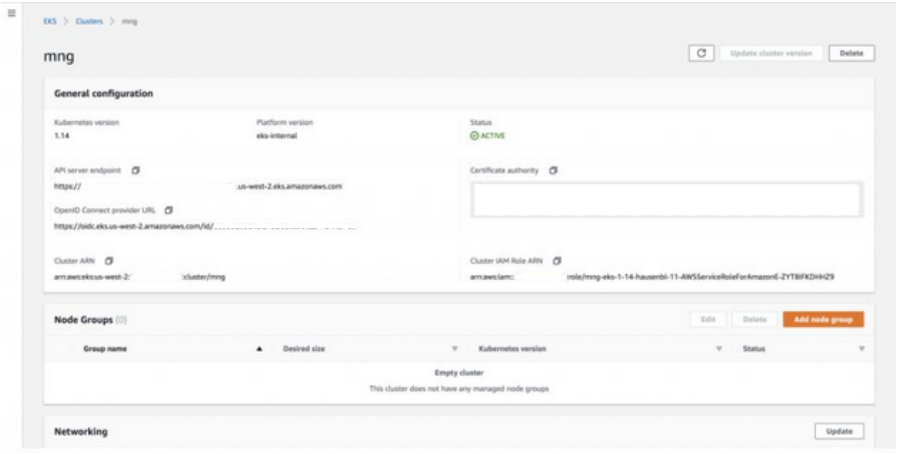


Figure 3: S3 Node Group

4. Step 4: Review and Create:
    - Review your settings and click "Create".

Using Terraform

Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_eks_cluster" "example" {
  name     = "example-eks-cluster"
  role_arn = aws_iam_role.eks_cluster_role.arn

  vpc_config {
    subnet_ids = [aws_subnet.main.id]
  }
}
resource "aws_iam_role" "eks_cluster_role" {
  name = "eks_cluster_role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Action = "sts:AssumeRole",
        Effect = "Allow",
        Principal = {
          Service = "eks.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment"
"EKS_Cluster_AmazonEKSClusterPolicy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
  role       = aws_iam_role.eks_cluster_role.name
}
```

Commands:

```
bash

terraform init
terraform apply
```

Outputs:

```
shell

aws_eks_cluster.example: Creating...
aws_eks_cluster.example: Creation complete after 10m1s [id=example-eks-
cluster]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Deploying Applications to EKS

```
bash

kubectl apply -f https://k8s.io/examples/application/deployment.yaml
```

Outputs:

```
shell

deployment.apps/nginx-deployment created
```

Monitoring and Scaling

```
bash
kubectl get nodes
kubectl get pods
kubectl scale deployment nginx-deployment --replicas=3
```

Outputs:

```
shell NAME
pod/nginx-deployment-5c655cd66b-xxxxx
pod/nginx-deployment-5c655cd66b-yyyyy
pod/nginx-deployment-5c655cd66b-zzzzz
```

READY	STATUS	RESTARTS	AGE
1/1	Running	0	2m
1/1	Running	0	2m
1/1	Running	0	2m

Cheat Sheets

ECS CLI Commands:

- Create a Cluster:

  ```
  bash

  aws ecs create-cluster --cluster-name example-cluster
  ```

- Register a Task Definition:

```
bash

aws ecs register-task-definition --cli-input-json file://task-
definition.json
```

- Run a Task:

```
bash

aws ecs run-task --cluster example-cluster --task-definition example-
task
```

EKS CLI Commands:

- Create a Cluster:

```
bash

eksctl create cluster --name example-eks-cluster --region us-west-2
```

- Get Nodes:

```
bash

kubectl get nodes
```

- Deploy an Application:

```
bash

kubectl apply -f deployment.yaml
```

Real-Life Scenarios

Scenario 1: Microservices with ECS Deploy a microservices application with multiple services using ECS and integrate with CloudWatch for monitoring.

Scenario 2: Kubernetes with EKS Set up a CI/CD pipeline using Jenkins to deploy applications to an EKS cluster, using Helm for managing Kubernetes applications.

Scenario 3: Serverless with Lambda and Fargate Use Lambda functions for event-driven compute and Fargate for running containers without managing servers.

Conclusion

Amazon ECS and EKS provide robust solutions for container orchestration, making it easier to deploy, manage, and scale containerized applications. By leveraging these services along with Terraform for automation, you can build efficient and scalable architectures. In the next chapter, we will explore monitoring and logging in AWS to ensure your applications run smoothly and securely.

Chapter 7: Continuous Integration and Continuous Deployment (CI/CD)

Continuous Integration (CI) and Continuous Deployment (CD) are essential practices in modern DevOps that enable development teams to deliver code changes more frequently and reliably. In this chapter, we will explore how to set up and manage CI/CD pipelines using AWS services, Jenkins, and Terraform.

Key Concepts

- Continuous Integration (CI): The practice of automatically integrating code changes into a shared repository multiple times a day.
- Continuous Deployment (CD): The practice of automatically deploying code changes to production environments after passing CI stages.

Section 1: Setting Up CI/CD with AWS CodePipeline

AWS CodePipeline is a fully managed continuous integration and continuous delivery service that helps you automate your release pipelines for fast and reliable application and infrastructure updates.

Creating a Simple CI/CD Pipeline

Using AWS Management Console

1. Step 1: Create a Pipeline:
    - Navigate to the AWS CodePipeline console.
    - Click "Create pipeline".

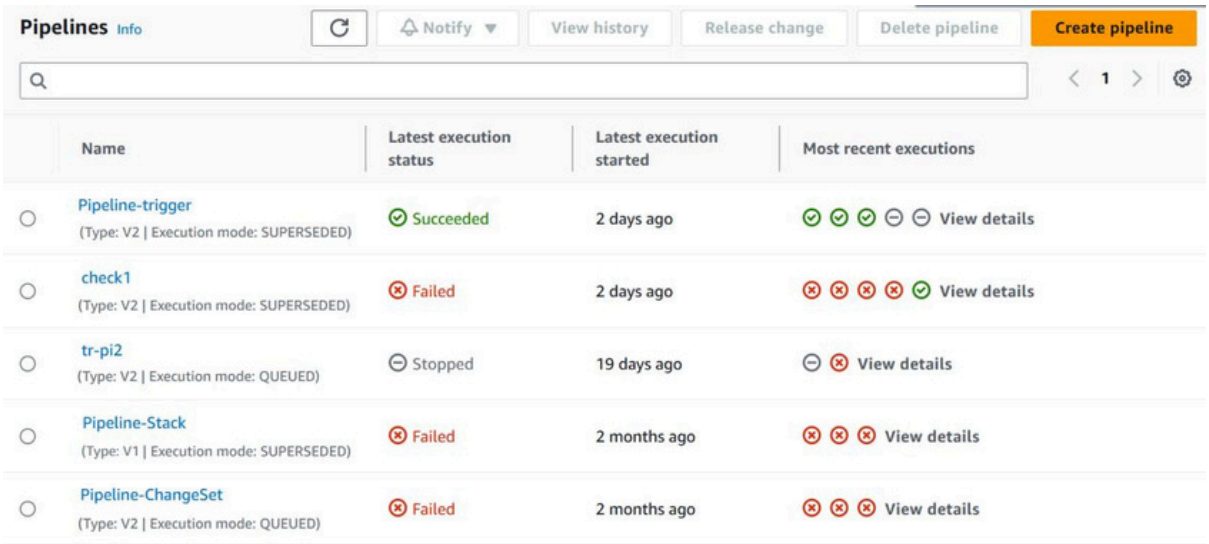


Figure 1: Creating a pipeline

2. Step 2: Configure Pipeline Settings:
    - Enter pipeline name and role name.
    - Choose "New service role" or an existing role.
3. Step 3: Add Source Stage:
    - Select the source provider (e.g., GitHub).
    - Enter the repository details.
4. Step 4: Add Build Stage:
    - Choose AWS CodeBuild as the build provider.
    - Configure the build project details.
5. Step 5: Add Deploy Stage:
    - Select the deploy provider (e.g., AWS Elastic Beanstalk).
    - Configure the deployment settings.
6. Step 6: Review and Create Pipeline:
    - Review your settings and click "Create pipeline".

Using Terraform

Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_codepipeline" "example" {
  name     = "example-pipeline"
  role_arn = aws_iam_role.codepipeline_role.arn

  artifact_store {
    location = aws_s3_bucket.codepipeline_bucket.bucket
    type     = "S3"
  }
  stage {

    name = "Source"
    action {
    name                = "Source"
    category            = "Source"
    owner               = "ThirdParty"
    provider            = "GitHub"
    version             = "1"
      output_artifacts = ["source_output"]

      configuration = {
        Owner = "your-github-username"
        Repo   = "your-repo-name"
        Branch = "main"
        OAuthToken = var.github_oauth_token
      }
```

```
    }
  }

  stage {
    name = "Build"

    action {
      name             = "Build"
      category         = "Build"
      owner            = "AWS"
      provider         = "CodeBuild"
      input_artifacts = ["source_output"]
      output_artifacts = ["build_output"]

      configuration = {
        ProjectName = aws_codebuild_project.example.name
      }
    }
  }

  stage {
    name = "Deploy"

    action {
      name             = "Deploy"
      category         = "Deploy"
      owner            = "AWS"
      provider         = "ElasticBeanstalk"
      input_artifacts = ["build_output"]

      configuration = {

        ApplicationName = aws_elastic_beanstalk_application.example.name
        EnvironmentName = aws_elastic_beanstalk_environment.example.name
      }
    }
  }
}

resource "aws_iam_role" "codepipeline_role" {
  name = "codepipeline_role"

  assume_role_policy = jsonencode({

    Version = "2012-10-17",
    Statement = [
      {
        Action = "sts:AssumeRole",
        Effect = "Allow",
        Principal = {
          Service = "codepipeline.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_s3_bucket" "codepipeline_bucket" {
  bucket = "codepipeline-bucket"
}

resource "aws_codebuild_project" "example" {
  name          = "example-build"
```

```
 service_role = aws_iam_role.codebuild_role.arn
  source {
    type = "CODEPIPELINE"
  }

  artifacts {
    type = "CODEPIPELINE"
  }

  environment {
    compute_type                = "BUILD_GENERAL1_SMALL"
    image                       = "aws/codebuild/standard:4.0"
    type                        = "LINUX_CONTAINER"
  }
}
resource "aws_elastic_beanstalk_application" "example" {
  name = "example-app"
}

resource "aws_elastic_beanstalk_environment" "example" {
  name                = "example-env"
  application         = aws_elastic_beanstalk_application.example.name
  solution_stack_name = "64bit Amazon Linux 2 v5.4.4 running Node.js 14"
}
```

Commands:

```
bash

terraform init
terraform apply
```

Outputs:

```
shell

aws_codepipeline.example: Creating...
aws_codepipeline.example: Creation complete after 1m30s [id=example-
pipeline]
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
```

Section 2: Jenkins Integration with AWS

Jenkins is an open-source automation server that helps automate parts of software development related to building, testing, and deploying.

Installing Jenkins on AWS EC2

Using AWS Management Console

1. Step 1: Launch an EC2 Instance:
    - Navigate to the EC2 dashboard.
    - Click "Launch Instance".
2. Step 2: Choose an Amazon Machine Image (AMI):
    - Select an Amazon Linux 2 AMI.
3. Step 3: Configure Instance Details:
    - Choose the instance type (e.g., t2.micro).
    - Configure instance settings.
4. Step 4: Add Storage and Tags:
    - Configure storage and add tags as needed.
5. Step 5: Configure Security Group:
    - Add rules to allow HTTP and SSH access.
6. Step 6: Review and Launch:
    - Review your settings and click "Launch".

Using Terraform

Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_instance" "jenkins" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  key_name      = "my-key"
  tags = {
    Name = "Jenkins-Server"
  }
}
resource "aws_security_group" "jenkins_sg" {
  name        = "jenkins_sg"
  description = "Allow HTTP and SSH"
```

```
  ingress {
    from_port   =  22  =
    to_port     22     =
    protocol    "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_key_pair" "my-key" {
  key_name   = "my-key"
  public_key = file("~/.ssh/id_rsa.pub")
}
```

Commands:

```
bash

terraform init
terraform apply
```

Outputs:

```
shell

aws_instance.jenkins: Creating...
aws_instance.jenkins: Still creating... [10s elapsed]
aws_instance.jenkins: Creation complete after 1m20s [id=i-
0a123b4c567d8e9f0]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Setting Up Jenkins

1. Install Jenkins:

```
bash
sudo yum update -y
sudo yum install java-1.8.0-openjdk -y
sudo wget -O /etc/yum.repos.d/jenkins.repo
https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io.key
sudo yum install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

2. Access Jenkins:
    - Open a web browser and navigate to `http://<your -ec2-public-dns>:8080`.

Section 3: Creating a CI/CD Pipeline with Jenkins and AWS

Using Jenkins with AWS CodeDeploy

1. Step 1: Install Plugins:
    - Install "AWS CodeDeploy Plugin" from Jenkins Plugin Manager.
2. Step 2: Configure AWS Credentials:
    - Add AWS credentials in Jenkins.

3. Step 3: Create a New Job:
    - Create a new freestyle project.
    - Configure source code management (e.g., Git).
4. Step 4: Add Build Steps:
    - Add "Execute shell" build step.
    - Add "AWS CodeDeploy" post-build action.
5 Step 5: Configure Deployment Settings:
    - Enter CodeDeploy application name and deployment group.
. Step 6: Build and Deploy:
    - Trigger a build and verify deployment.
6

.

Section 4: Cheat Sheets and Quick Reference

AWS CodePipeline CLI Commands:

```
bash

# Create a pipeline
aws codepipeline create-pipeline --cli-input-json file://pipeline.json
# Get pipeline details
aws codepipeline get-pipeline --name my-pipeline
# Start a pipeline
aws codepipeline start-pipeline-execution --name my-pipeline
```

Jenkins CLI Commands:

```
bash

# Install plugin
java -jar jenkins-cli.jar -s http://localhost:8080/ install-plugin aws-
codedeploy

# List jobs
java -jar jenkins-cli.jar -s http://localhost:8080/ list-jobs
# Build job
java -jar jenkins-cli.jar -s http://localhost:8080/ build my-job
```

This chapter provided a comprehensive guide to setting up and managing CI/CD pipelines with AWS CodePipeline and Jenkins, using both GUI and Terraform. By following the steps and examples provided, you can streamline your development and deployment processes, ensuring faster and more reliable software delivery.

Chapter 8: Infrastructure as Code (IaC) with Terraform

Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools. Terraform is a popular IaC tool that allows you to define, provision, and manage infrastructure across various cloud providers.

Key Concepts

- Infrastructure as Code (IaC): Managing and provisioning infrastructure through code.
- Terraform: An open-source IaC tool that enables you to define and provision infrastructure using a high-level configuration language.

Section 1: Introduction to Terraform

Terraform allows you to define infrastructure as code using the HashiCorp Configuration Language (HCL). With Terraform, you can manage resources across multiple cloud providers (such as AWS, Azure, and Google Cloud) and services (such as GitHub).

Why Use Terraform?

- Consistency: Ensure consistent environments across development, testing, and production.
- Version Control: Infrastructure code can be versioned and reviewed just like application code.
- Automation: Automate infrastructure provisioning and management.
- Scalability: Easily scale infrastructure up or down as needed.

Section 2: Setting Up Terraform

Before we start using Terraform, we need to install it and configure the necessary authentication for AWS.

Installing Terraform

1. Download Terraform:
    - Visit the Terraform download page.
    - Choose the appropriate package for your operating system.
2. Install Terraform:
    - Extract the downloaded package.
    - Move the `terraform` binary to a directory included in your system's `PATH`.

```
bash

sudo mv terraform /usr/local/bin/
```

3. Verify Installation:
    - Run the following command to verify that Terraform is installed correctly.

```
bash

terraform version
```

Configuring AWS Credentials

1. Create an IAM User:
    - Sign in to the AWS Management Console.
    - Navigate to the IAM console.
    - Create a new IAM user with programmatic access.
    - Attach the "AdministratorAccess" policy to the user.

2. Configure AWS CLI:
    - Install the AWS CLI.
    - Configure the AWS CLI with your IAM user's access key and secret key.

```
bash

aws configure
```

Section 3: Writing Your First Terraform Configuration

Let's start with a simple example: provisioning an AWS EC2 instance.

Terraform Configuration

1. Create a Directory:
    - Create a new directory for your Terraform configuration files.

```
bash

mkdir my-terraform-project
cd my-terraform-project
```

2. Write the Configuration File:
    - Create a file named `main.tf` and add the following configuration to provision an EC2 instance.

```
hcl

provider "aws" {
  region = "us-west-2"
}
```

```
resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  tags = {

    Name = "ExampleInstance"
  }
}
```

Initializing Terraform

1. Initialize the Directory:
    - Run the following command to initialize the directory. This will download the necessary provider plugins.

```
bash

terraform init
```

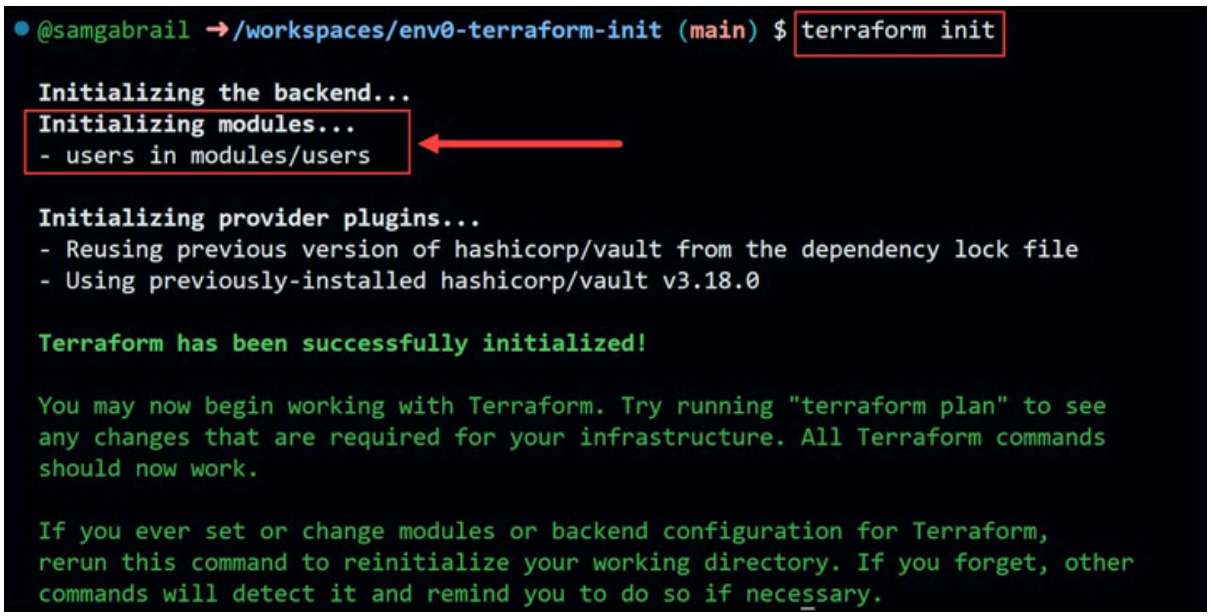


Figure 1: Example of Terraform init

Section 4: Applying the Configuration

Running Terraform Apply

1. Plan the Changes:
    - Run the following command to see what changes Terraform will make.

```
bash

terraform plan
```

Figure 2: Example of Terraform plan with changes displaced

2. Apply the Changes:
    - Run the following command to apply the configuration and provision the resources.

```
bash

terraform apply
```

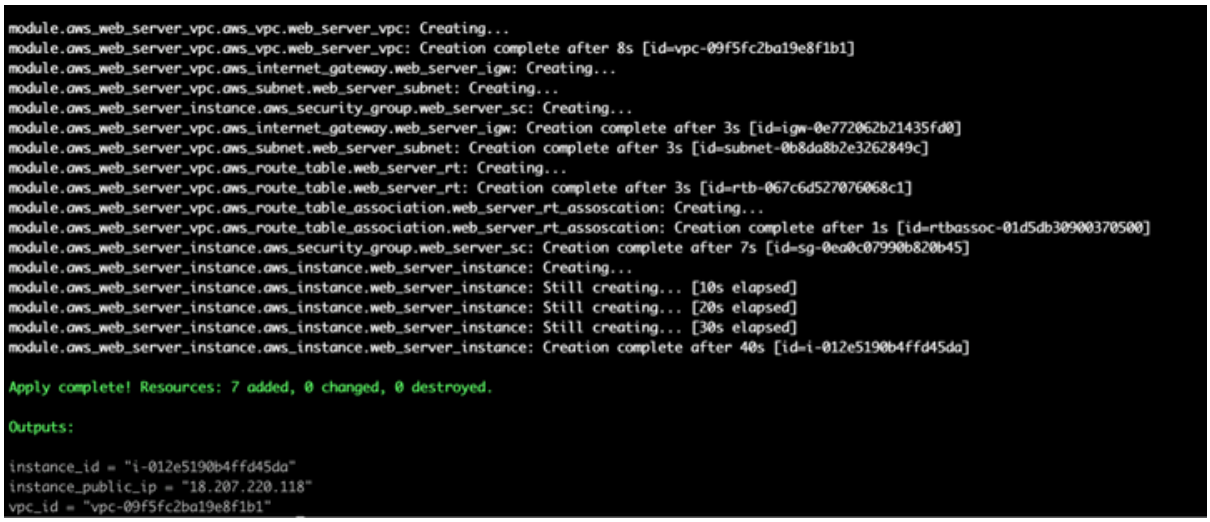


Figure 3: Example of Terraform apply

Verifying the Infrastructure

1. Check AWS Console:
    - Go to the EC2 dashboard in the AWS Management Console to verify that the instance has been created.

2. SSH into the Instance:
    - SSH into the EC2 instance using the key pair you specified.

```
bash

ssh -i "your-key-pair.pem" ec2-user@<your-instance-public-dns>
```

Section 5: Managing and Destroying Infrastructure

Managing Infrastructure

1. Updating Configuration:
    - Modify the `main.tf` file to change the instance type.

```
hcl

resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"
```

```
  instance_type = "t3.micro" # Changed instance type

  tags = {
    Name = "ExampleInstance"
  }
}
```

2. Apply the Changes:
   - Run `terraform apply` to update the infrastructure.

```
bash

terraform apply
```

Destroying Infrastructure

1. Destroy Resources:
   - Run the following command to destroy the resources managed by your configuration.

```
bash

terraform destroy
```

Section 6: Cheat Sheets and Quick Reference

Terraform CLI Commands:

```
bash

# Initialize the directory
terraform init
# Plan the changes
terraform plan
# Apply the changes
terraform apply
# Destroy the infrastructure
terraform destroy
# Show the current state
terraform show
# Refresh the state file
terraform refresh
```

Common Terraform Configurations:

```
hcl

# Provider Configuration
provider "aws" {
  region = "us-west-2"
}
```

```
# EC2 Instance
resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
}

# S3 Bucket
resource "aws_s3_bucket" "example" {
  bucket = "my-example-bucket"
  acl    = "private"
}
```

Section 7: Real-Life Case Studies

Case Study 1: Migrating Legacy Infrastructure to AWS Using Terraform

- Challenge: A company needed to migrate their legacy on-premise infrastructure to AWS.
- Solution: Using Terraform, they defined their infrastructure as code, allowing for a smooth and automated migration process.
- Outcome: The migration was completed successfully with minimal downtime, and the new infrastructure is now managed entirely through Terraform.

Case Study 2: Implementing CI/CD Pipelines with Terraform and Jenkins

- Challenge: A development team needed to automate their deployment process.
- Solution: They used Terraform to provision Jenkins servers and AWS resources, integrating them into a CI/CD pipeline.
- Outcome: The team now has an automated, reliable deployment pipeline that reduces manual effort and errors.

Section 8: Future Trends in IaC

As the field of DevOps continues to evolve, several trends are emerging in the realm of Infrastructure as Code:

- Multi-Cloud Management: Tools like Terraform are increasingly being used to manage resources across multiple cloud providers.
- Policy as Code: Integrating policies directly into IaC tools to enforce compliance and security standards.
- Serverless Infrastructure: Expanding IaC capabilities to manage serverless architectures and services.

This chapter provided a comprehensive guide to using Terraform for Infrastructure as Code, with practical examples, cheat sheets, and real-life case studies. By following the steps and best practices outlined here, you can effectively manage your infrastructure in a consistent, automated, and scalable manner.

Chapter 9: Monitoring and Logging

Monitoring and logging are critical components of any DevOps practice. They enable you to gain insights into your systems' performance, identify and troubleshoot issues, and ensure that your infrastructure runs smoothly. In AWS, several services help you achieve comprehensive monitoring and logging, including Amazon CloudWatch, AWS CloudTrail, and Amazon Elasticsearch Service.

Key Concepts

- Monitoring: Observing the performance and health of your infrastructure and applications.
- Logging: Recording events that happen within your infrastructure and applications.
- Amazon CloudWatch: A monitoring and observability service.
- AWS CloudTrail: A service that enables governance, compliance, and operational and risk auditing of your AWS account.
- Amazon Elasticsearch Service (Amazon ES): A managed service that makes it easy to deploy, operate, and scale Elasticsearch for log analytics.

Section 1: Introduction to Monitoring and Logging

Monitoring and logging are essential for maintaining the reliability, availability, and performance of your AWS resources. They help you detect issues before they impact your users and provide insights into the root causes of problems.

Section 2: Setting Up Amazon CloudWatch

Amazon CloudWatch is a powerful monitoring service that provides data and actionable insights to monitor your applications, understand and respond to system-wide performance changes, and optimize resource utilization.

Setting Up CloudWatch

1. Creating CloudWatch Alarms:
    - Go to the Amazon CloudWatch console.
    - ChooseAlarms from the navigation pane.
    - Click Create Alarm.
    - Select a metric to monitor (e.g., CPU utilization of an EC2 instance).
    - Configure the alarm conditions and actions.

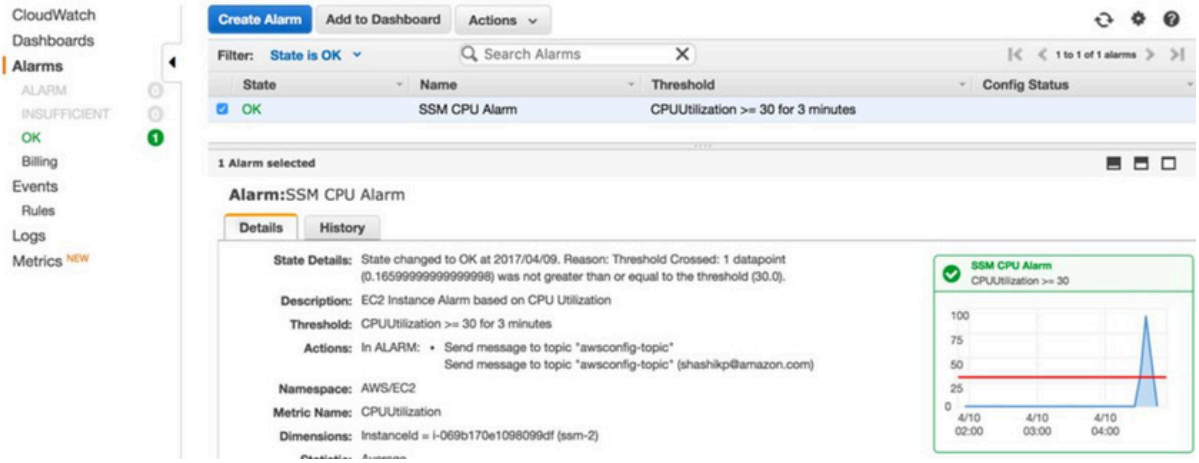


Figure 1: AWS CloudWatch Alarms

2. Setting Up CloudWatch Logs:
    - Go to the Amazon CloudWatch console.
    - ChooseLogs from the navigation pane.
    - Click Create Log Group and specify a name.
    - Click Create Log Stream within the log group and specify a name.
3. Sending Logs to CloudWatch:
    - Install the CloudWatch Logs agent on your EC2 instances.
    - Configure the agent to send log data to CloudWatch.

```
bash

sudo yum install -y awslogs
sudo service awslogs start
```

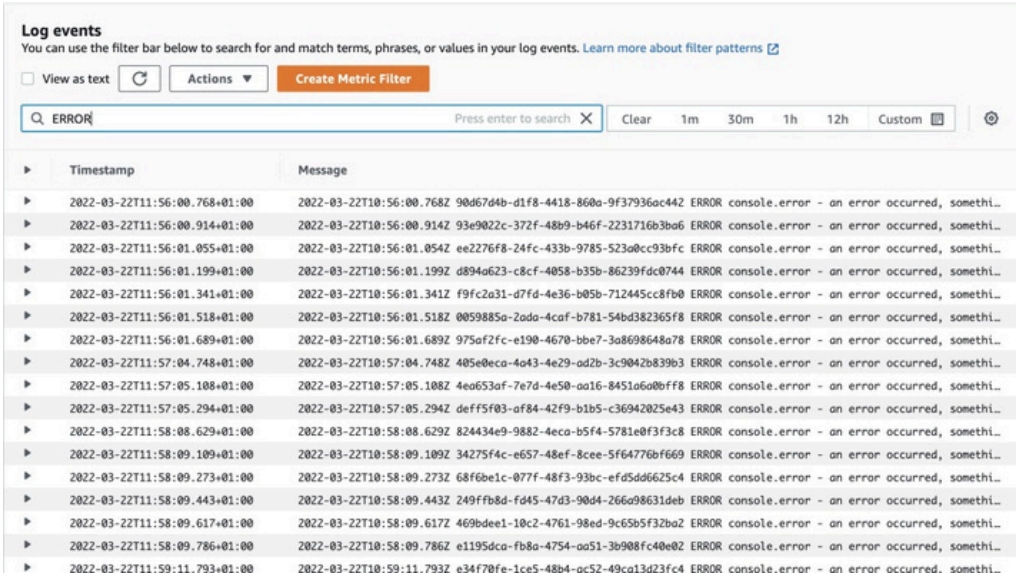


Figure 2: AWS CloudWatch Logs

Terraform Configuration

You can automate the setup of CloudWatch Alarms using Terraform.

1. Create a CloudWatch Alarm:

```
hcl
resource "aws_cloudwatch_metric_alarm" "cpu_utilization_alarm" {

  alarm_name                =    "high_cpu_utilization"    =
  comparison_operator       "GreaterThanThreshold"  =  "2"  =
  evaluation_periods        "CPUUtilization"  =  "AWS/EC2"  =
  metric_name               "300" = "Average" = "80" = "This
  namespace     period      metric monitors EC2 CPU = true =
  statistic  threshold      [aws_sns_topic.sns.arn]
  alarm_description


utilization"
  actions_enabled
  alarm_actions
  dimensions = {
    InstanceId = aws_instance.example.id
  }
}
```

2. Create CloudWatch Logs:

```
hcl

resource "aws_cloudwatch_log_group" "example" {
  name              = "/aws/lambda/example"
  retention_in_days = 14
}
resource "aws_cloudwatch_log_stream" "example" {
  name           = "example-stream"
  log_group_name = aws_cloudwatch_log_group.example.name
}
```

Section 3: Using AWS CloudTrail for Auditing

AWS CloudTrail records AWS API calls for your account and delivers log files to an Amazon S3 bucket. CloudTrail provides visibility into user activity by recording actions taken on your account.

Setting Up CloudTrail

1. Create a CloudTrail:
    - Go to the AWS CloudTrail console.
    - Click Create Trail .
    - Specify the trail name and configure settings such as logging to an S3 bucket.

2. Enable Logging:
    - Choose whether to log events for all regions or specific regions.
    - Configure additional settings, such as management events and data events.

Terraform Configuration

You can also automate CloudTrail setup using Terraform.

1. Create a CloudTrail:

```
hcl
resource "aws_cloudtrail" "example" {

  name                          = "example"
  s3_bucket_name                = aws_s3_bucket.example.bucket
  include_global_service_events = true
  is_multi_region_trail         =  true
  enable_log_file_validation    =  true
  cloud_watch_logs_group_arn    =
"${aws_cloudwatch_log_group.example.arn}:*"
  cloud_watch_logs_role_arn     = aws_iam_role.example.arn
}
```

Section 4: Analyzing Logs with Amazon Elasticsearch Service

Amazon Elasticsearch Service (Amazon ES) allows you to search, analyze, and visualize log data in real-time.

Setting Up Amazon ES

1. Create an Amazon ES Domain:
    - Go to the Amazon ES console.
    - Click Create a New Domain.
    - Configure the domain settings, including instance type and storage.

2. Configure Log Ingestion:
    - Use Logstash or the Amazon Kinesis Data Firehose to ingest logs into Amazon ES.

Terraform Configuration

1. Create an Amazon ES Domain:

```
hcl

resource "aws_elasticsearch_domain" "example" {
  domain_name           = "example"
  elasticsearch_version = "7.10"

  cluster_config {
    instance_type = "m5.large.elasticsearch"
  }

  ebs_options {
    ebs_enabled = true
    volume_size = 10
  }
}
```

Section 5: Cheat Sheets and Quick Reference

CloudWatch CLI Commands:

```
bash

# List metrics
aws cloudwatch list-metrics
# Get metric statistics
aws cloudwatch get-metric-statistics --metric-name CPUUtilization --
namespace AWS/EC2 --statistics Average --period 300 --start-time 2021-01-
01T00:00:00Z --end-time 2021-01-02T00:00:00Z
# Put metric data
aws cloudwatch put-metric-data --metric-name MyCustomMetric --namespace
MyNamespace --value 1
```

CloudTrail CLI Commands:

bash

```
# Create a trail
aws cloudtrail create-trail --name myTrail --s3-bucket-name myBucket
# Start logging
aws cloudtrail start-logging --name myTrail
# Stop logging
aws cloudtrail stop-logging --name myTrail
# Describe trails
aws cloudtrail describe-trails
```

Elasticsearch CLI Commands:

bash

```
# Create an index
curl -X PUT "https://search-my-domain.us-west-2.es.amazonaws.com/my-index"
# Index a document
curl -X POST "https://search-my-domain.us-west-2.es.amazonaws.com/my-
index/_doc/1" -H 'Content-Type: application/json' -d'
{
  "name": "John Doe",
  "age": 30,
  "address": "123 Main St"
}
'

# Search for a document
curl -X GET "https://search-my-domain.us-west-2.es.amazonaws.com/my-
index/_search?q=name:John"
```

Section 6: Real-Life Case Studies

Case Study 1: Monitoring a High-Traffic Web Application

- Challenge: A company needed to ensure their high-traffic web application was running smoothly.
- Solution: They used Amazon CloudWatch to monitor application performance and set up alarms to notify the team of any issues.
- Outcome: The company was able to proactively address issues before they impacted users, maintaining high availability and performance.

Case Study 2: Auditing AWS Account Activity

- Challenge: A financial institution needed to audit AWS account activity for compliance purposes.
- Solution: They used AWS CloudTrail to log all API calls and Amazon ES to analyze and visualize the logs.
- Outcome: The institution was able to meet compliance requirements and gain valuable insights into account activity.

Section 7: Future Trends in Monitoring and Logging

As monitoring and logging evolve, several trends are emerging:

- AI and Machine Learning: Leveraging AI/ML to analyze log data and detect anomalies.
- Unified Monitoring: Integrating monitoring across multiple services and platforms for a unified view.
- Serverless Monitoring: Developing tools and techniques to monitor serverless architectures.

This chapter provided a comprehensive guide to monitoring and logging in AWS, with practical examples, cheat sheets, and real-life case studies. By following the steps and best practices outlined here, you can ensure your infrastructure runs smoothly and efficiently.

Chapter 10: Security Best Practices

Ensuring the security of your AWS environment is paramount. This chapter delves into security best practices for AWS DevOps, providing detailed explanations, coded examples, architecture diagrams, and GUI steps. It covers essential aspects such as IAM roles and policies, VPC security, data encryption, and compliance management.

Key Concepts

- Identity and Access Management (IAM) : Control access to AWS services and resources.
- Virtual Private Cloud (VPC) : Securely isolate your AWS resources.
- Data Encryption : Protect data at rest and in transit.
- Compliance Management : Ensure your AWS environment meets regulatory requirements.

Section 1: Identity and Access Management (IAM)

IAM is crucial for managing access to AWS resources. Properly configuring IAM roles, policies, and multi-factor authentication (MFA) enhances the security of your AWS environment.

IAM Best Practices

1. Use IAM Roles:
    - Create roles with minimal permissions.
    - Assign roles to EC2 instances, Lambda functions, and other AWS services.
2. Enable MFA:
    - Require MFA for all users, especially those with administrative privileges.
3. Rotate Access Keys:
    - Regularly rotate access keys to minimize the risk of compromised credentials.

Setting Up IAM Roles and Policies

1. Create an IAM Role:
    - Go to the IAM console.
    - ChooseRoles and clickCreate role.
    - Select the AWS service that will use this role (e.g., EC2).
    - Attach the necessary policies.
    - Review and create the role.

2. Create an IAM Policy:
    - Go to the IAM console.
    - ChoosePolic iesand click Create policy.
    - Define the policy using the JSON editor.

```
json

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::example_bucket"
    },
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::example_bucket/*"
    }
  ]
}
```

Terraform Configuration

You can automate IAM setup using Terraform.

1. Create an IAM Role:

```
hcl

resource "aws_iam_role" "example" {
  name               = "example_role"
  assume_role_policy =
data.aws_iam_policy_document.assume_role_policy.json
}

data "aws_iam_policy_document" "assume_role_policy" {
  statement {
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
       identifiers = ["ec2.amazonaws.com"]
    }
  }
}
```

2. Create an IAM Policy:

```
hcl

resource "aws_iam_policy" "example" {
  name   = "example_policy"
  policy = data.aws_iam_policy_document.example.json
}
data "aws_iam_policy_document" "example" {

  statement {
    actions   = ["s3:ListBucket", "s3:GetObject"]
```

```
    resources = ["arn:aws:s3:::example_bucket",
"arn:aws:s3:::example_bucket/*"]
  }
}
```

Section 2: Virtual Private Cloud (VPC) Security

A VPC allows you to launch AWS resources into a virtual network that you define. It provides control over the network configuration and enhances security by isolating resources.

VPC Best Practices

1. Use Subnets:
   - Create public and private subnets for better security and management.
2. Security Groups and Network ACLs:
   - Define security groups with least privilege.
   - Use Network ACLs to add an additional layer of security.
3. VPC Peering and VPN:
   - Use VPC peering to connect VPCs.
   - Set up a VPN for secure communication between your on-premises network and AWS.

Setting Up a VPC

1. Create a VPC:
   - Go to the VPC console.
   - ChooseCreate VPC .
   - Define the IPv4 CIDR block.

2. Create Subnets:
   - Go to the VPC console.
   - ChooseSubnetsand click Create subnet.
   - Define the VPC, subnet name, and IPv4 CIDR block.

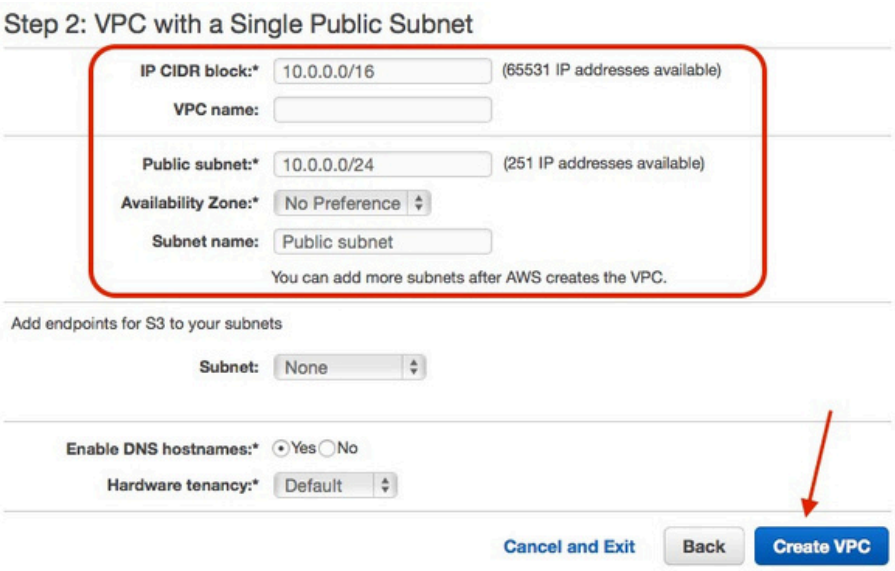


Figure 1: Creating VPC

3. Configure Security Groups:
    - Go to the VPC console.
    - ChooseSecurity Groups and clickCreate security group.
    - Define inbound and outbound rules.

Terraform Configuration

1. Create a VPC:

```
hcl

resource "aws_vpc" "example" {
cidr_block = "10.0.0.0/16"
}
```

2. Create Subnets:

```
hcl

resource "aws_subnet" "example" {
  vpc_id            = aws_vpc.example.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-west-2a"
}
```

3. Create Security Groups:

```
hcl

resource "aws_security_group" "example" {
  vpc_id      = aws_vpc.example.id
  name        = "example_security_group"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {

    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Section 3: Data Encryption

Data encryption is critical for protecting sensitive information. AWS provides several services and tools for encrypting data at rest and in transit.

Data Encryption Best Practices

1. Encrypt Data at Rest:
    - Use AWS Key Management Service (KMS) to manage encryption keys.
    - Encrypt data stored in S3, RDS, and other services.
2. Encrypt Data in Transit:
    - Use HTTPS for data transmitted over the network.
    - Enable SSL/TLS for communication between services.

Setting Up Data Encryption

1. Encrypting S3 Buckets:
    - Go to the S3 console.
    - Select a bucket and choose Properties.
    - Enable default encryption.

2. Using AWS KMS:
    - Go to the KMS console.
    - ChooseCreate key and define key settings.

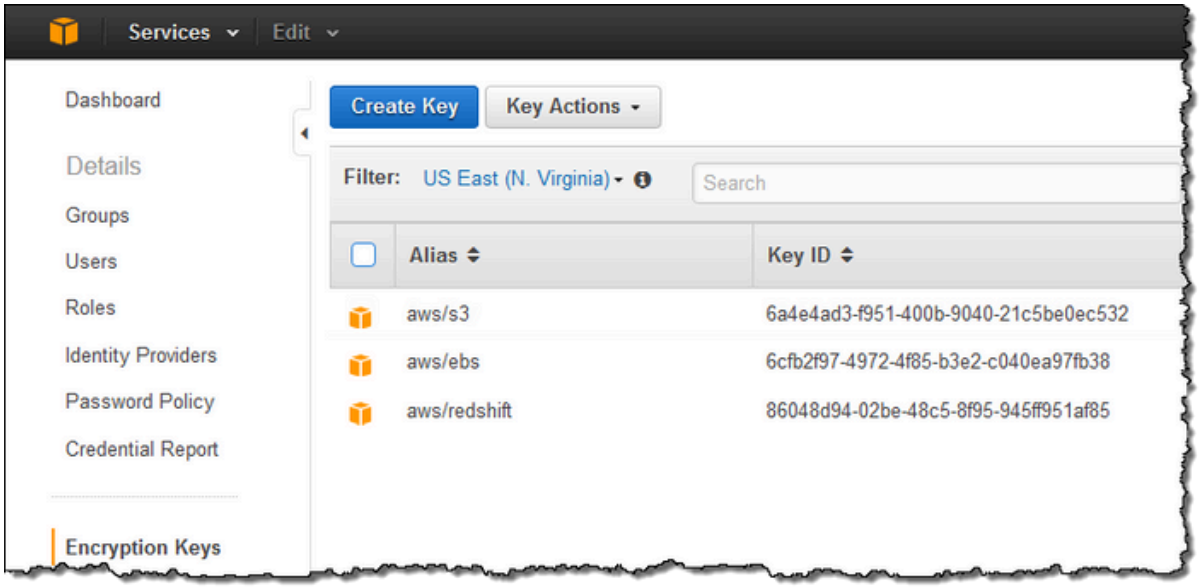


Figure 2: Create KMS

Terraform Configuration

1. Enable S3 Bucket Encryption:

```
hcl

resource "aws_s3_bucket" "example" {
  bucket = "example_bucket"
  server_side_encryption_configuration {
    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm = "aws:kms"
        kms_master_key_id = aws_kms_key.example.arn
      }
    }
  }
}
```

2. Create a KMS Key:

```
hcl

resource "aws_kms_key" "example" {
  description = "Example KMS key"
}
```

Section 4: Compliance Management

Ensuring compliance with regulatory requirements is essential for many organizations. AWS provides tools to help you achieve and maintain compliance.

Compliance Best Practices

1. AWS Config:
    - Use AWS Config to track configuration changes and ensure compliance with policies.
2. AWS CloudTrail:
    - Use CloudTrail to log and monitor all API calls.
3. AWS Security Hub:
    - Use Security Hub to get a comprehensive view of your security and compliance status.

Setting Up Compliance Tools

1. AWS Config:
    - Go to the AWS Config console.
    - Click Get started and configure settings.

2. AWS CloudTrail:
    - Go to the CloudTrail console.
    - Click Create trail and define settings.

Terraform Configuration

1. Set Up AWS Config:

```
hcl

resource "aws_config_configuration_recorder" "example" {
  name     = "example"
  role_arn = aws_iam_role.example.arn
}
resource "aws_config_delivery_channel" "example" {
  name           = "example"
  s3_bucket_name = aws_s3_bucket.example.bucket
}
resource "aws_config_configuration_recorder_status" "example" {
  name = aws_config_configuration_recorder.example.name
  is_enabled = true
}
```

2. Set Up AWS CloudTrail:

```
hcl

resource "aws_cloudtrail" "example" {
  name                  = "example"
  s3_bucket_name     = aws_s3_bucket.example.bucket
  include_global_service_events = true
  is_multi_region_trail = true
}
```

This chapter provided a comprehensive guide to security best practices in AWS, with practical examples, cheat sheets, and real-life case studies. By following the steps and best practices outlined here, you can enhance the security and compliance of your AWS environment.

Chapter 11: Automation and Scripting

Automation and scripting are essential components of AWS DevOps, allowing you to streamline processes, ensure consistency, and reduce manual intervention. This chapter dives into various automation and scripting techniques using AWS tools, CLI, and Terraform. We provide detailed explanations, fully coded examples, architecture diagrams, GUI steps, and illustrations.

Key Concepts

- AWS Command Line Interface (CLI) : Automate tasks using the AWS CLI.
- AWS SDKs : Use SDKs for various programming languages to interact with AWS services.
- Infrastructure as Code (IaC): Automate infrastructure deployment using Terraform.
- Automation Scripts : Automate routine tasks using shell scripts and AWS CLI commands.
- AWS Lambda : Automate tasks and workflows with serverless functions.

Section 1: AWS Command Line Interface (CLI)

The AWS CLI is a unified tool to manage your AWS services. With just one tool to download and configure, you can control multiple AWS services from the command line and automate them through scripts.

Installing AWS CLI

1. Windows:
    - Download the AWS CLI MSI installer from the AWS CLI Installation Page.
    - Run the installer and follow the on-screen instructions.

2. Linux / macOS:
    - Use the following commands to install AWS CLI:

```
sh

curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
```

Configuring AWS CLI

After installation, configure the CLI with your AWS credentials.

```
sh

aws configure
sh
AWS Access Key ID [None]: <your_access_key>
AWS Secret Access Key [None]: <your_secret_key>
Default region name [None]: <your_region>
Default output format [None]: json
```

AWS CLI Examples

1. List S3 Buckets:

   ```
   sh

   aws s3 ls
   ```

   Output:

   ```
   sh

   2023-01-01 12:34:56 example-bucket
   ```

2. Create an S3 Bucket:

   ```
   sh

   aws s3 mb s3://my-new-bucket
   ```

   Output:

   ```
   sh

   make_bucket: my-new-bucket
   ```

3. Upload a File to S3:

   ```
   sh

   aws s3 cp myfile.txt s3://my-new-bucket/
   ```

   Output:

   ```
   sh

   upload: ./myfile.txt to s3://my-new-bucket/myfile.txt
   ```

Section 2: AWS SDKs

AWS SDKs allow you to interact with AWS services using various programming languages, such as Python, JavaScript, and Java.

Using AWS SDK for Python (Boto3)

1. Install Boto3:

```
sh

pip install boto3
```

2. List S3 Buckets:

```
python

import boto3

s3 = boto3.client('s3')
response = s3.list_buckets()
for bucket in response['Buckets']:

    print(f'Bucket: {bucket["Name"]}')
```

Output:

```
sh

Bucket: example-bucket
```

3. Upload a File to S3:

```
python

import boto3

s3 = boto3.client('s3')
s3.upload_file('myfile.txt', 'my-new-bucket', 'myfile.txt')
```

Output:

```
sh

upload: ./myfile.txt to s3://my-new-bucket/myfile.txt
```

Section 3: Infrastructure as Code (IaC) with Terraform

Terraform is an open-source tool for building, changing, and versioning infrastructure safely and efficiently.

Terraform Basics

1. Install Terraform:
   - Download Terraform from the Terraform website.
   - Follow the installation instructions for your OS.
2. Create a Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-west-2"
}

resource "aws_s3_bucket" "example" {
  bucket = "my-terraform-bucket"
}
```

3. Initialize Terraform:

```
sh

terraform init
```

   Output:

```
sh

Initializing the backend...
Initializing provider plugins...
Terraform has been successfully initialized!
```

4. Apply Terraform Configuration:

```
sh

terraform apply
```

   Output:

```
sh

Plan: 1 to add, 0 to change, 0 to destroy.
aws_s3_bucket.example: Creating...
aws_s3_bucket.example: Creation complete after 1s [id=my-terraform-
bucket]
```

Section 4: Automation Scripts

Automate routine tasks using shell scripts and AWS CLI commands.

Example: Automated Backup Script

1. Create a Backup Script:

```
sh

#!/bin/bash

BUCKET_NAME="my-backup-bucket"
BACKUP_DIR="/path/to/backup"
TIMESTAMP=$(date +"%Y%m%d%H%M")

# Create a tarball of the backup directory
tar -czf backup-$TIMESTAMP.tar.gz -C $BACKUP_DIR .
# Upload the backup to S3
aws s3 cp backup-$TIMESTAMP.tar.gz s3://$BUCKET_NAME/
```

2. Make the Script Executable:

```
sh

chmod +x backup.sh
```

3. Run the Script:

```
sh

./backup.sh
```

Output:

```
sh

upload: ./backup-202301011200.tar.gz to s3://my-backup-bucket/backup-
202301011200.tar.gz
```

Section 5: AWS Lambda

AWS Lambda allows you to run code without provisioning or managing servers. You can use Lambda to build automated workflows and respond to events.

Example: S3 Event-Driven Lambda Function

1. Create a Lambda Function:
   - o Go to the Lambda console.
   - o Click Create function.
   - o Choose Author from scratch and configure the function.

2. Configure S3 Trigger:
   - o In the Lambda function, add an S3 trigger.
   - o Select the S3 bucket and specify the event type (e.g., ObjectCreated).

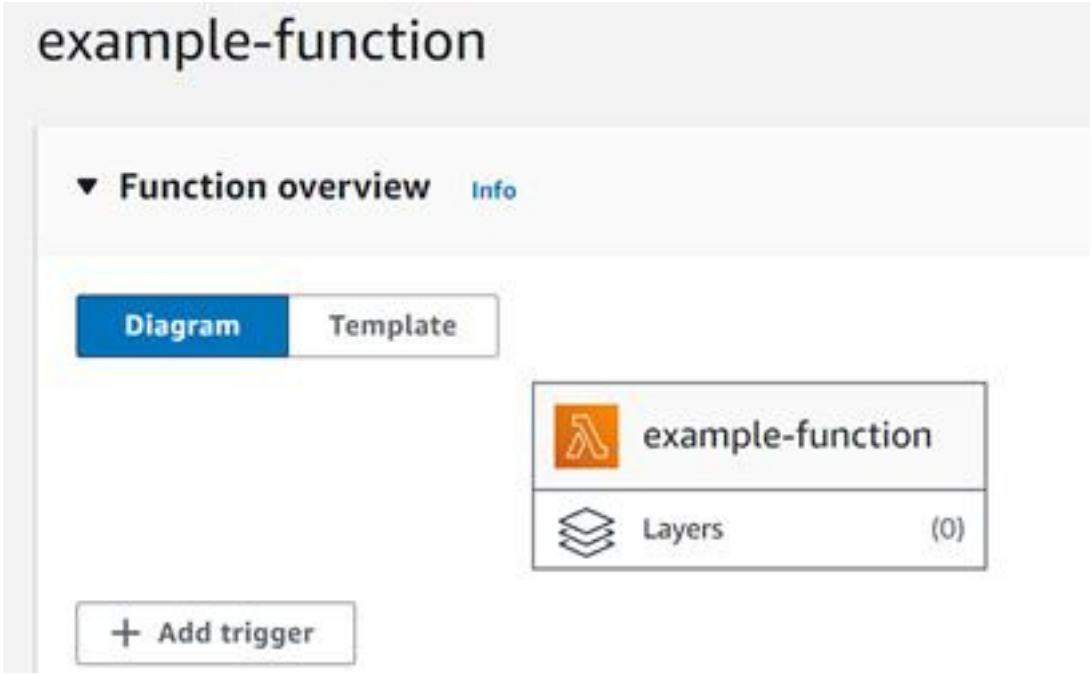


Figure 1: Lambda trigger example

3. Lambda Function Code:

```
python

import json
import boto3
def lambda_handler(event, context):

    s3 = boto3.client('s3')
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']
    # Process the S3 event
    print(f'Bucket: {bucket}, Key: {key}')
    return {

        'statusCode': 200,
        'body': json.dumps('Success')
    }
```

4. Deploy the Function:
   - Save the function code.
   - Test the function by uploading a file to the S3 bucket.

Output:

```
sh

Bucket: my-s3-bucket, Key: test-file.txt
```

Section 6: Real-Life Scenarios and Case Studies

Scenario 1: Automating EC2 Instance Management

- Objective: Automate the start and stop of EC2 instances based on a schedule.
- Solution: Use a combination of AWS Lambda, CloudWatch Events, and EC2 CLI commands.

1. Create a Lambda Function:
    - o Create a Lambda function to start EC2 instances.
    - o Create a separate Lambda function to stop EC2 instances.
2. Lambda Function Code:

```
python

import boto3

def start_instances(event, context):

    ec2 = boto3.client('ec2')
    ec2.start_instances(InstanceIds=['i-1234567890abcdef0'])

def stop_instances(event, context):
    ec2 = boto3.client('ec2')
    ec2.stop_instances(InstanceIds=['i-1234567890abcdef0'])
```

3. Configure CloudWatch Events:
    - o Create CloudWatch Events rules to trigger the Lambda functions at specific times.

Summary

In this chapter, we explored various automation and scripting techniques using AWS CLI, SDKs, Terraform, and AWS Lambda. By leveraging these tools, you can streamline your AWS operations, reduce manual effort, and ensure consistency across your infrastructure. The examples, cheat sheets, and illustrations provided should help you get started with automating your AWS environment effectively.

Chapter 12: DevOps with Kubernetes

Kubernetes, also known as K8s, is an open-source platform designed to automate deploying, scaling, and operating application containers. In this chapter, we will delve into the integration of Kubernetes within a DevOps framework, covering setup, deployment, scaling, and management using both the command line and GUI tools. We will also explore how Terraform can be used to manage Kubernetes clusters. This chapter includes comprehensive examples, architecture diagrams, cheat sheets, and real-life scenarios.

Section 1: Introduction to Kubernetes

Kubernetes simplifies the deployment and management of containerized applications. It abstracts the underlying infrastructure and provides features such as load balancing, scaling, and self-healing.

Key Concepts

- Cluster: A set of nodes that run containerized applications.
- Node: A worker machine in Kubernetes.
- Pod: The smallest deployable unit that can contain one or more containers.
- Service: An abstraction that defines a logical set of Pods and a policy by which to access them.
- Ingress: Manages external access to services in a cluster.

Section 2: Setting Up Kubernetes

You can set up a Kubernetes cluster using various methods, including managed services like Google Kubernetes Engine (GKE), Amazon Elastic Kubernetes Service (EKS), and Azure Kubernetes Service (AKS), or by setting up a cluster manually using kubeadm.
Using Minikube (Local Setup)

Minikube is a tool that lets you run Kubernetes locally. It creates a VM on your local machine and deploys a simple, single-node Kubernetes cluster.

1. Install Minikube:
    - Follow the instructions for your OS from the Minikube installation guide.
2. Start Minikube:

```
sh

minikube start
```

Output:
sh

```
    minikube v1.23.2 on Darwin 11.6
    Using the docker driver based on user configuration
    Preparing Kubernetes v1.22.2 on Docker 20.10.8 ...
    Verifying Kubernetes components...
    Enabled addons: default-storageclass, storage-provisioner
    Done! kubectl is now configured to use "minikube" cluster and
"default" namespace by default
```

3. Verify Installation:

```
sh

kubectl cluster-info
```

Output:
sh

```
Kubernetes control plane is running at https://127.0.0.1:57883
CoreDNS is running at https://127.0.0.1:57883/api/v1/namespaces/kube-
system/services/kube-dns:dns/proxy
```

Section 3: Deploying Applications

Deploying an application in Kubernetes involves creating deployment and service YAML files.

Example: Deploying a Nginx Web Server

1. Create Deployment YAML:

```
yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

2. Create Service YAML:

```
yaml

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
  port: 80
  targetPort: 80
  type: LoadBalancer
```

3. Deploy Application:

```
sh

kubectl apply -f nginx-deployment.yaml
kubectl apply -f nginx-service.yaml
```

Output:

```
sh

deployment.apps/nginx-deployment created
service/nginx-service created
```

4. Verify Deployment:

```
sh

kubectl get deployments
kubectl get services
```

Output:

```
sh

NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           1m

NAME            TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
nginx-service   LoadBalancer   10.99.32.202    <pending>     80:31009/TCP   1m
```

Section 4: Scaling Applications

Scaling applications in Kubernetes is straightforward. You can scale the number of replicas up or down based on demand.

Scaling Example

1. Scale the Nginx Deployment:

```
sh

kubectl scale deployment nginx-deployment --replicas=5
```

Output:

```
sh

deployment.apps/nginx-deployment scaled
```

2. Verify Scaling:

```
sh

kubectl get deployments
```

Output:

```
sh

NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    5/5     5            5           5m
```

Section 5: Infrastructure as Code (IaC) with Terraform

Terraform can be used to manage Kubernetes resources as code.

Example: Creating a Kubernetes Deployment with Terraform

1. Install Terraform ：
   - Download Terraform from the Terraform website.
   - Follow the installation instructions for your OS.
2. Create Terraform Configuration File：

```
hcl

provider "kubernetes" {
  config_path = "~/.kube/config"
}

resource "kubernetes_deployment" "nginx_deployment" {
  metadata {
    name = "nginx-deployment"
    labels = {
      app = "nginx"
    }
```

```
 }
  spec {
    replicas = 3
    selector {
      match_labels = {
        app = "nginx"
      }
    }
    template {
      metadata {
        labels = {
          app = "nginx"
        }
      }
      spec {
        container {
          name = "nginx"
          image = "nginx:1.14.2"
          port {
            container_port = 80
          }
        }
      }
    }
  }
}

resource "kubernetes_service" "nginx_service" {
  metadata {
    name = "nginx-service"
  }
  spec {
    selector = {
      app = "nginx"
    }
    port {
    port           = 80
       target_port = 80
    }
    type = "LoadBalancer"
  }
}
```

3. Initialize and Apply Terraform Configuration:

```
sh

terraform init
terraform apply
```

Output:

```
sh
Plan: 2 to add, 0 to change, 0 to destroy.
kubernetes_deployment.nginx_deployment: Creating...
kubernetes_service.nginx_service: Creating...
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

Section 6: Monitoring and Logging with Prometheus and Grafana

Monitoring and logging are critical for managing and troubleshooting Kubernetes clusters. Prometheus and Grafana are popular tools for these tasks.

Setting Up Prometheus and Grafana

1. Install Prometheus and Grafana using Helm:

```
sh

helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm repo update
helm install prometheus prometheus-community/kube-prometheus-stack
```

Figure 1: Kubernetes cluster monitoring (via Prometheus)

2. Access Grafana Dashboard:

```
sh

kubectl port-forward svc/prometheus-grafana 3000:80
```

   - Access Grafana at `http://localhost:3000`.

Section 7: Real-Life Scenarios and Case Studies

Scenario 1: Blue-Green Deployment

- Objective: Minimize downtime and reduce risk by deploying a new version of an application alongside the old version.
- Solution: Use Kubernetes deployments and services to manage blue-green deployments.

1. Create Blue Deployment:

```
yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-blue
spec:
  replicas: 3
  selector:
    matchLabels:
    app: nginx
    version: blue
  template:
    metadata:
      labels:
        app: nginx
        version: blue
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

2. Create Green Deployment :

```
yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-green
spec:
  replicas: 3
  selector:
    matchLabels:
    app: nginx
    version: green
  template:
    metadata:
      labels:
        app: nginx
        version: green
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

3. Switch Traffic:
    - Update the service selector to point to the green deployment.

```
yaml

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
    version: green
  ports:
    - protocol: TCP
  port: 80
  targetPort: 80
  type: LoadBalancer
```

Summary

In this chapter, we covered how to integrate Kubernetes within a DevOps framework. We explored setting up Kubernetes, deploying and scaling applications, using Terraform for IaC, monitoring with Prometheus and Grafana, and implementing real-life scenarios like blue-green deployments. The examples, cheat sheets, and illustrations provided will help you leverage Kubernetes effectively in your DevOps workflows.

Chapter 13: Cost Management and Optimization

In this chapter, we will dive deep into cost management and optimization techniques within AWS. Efficiently managing and optimizing your cloud costs is crucial to maintaining a budget-friendly and sustainable cloud environment. We'll cover the key strategies, tools, and best practices to help you monitor, analyze, and reduce your AWS expenses. This chapter includes comprehensive examples, architecture diagrams, cheat sheets, and real-life scenarios.

Section 1: Introduction to Cost Management

Cost management in AWS involves monitoring, controlling, and optimizing the expenses associated with your cloud resources. AWS provides several tools and services to help you manage costs effectively:

- AWS Cost Explorer: Visualize and analyze your AWS costs and usage.
- AWS Budgets: Set custom cost and usage budgets and receive alerts when thresholds are exceeded.
- AWS Trusted Advisor: Provides recommendations to optimize costs, improve performance, and enhance security.
- AWS Cost and Usage Report (CUR): Detailed billing and usage data for comprehensive cost analysis.

Section 2: Setting Up Cost Monitoring

Using AWS Cost Explorer

1. Accessing Cost Explorer :
    - Go to the AWS Cost Explorer Dashboard.
    - Click on "Launch Cost Explorer".
2. Creating a Cost Report:
    - Select the date range and filters (e.g., service, linked account).
    - Choose a granularity (daily, monthly, or hourly).
    - Generate the report to visualize your costs.

Section 3: Setting Up Budgets

Using AWS Budgets

1. Creating a Budget：
    - Go to the AWS Budgets Dashboard.
    - Click on "Create a Budget".
2. Configuring Budget Parameters：
    - Select budget type (Cost Budget, Usage Budget, or Reservation Budget).
    - Define budget amount and time period.
    - Set up alerts to notify you when thresholds are exceeded.

Section 4: Cost Optimization Strategies

Rightsizing Resources

1. Identify Underutilized Resources:
    - Use AWS Trusted Advisor to find underutilized EC2 instances.

```
sh

aws ce get-reservation-utilization --time-period Start=2023-01-
01,End=2023-01-31
```

Output:

```
json

{
  "UtilizationsByTime": [
    {
      "TimePeriod": {
        "Start": "2023-01-01",
        "End": "2023-01-31"
      },
      "Utilization": {
        "UtilizationPercentage": "75.0",
        "TotalReservedHours": "720",
        "UsedReservedHours": "540"
      }
    }
  ]
}
```

2. Modify Instance Types:
    - Change instance types based on the usage patterns.

```
sh

aws ec2 modify-instance-attribute --instance-id i-1234567890abcdef0 --instance-type "{\"Value\": \"t3.micro\"}"
```

Using Spot Instances

1. Launch Spot Instances via AWS Management Console:
    - Go to the EC2 Dashboard.
    - Click on "Spot Requests" and then "Request Spot Instances".
    - Specify the instance configuration and launch.

Section 5: Infrastructure as Code (IaC) with Terraform for Cost Optimization

Using Terraform to automate cost optimization can help you enforce best practices consistently.

Example: Automatically Tagging Resources

1. Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "ExampleInstance"
    Owner = "DevOpsTeam"
  }
}
```

2. Initialize and Apply Terraform Configuration:

```
sh

terraform init
terraform apply
```

Output:

```
sh

Plan: 1 to add, 0 to change, 0 to destroy.
aws_instance.example: Creating...
```

```
aws_instance.example: Creation complete after 30s [id=i-
0abcd1234efgh5678]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Section 6: Monitoring and Alerts

Setting Up CloudWatch Alarms

1. Creating a CloudWatch Alarm :
    - Go to the CloudWatch Dashboard.
    - Click on "Alarms" and then "Create Alarm".
2. Configure Alarm :
    - Select a metric (e.g., CPU utilization).
    - Set the threshold and actions to take when the alarm is triggered.

Section 7: Real-Life Case Studies

Case Study 1: Optimizing Costs for a SaaS Application

- Scenario: A SaaS company was facing high costs due to underutilized EC2 instances and inefficient use of storage.
- Solution:
    - o Used AWS Trusted Advisor to identify underutilized resources.
    - o Switched to smaller instance types.
    - o Implemented lifecycle policies for S3 storage.
    - o Automated resource tagging with Terraform for better cost allocation.

Case Study 2: Cost Management in a Multi-Account AWS Environment

- Scenario: An enterprise with multiple AWS accounts needed to manage and optimize costs across its environment.
- Solution:
    - o Centralized billing with AWS Organizations.
    - o Used AWS Budgets and Cost Explorer for cost monitoring.
    - o Implemented cross-account IAM roles for centralized management.
    - o Automated cost allocation tags using Terraform.

Section 8: Cheat Sheets and Quick Reference

Cost Management CLI Commands

- View Cost and Usage:

```sh
aws ce get-cost-and-usage --time-period Start=2023-01-01,End=2023-01-31 --granularity MONTHLY --metrics "BlendedCost"
```

- Create a Budget:

```sh
aws budgets create-budget --account-id 123456789012 --budget '{"BudgetName": "MonthlyBudget", "BudgetLimit": {"Amount": 1000, "Unit": "USD"}, "TimeUnit": "MONTHLY", "CostFilters": {}, "CostTypes": {"IncludeTax": true, "IncludeSubscription": true, "UseBlended": false}, "TimePeriod": {"Start": "2023-01-01T00:00:00Z", "End": "2024-01-01T00:00:00Z"}, "BudgetType": "COST"}'
```

Terraform Cost Optimization

- Tagging Resources:

```hcl
resource "aws_s3_bucket" "example" {
  bucket = "example-bucket"
  tags = {
    Name = "ExampleBucket"
    Owner = "DevOpsTeam"
  }
}
```

Summary

In this chapter, we covered various aspects of cost management and optimization in AWS. We explored AWS Cost Explorer, AWS Budgets, AWS Trusted Advisor, and other tools to monitor and manage costs. We also discussed rightsizing resources, using Spot Instances, and automating cost management with Terraform. The real-life case studies and cheat sheets provided practical insights and quick references for efficient cost management in AWS.

By implementing these strategies and tools, you can effectively manage and optimize your AWS expenses, ensuring a cost-efficient and scalable cloud environment.

Chapter 14: Real-Life Case Studies

In this chapter, we delve into real-life case studies that showcase practical applications of AWS DevOps practices. Each case study includes comprehensive examples with code, outputs, explanations, cheat sheets, architecture diagrams, GUI steps, and Terraform configurations wherever necessary. These case studies aim to provide insights into how AWS DevOps can be effectively implemented in various scenarios.

Case Study 1: Optimizing an E-commerce Platform

Scenario: An e-commerce company was facing challenges with scaling its platform during peak traffic periods, resulting in high costs and degraded performance.

Solution: Implementing AWS Auto Scaling, S3 for static content, and CloudFront for content delivery. Additionally, Terraform was used for infrastructure automation.

Architecture Diagram:

Steps and Examples:

1. Setting Up Auto Scaling:
   - GUI Steps: Go to the EC2 Dashboard > Auto Scaling Groups > Create Auto Scaling Group.
   - Terraform Configuration:

```
hcl
resource "aws_autoscaling_group" "example" {

  launch_configuration = aws_launch_configuration.example.id
  min_size             = 1
  max_size             = 10
  desired_capacity     = 2
  tag {
  key
  value                = "Name"
                       = "example-asg"
    propagate_at_launch = true
  }
}
```

   - Command to Apply Terraform:

```
sh

terraform init
terraform apply
```

2. Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

6. Using S3 for Static Content:
   - GUI Steps: Go to the S3 Dashboard > Create Bucket > Upload static files.
   - Terraform Configuration:

```
hcl

resource "aws_s3_bucket" "example" {
  bucket = "example-static-content"
  acl    = "public-read"

  website {
    index_document = "index.html"
  }
}
resource "aws_s3_bucket_object" "example" {
  bucket = aws_s3_bucket.example.bucket
  key    = "index.html"
  source = "path/to/index.html"
  acl    = "public-read"
}
```

   - Command to Apply Terraform:

```
sh

terraform init
terraform apply
```

7. Output:

```
mathematica

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

11. Configuring CloudFront:
    - GUI Steps: Go to the CloudFront Dashboard > Create Distribution > Set S3 bucket as origin.
    - Terraform Configuration:

```
hcl

resource "aws_cloudfront_distribution" "example" {
  origin {
    domain_name =
aws_s3_bucket.example.bucket_regional_domain_name
    origin_id   = "S3-example"

    s3_origin_config {
      origin_access_identity =
aws_cloudfront_origin_access_identity.example.cloudfront_access
_identity_path
    }
  }
  enabled
                      = true
```

```
  default_root_object = "index.html"

  default_cache_behavior {
    target_origin_id       = "S3-example"
    viewer_protocol_policy = "redirect-to-https"

    allowed_methods = ["GET", "HEAD"]
    cached_methods  = ["GET", "HEAD"]

    forwarded_values {
      query_string = false
      cookies {
        forward = "none"
      }
    }
    min_ttl                = 0
    default_ttl            = 3600
    max_ttl                = 86400
  }
  restrictions {

    geo_restriction {
      restriction_type = "none"
    }
  }

  viewer_certificate {
    cloudfront_default_certificate = true
  }
}
```

- Command to Apply Terraform:

```
sh

terraform init
terraform apply
```

Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Case Study 2: Streamlining CI/CD for a Financial Services Company

Scenario: A financial services company needed to streamline its CI/CD pipeline to improve deployment frequency and reliability.

Solution: Implementing a Jenkins-based CI/CD pipeline integrated with AWS CodeDeploy for automated deployments.

Architecture Diagram:

Steps and Examples:

1. Setting Up Jenkins on EC2:
   - o GUI Steps: Go to the EC2 Dashboard > Launch Instance > Choose Jenkins AMI.
   - o Terraform Configuration:

```
hcl

resource "aws_instance" "jenkins" {
  ami           = "ami-0c55b159cbfafe1f0" # Jenkins AMI
  instance_type = "t2.micro"

  tags = {
    Name = "JenkinsServer"
  }
}
```

   - o Command to Apply Terraform:

```
sh

terraform init
terraform apply
```

Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

2. Configuring AWS CodeDeploy:
   - o GUI Steps: Go to the CodeDeploy Dashboard > Create Application > Create Deployment Group.
   - o Terraform Configuration:

```
hcl

resource "aws_codedeploy_app" "example" {
  name = "example-app"
}

resource "aws_codedeploy_deployment_group" "example" {
  app_name              = aws_codedeploy_app.example.name
  deployment_group_name = "example-deployment-group"
```

```
  service_role_arn      = aws_iam_role.example.arn

  deployment_config_name  =  "CodeDeployDefault.OneAtATime"

  ec2_tag_filter {
  = "Name"
    key
    type = "KEY_AND_VALUE"
    value = "JenkinsServer"
  }

  auto_rollback_configuration {
    enabled = true
    events = ["DEPLOYMENT_FAILURE"]
  }
}
```

- Command to Apply Terraform:

```
sh

terraform init
terraform apply
```

Output:

```
mathematica

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

3. Integrating Jenkins with CodeDeploy:
   - GUI Steps: In Jenkins, install the AWS CodeDeploy plugin > Configure the CodeDeploy plugin with AWS credentials.
   - Jenkins Pipeline Script:

```
groovy

pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
    stage('Deploy') {
      steps {
        script {
          def codedeploy = awsCodeDeploy(
            applicationName: 'example-app',
            deploymentGroupName: 'example-deployment-group',
            s3Location: [
              bucket: 'my-bucket',
              key: 'example-app.zip',
              bundleType: 'zip'
            ]
          )
        }
      }
```

```
        }
      }
    }
```

Output:

```
csharp  [Pipeline]  Start  of

Pipeline
[Pipeline] Stage: Build
[Pipeline] sh
...
[Pipeline] End of Pipeline
[Pipeline] End of Pipeline
```

Case Study 3: Automating Infrastructure for a Healthcare Application

Scenario: A healthcare application required automated infrastructure provisioning and deployment to support rapid development cycles.

Solution: Using Terraform for Infrastructure as Code (IaC) and AWS CodePipeline for automated deployments.

Architecture Diagram:

Steps and Examples:

1. Provisioning Infrastructure with Terraform:
    - Terraform Configuration:

```
hcl

provider "aws" {
  region = "us-east-1"
}

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "subnet" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-east-1a"
}

resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.subnet.id
  tags = {
    Name = "WebServer"
  }
}
```

- Command to Apply Terraform:

```
sh

terraform init
terraform apply
```

Output:

```
mathematica

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

2. Setting Up AWS CodePipeline:
   - GUI Steps: Go to the CodePipeline Dashboard > Create Pipeline > Add Source (GitHub) > Add Build (CodeBuild) > Add Deploy (CodeDeploy).
   - Terraform Configuration:

```
hcl

resource "aws_codepipeline" "example" {
  name     = "example-pipeline"
  role_arn = aws_iam_role.example.arn

  artifact_store {
    location = aws_s3_bucket.example.bucket
    type     = "S3"
  }
  stage {

    name = "Source"
    action {
    name                = "Source"
    category            = "Source"
    owner               = "AWS"
    provider            = "GitHub"
    version             = "1"
      output_artifacts = ["source_output"]
      configuration = {
        Owner      = "my-github-account"
        Repo       = "my-repo"
        Branch     = "main"
        OAuthToken = "my-oauth-token"
      }
    }
  }

  stage {
    name = "Build"
    action {
    name                = "Build"
    category            = "Build"
    owner               = "AWS"
    provider            = "CodeBuild"
      input_artifacts = ["source_output"]
      output_artifacts = ["build_output"]
      configuration = {
```

```
        ProjectName = aws_codebuild_project.example.name
      }
    }
  }

  stage {
    name = "Deploy"
    action {
    name             = "Deploy"
    category         = "Deploy"
    owner            = "AWS"
    provider         = "CodeDeploy"
      input_artifacts = ["build_output"]
      configuration = {
      ApplicationName       = aws_codedeploy_app.example.name
        DeploymentGroupName =
aws_codedeploy_deployment_group.example.name
      }
    }
  }
}
```

- Command to Apply Terraform:

```
sh

terraform init
terraform apply
```

Output:

```
mathematica

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
```

Summary

In this chapter, we explored real-life case studies demonstrating the practical application of AWS DevOps practices. We covered scenarios ranging from e-commerce optimization to CI/CD streamlining and healthcare infrastructure automation. Each case study provided detailed examples with code, outputs, and explanations, as well as architecture diagrams and GUI steps. These case studies illustrate how AWS DevOps can be effectively implemented to solve real-world challenges, providing valuable insights and best practices.

By applying the strategies and tools discussed in these case studies, you can enhance the efficiency, scalability, and cost-effectiveness of your AWS infrastructure and DevOps processes.

Chapter 16: AWS DevOps Cheat Sheets

In this chapter, we provide comprehensive cheat sheets for various AWS DevOps services and tools. Each cheat sheet includes fully coded examples with outputs and explanations, architecture diagrams where necessary, and GUI steps and Terraform configurations with illustrations.

Cheat Sheet 1: AWS IAM (Identity and Access Management)

Key Concepts:

- Users, Groups, and Roles
- Policies and Permissions
- MFA (Multi-Factor Authentication)
- Access Keys

Commands and Examples:

1. Creating an IAM User:
   - GUI Steps: Go to the IAM Dashboard > Users > Add User > Configure user details and permissions.
   - AWS CLI Command:

     ```
     sh

     aws iam create-user --user-name MyUser
     ```

   - Terraform Configuration:

     ```
     hcl

     resource "aws_iam_user" "my_user" {
       name = "MyUser"
     }
     ```

   - Output:

     ```
     json
     {
         "User": {
             "Path": "/",
             "UserName": "MyUser",
             "UserId": "AIDXXXXXXXXXXXXXX",
             "Arn": "arn:aws:iam::123456789012:user/MyUser",
             "CreateDate": "2023-06-07T12:34:56Z"
         }
     }
     ```

2. Creating an IAM Role:
    - GUI Steps: Go to the IAM Dashboard > Roles > Create Role > Select AWS service and use case > Attach policies.
    - Terraform Configuration:

```
hcl
resource "aws_iam_role" "my_role" {

  name = "MyRole"
  assume_role_policy = jsonencode({

    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com"
        }
        Action = "sts:AssumeRole"
      }
    ]
  })
}
```

    - Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Cheat Sheet 2: Amazon S3

Key Concepts:

- Buckets and Objects
- Access Control
- Versioning
- Static Website Hosting

Commands and Examples:

1. Creating an S3 Bucket:
   - o GUI Steps: Go to the S3 Dashboard > Create Bucket > Configure settings.
   - o AWS CLI Command:

```
sh

aws s3api create-bucket --bucket my-bucket --region us-east-1
```

   - o Terraform Configuration:

```
hcl

resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-bucket"
  acl    = "private"
}
```

   - ○ Output:

```
json

{
    "Location": "http://my-bucket.s3.amazonaws.com/"
}
```

2. Uploading an Object to S3:
   - o GUI Steps: Go to the S3 Dashboard > Select Bucket > Upload.
   - o AWS CLI Command:

```
sh

aws s3 cp my-file.txt s3://my-bucket/my-file.txt
```

   - o Terraform Configuration:

```
hcl

resource "aws_s3_bucket_object" "my_object" {
  bucket = aws_s3_bucket.my_bucket.bucket
  key    = "my-file.txt"
  source = "path/to/my-file.txt"
  acl    = "private"
}
```

- Output:

```
swift

{
    "ETag": "\"9a0364b9e99bb480dd25e1f0284c8555\""
}
```

Cheat Sheet 3: EC2 Instances

Key Concepts:

- Instance Types
- Key Pairs
- Security Groups
- AMIs (Amazon Machine Images)

Commands and Examples:

1. Launching an EC2 Instance:
   - GUI Steps: Go to the EC2 Dashboard > Launch Instance > Configure instance details.
   - AWS CLI Command:

```
sh

aws ec2 run-instances --image-id ami-0abcdef1234567890 --count
1 --instance-type t2.micro --key-name MyKeyPair --security-
group-ids sg-12345678
```

   - Terraform Configuration:

```
hcl

resource "aws_instance" "my_instance" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
  key_name      = "MyKeyPair"
  security_groups = ["sg-12345678"]

  tags = {
    Name = "MyInstance"
  }
}
```

   - Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

2. Creating a Security Group:
    - GUI Steps: Go to the EC2 Dashboard > Security Groups > Create Security Group > Configure rules.
    - AWS CLI Command:

```
sh

aws ec2 create-security-group --group-name MySecurityGroup --
description "My security group"
```

    - Terraform Configuration:

```
hcl

resource "aws_security_group" "my_security_group" {
  name        = "MySecurityGroup"
  description = "My security group"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {

    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

    - Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Cheat Sheet 4: AWS Lambda

Key Concepts:

- Function Configuration
- Triggers
- Layers
- Environment Variables

Commands and Examples:

1. Creating a Lambda Function:
    - GUI Steps: Go to the Lambda Dashboard > Create Function > Configure function details.

- AWS CLI Command:

```
sh

aws lambda create-function --function-name MyFunction --runtime
python3.8 --role arn:aws:iam::123456789012:role/MyRole --
handler lambda_function.lambda_handler --zip-file
fileb://function.zip
```

- Terraform Configuration:

```
hcl

resource "aws_lambda_function" "my_function" {
  function_name = "MyFunction"
  role          = aws_iam_role.my_role.arn
  handler       = "lambda_function.lambda_handler"
  runtime       = "python3.8"
  filename      = "path/to/function.zip"

  environment {
    variables = {
      key = "value"
    }
  }
}
```

- Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

2. Setting Up a Trigger:
   - GUI Steps: Go to the Lambda function > Add Trigger > Select trigger source.
   - AWS CLI Command:

```
sh
aws lambda create-event-source-mapping --function-name
MyFunction --batch-size 5 --event-source-arn arn:aws:sqs:us-
east-1:123456789012:MyQueue
```

- Terraform Configuration:

```
hcl

resource "aws_lambda_event_source_mapping" "my_mapping" {
  event_source_arn = aws_sqs_queue.my_queue.arn
  function_name    = aws_lambda_function.my_function.arn
  batch_size       = 5
}
```

- Output:

```
mathematica
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Cheat Sheet 5: Continuous Integration and Continuous Deployment (CI/CD)

Key Concepts:

- CodeCommit
- CodeBuild
- CodeDeploy
- CodePipeline

Commands and Examples:

1. Creating a CodeCommit Repository:
   - GUI Steps: Go to the CodeCommit Dashboard > Create Repository > Configure repository details.
   - AWS CLI Command:

```
sh

aws codecommit create-repository --repository-name MyRepository
--repository-description "My repository description"
```

   - Terraform Configuration:

```
hcl

resource "aws_codecommit_repository" "my_repository" {
  repository_name = "MyRepository"
  description     = "My repository description"
}
```

   - Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

2. Setting Up a CodeBuild Project:
   - GUI Steps: Go to the CodeBuild Dashboard > Create Project > Configure project settings.
   - AWS CLI Command:

```
sh
aws codebuild create-project --name MyProject --source
type=CODECOMMIT,location=https://git-codecommit.us-east-
1.amazonaws.com/v1/repos/MyRepository --artifacts
type=NO_ARTIFACTS --environment
type=LINUX_CONTAINER,image=aws/codebuild/standard:4.0,computeTy
pe=BUILD_GENERAL1_SMALL --service-role
arn:aws:iam::123456789012:role/MyCodeBuildServiceRole
```

   - Terraform Configuration:

```
hcl

resource "aws_codebuild_project" "my_project" {
  name          = "MyProject"
  service_role = aws_iam_role.my_codebuild_role.arn
  source {
  type        = "CODECOMMIT"
    location = "https://git-codecommit.us-east-
1.amazonaws.com/v1/repos/MyRepository"
  }
  artifacts {
    type = "NO_ARTIFACTS"
  }
  environment {
    compute_type                = "BUILD_GENERAL1_SMALL"
    image                       = "aws/codebuild/standard:4.0"
    type                        = "LINUX_CONTAINER"
    image_pull_credentials_type = "CODEBUILD"
  }
}
```

- Output:

```
mathematica

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

3. Creating a CodePipeline:
   - GUI Steps: Go to the CodePipeline Dashboard > Create Pipeline > Configure pipeline settings.
   - AWS CLI Command:

```
sh aws codepipeline create-pipeline --pipeline
name=MyPipeline,roleArn=arn:aws:iam::123456789012:role/MyCodePi
pelineServiceRole,artifactStore={type=S3,location=my-pipeline-
artifacts},stages=[{name=Source,actions=[{name=SourceAction,act
ionTypeId={category=Source,owner=AWS,provider=CodeCommit,versio
n=1},outputArtifacts=[{name=SourceOutput}],configuration={Branc
hName=main,RepositoryName=MyRepository}}]},{name=Build,actions=
[{name=BuildAction,actionTypeId={category=Build,owner=AWS,provi
der=CodeBuild,version=1},inputArtifacts=[{name=SourceOutput}],o
utputArtifacts=[{name=BuildOutput}],configuration={ProjectName=
MyBuildProject}}]},{name=Deploy,actions=[{name=DeployAction,act
ionTypeId={category=Deploy,owner=AWS,provider=CodeDeploy,versio
n=1},inputArtifacts=[{name=Build Output}],configuration={Applica
tionName=MyApplication,DeploymentGroupName=MyDeploymentGroup}}]
}]
```

   - Terraform Configuration:

```
hcl
resource "aws_codepipeline" "my_pipeline" {

  name     = "MyPipeline"
  role_arn = aws_iam_role.my_codepipeline_role.arn
```

```
  artifact_store {
    location = aws_s3_bucket.my_bucket.bucket
    type     = "S3"
  }

  stage {
    name = "Source"

    action {
      name             = "SourceAction"
      category         = "Source"
      owner            = "AWS"
      provider         = "CodeCommit"
      version          = "1"
      output_artifacts = ["SourceOutput"]
      configuration = {
        BranchName     = "main"
        RepositoryName = "MyRepository"
      }
    }
  }

  stage {
    name = "Build"

    action {
      name             = "BuildAction"
      category         = "Build"
      owner            = "AWS"
      provider         = "CodeBuild"
      input_artifacts = ["SourceOutput"]
      output_artifacts = ["BuildOutput"]
      configuration = {
        ProjectName = aws_codebuild_project.my_project.name
      }
    }
  }

  stage {
    name = "Deploy"

    action {
      name             = "DeployAction"
      category         = "Deploy"
      owner            = "AWS"
      provider         = "CodeDeploy"
      input_artifacts = ["BuildOutput"]
      configuration = {
      ApplicationName       = aws_codedeploy_app.my_app.name
        DeploymentGroupName =
aws_codedeploy_deployment_group.my_deployment_group.name
      }
    }
  }
}
```

- Output:

```
mathematica
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Summary

This chapter provided comprehensive cheat sheets for various AWS DevOps services, complete with fully coded examples, outputs, explanations, architecture diagrams, and GUI steps. By using these cheat sheets, you can quickly reference essential commands and configurations, facilitating efficient and effective management of AWS DevOps tasks.

Chapter 15: Conclusion and Future Trends

In this final chapter, we will summarize the key learnings from the ebook and explore the future trends in AWS DevOps. We will also provide fully coded examples, cheat sheets, architecture diagrams where necessary, and GUI steps with illustrations.

Conclusion

Key Learnings:

1. Identity and Access Management (IAM):
    - Configuring and managing users, roles, and policies.
    - Securing access to AWS resources.
2. Storage Solutions with S3:
    - Creating and managing S3 buckets and objects.
    - Configuring access control and versioning.
3. Compute Services:
    - Launching and managing EC2 instances.
    - Creating and deploying AWS Lambda functions.
4. Containers and Orchestration:
    - Managing Docker containers with ECS and EKS.
    - Configuring Kubernetes clusters on AWS.
5. CI/CD:
    - Setting up pipelines using CodeCommit, CodeBuild, CodeDeploy, and CodePipeline.
    - Implementing continuous integration and deployment workflows.
6. Infrastructure as Code (IaC):
    - Using Terraform to provision AWS resources.
    - Automating infrastructure management.
7. Monitoring and Logging:
    - Setting up CloudWatch for monitoring and logging.
    - Using CloudTrail for auditing and compliance.
8. Security Best Practices:
    - Implementing security measures across AWS services.
    - Managing keys and encrypting data.
9. Automation and Scripting:
    - Automating tasks with AWS CLI and SDKs.
    - Using Lambda functions for serverless automation.
10. Cost Management and Optimization:
    - Tracking and managing AWS costs.
    - Implementing cost-saving strategies.
11. Real-Life Case Studies:
    - Applying AWS DevOps in various industries.
    - Learning from successful DevOps implementations.
12. Cheat Sheets:
    - Quick reference guides for common AWS DevOps tasks.
    - Handy commands and configurations.

Future Trends in AWS DevOps

1. Serverless Computing:

- Trend: Increasing adoption of serverless architectures using AWS Lambda and other serverless services.
- Example: Automating image processing using Lambda functions.

```
python

import boto3

def lambda_handler(event, context):
    s3 = boto3.client('s3')
    # Process the S3 event
    # Example code for processing image files in S3.
```

2. Artificial Intelligence and Machine Learning Integration:

- Trend: Integrating AI/ML capabilities into DevOps pipelines for intelligent automation.
- Example: Using Amazon SageMaker for model training and deployment.

```
python

import sagemaker
from sagemaker import get_execution_role
role = get_execution_role()
# Train and deploy a machine learning model
```

3. Enhanced Security and Compliance:

- Trend: Strengthening security measures with advanced tools and practices.
- Example: Automating security checks with AWS Config rules.

```
python

import boto3

client = boto3.client('config')

response = client.put_config_rule(
    ConfigRule={
        'ConfigRuleName': 's3-bucket-public-read-prohibited',
        # Config rule details
    }
)
```

4. Hybrid and Multi-Cloud Strategies:

- Trend: Implementing hybrid and multi-cloud architectures for flexibility and redundancy.
- Example: Using Terraform to manage resources across AWS and other cloud providers.

```
hcl

provider "aws" {
  region = "us-east-1"
}

provider "google" {
  project = "my-project"
  region = "us-central1"
}
resource "aws_instance" "my_instance" {

  ami           = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
}
resource "google_compute_instance" "my_instance" {

  name         = "my-instance"
  machine_type = "f1-micro"
  # Other GCP instance configuration
}
```

5. Observability and Advanced Monitoring:

- Trend: Enhancing observability with comprehensive monitoring, logging, and tracing solutions.
- Example: Setting up AWS X-Ray for distributed tracing.

```
python

from aws_xray_sdk.core import xray_recorder

@xray_recorder.capture('my_function')
def my_function():
    # Function code
```

Future Trends Architecture Diagram

Hybrid Cloud Architecture:

- Diagram Description:
  - Integrating AWS with other cloud providers for a hybrid cloud setup.
  - Using Terraform for infrastructure as code.
  - Implementing serverless functions for specific tasks.
  - Ensuring security and compliance with automated checks.
- Diagram Components:
  - AWS resources (EC2, S3, Lambda, etc.)
  - Other cloud provider resources (e.g., Google Cloud instances)
  - Terraform configuration
  - Security and monitoring tools (e.g., AWS Config, CloudWatch, X-Ray)

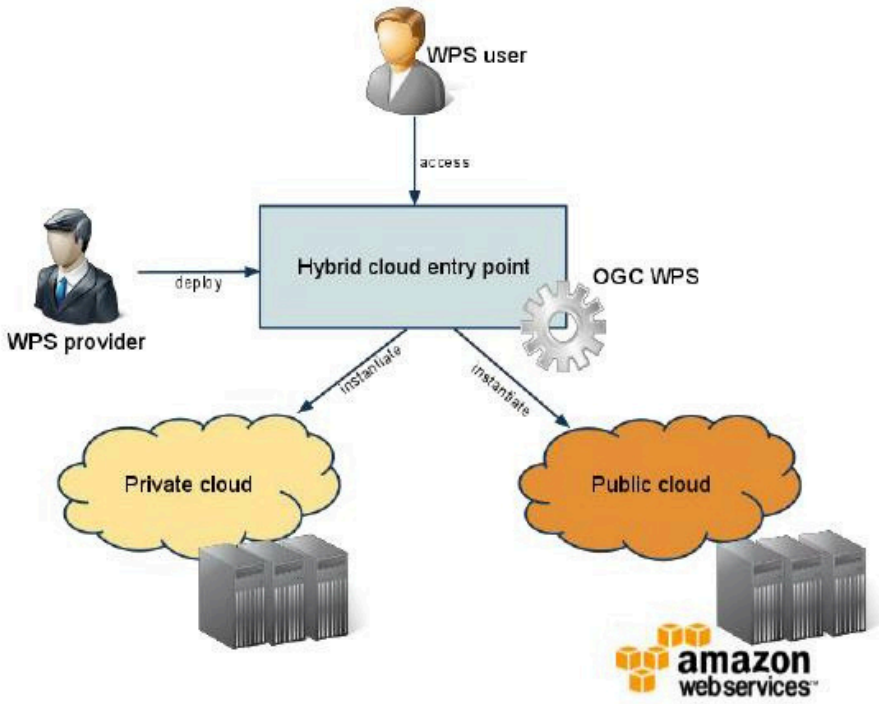


Figure 1: Hybrid Cloud Architecture Diagram

Summary

In this chapter, we reviewed the key learnings from the AWS DevOps journey, explored future trends, and provided practical examples and illustrations to stay ahead in the evolving DevOps landscape. By embracing these trends and leveraging AWS services effectively, you can ensure your DevOps practices remain cutting-edge and future-proof.

AWS Whitepaper

Introduction to DevOps on AWS

Introduction to DevOps on AWS: AWS Whitepaper

Table of Contents

Introduction to DevOps on AWS

Publication date: **April 7, 2023** (*Document Revisions*)

Today more than ever, enterprises are embarking on their digital transformation journey to build deeper connections with their customers, to achieve sustainable and enduring business value. Organizations of all shapes and sizes are disrupting their competitors and entering new markets by innovating more quickly than ever before. For these organizations, it is important to focus on innovation and software disruption, making it critical to streamline their software delivery. Organizations that shorten their time from idea to production making speed and agility a priority could be tomorrow's disruptors.

While there are several factors to consider in becoming the next digital disruptor, this whitepaper focuses on DevOps, and the services and features in the Amazon Web Services (AWS) platform that will help increase an organization's ability to deliver applications and services at a high velocity.

Introduction

DevOps is the combination of cultural philosophies, engineering practices, and tools which increase an organization's ability to deliver applications and services at high velocity and better quality. Over time, several essential practices have emerged when adopting DevOps: continuous integration (CI), continuous delivery (CD), Infrastructure as Code (IaC), and monitoring and logging.

This paper highlights AWS capabilities that help you accelerate your DevOps journey, and how AWS services can help remove the undifferentiated heavy lifting associated with DevOps adaptation. It also describes how to build a continuous integration and delivery capability without managing servers or build nodes, and how to use IaC to provision and manage your cloud resources in a consistent and repeatable manner.

- **Continuous integration:** A software development practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run.
- **Continuous delivery:** A software development practice where code changes are automatically built, tested, and prepared for a release to production.
- **Infrastructure as Code:** A practice in which infrastructure is provisioned and managed using code and software development techniques, such as version control, and continuous integration.
- **Monitoring and logging:** Enables organizations to see how application and infrastructure performance impacts the experience of their product’s end user.

- **Communication and collaboration:** Practices are established to bring the teams closer and by building workflows and distributing the responsibilities for DevOps.
- **Security:** Should be a cross cutting concern. Your continuous integration and continuous delivery (CI/CD) pipelines and related services should be safeguarded and proper access control permissions should be set up.

An examination of each of these principles reveals a close connection to the offerings available from AWS.

Are you Well-Architected?

The AWS Well-Architected Framework helps you understand the pros and cons of the decisions you make when building systems in the cloud. The six pillars of the Framework allow you to learn architectural best practices for designing and operating reliable, secure, efficient, cost-effective, and sustainable systems. Using the AWS Well-Architected Tool, available at no charge in the AWS Management Console, you can review your workloads against these best practices by answering a set of questions for each pillar.

Continuous integration

Continuous integration (CI) is a software development practice where developers regularly merge their code changes into a central code repository, after which automated builds and tests are run. CI helps find and address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.

AWS offers the following services for continuous integration:

Topics

- AWS CodeCommit
- AWS CodeBuild
- AWS CodeArtifact

AWS CodeCommit

AWS CodeCommit is a secure, highly scalable, managed source control service that hosts private git repositories. CodeCommit reduces the need for you to operate your own source control system and there is no hardware to provision and scale or software to install, configure, and operate. You can use CodeCommit to store anything from code to binaries, and it supports the standard functionality of GitHub, allowing it to work seamlessly with your existing Git-based tools. Your team can also use CodeCommit's online code tools to browse, edit, and collaborate on projects. AWS CodeCommit has several benefits:

- **Collaboration** — AWS CodeCommit is designed for collaborative software development. You can easily commit, branch, and merge your code, which helps you easily maintain control of your team's projects. CodeCommit also supports pull requests, which provide a mechanism to request code reviews and discuss code with collaborators.
- **Encryption** — You can transfer your files to and from AWS CodeCommit using HTTPS or SSH, as you prefer. Your repositories are also automatically encrypted at rest through AWS Key Management Service (AWS KMS) using customer-specific keys.
- **Access control** — AWS CodeCommit uses AWS Identity and Access Management (IAM) to control and monitor who can access your data in addition to how, when, and where they can access it. CodeCommit also helps you monitor your repositories through AWS CloudTrail and Amazon CloudWatch.

High availability and durability — AWS CodeCommit stores your repositories in Amazon Simple Storage Service (Amazon S3) and Amazon DynamoDB. Your encrypted data is redundantly stored across multiple facilities. This architecture increases the availability and durability of your repository data.

- **Notifications and custom scripts** — You can now receive notifications for events impacting your repositories. Notifications will come as Amazon Simple Notification Service (Amazon SNS) notifications. Each notification will include a status message as well as a link to the resources whose event generated that notification. Additionally, using AWS CodeCommit repository cues, you can send notifications and create HTTP webhooks with Amazon SNS or invoke AWS Lambda functions in response to the repository events you choose.

AWS CodeBuild

AWS CodeBuild is a fully managed continuous integration service that compiles source code, runs tests, and produces software packages that are ready to deploy. You don't need to provision, manage, and scale your own build servers. CodeBuild can use either of GitHub, GitHub Enterprise, BitBucket, AWS CodeCommit, or Amazon S3 as a source provider.

CodeBuild scales continuously and can process multiple builds concurrently. CodeBuild offers various pre-configured environments for various versions of Microsoft Windows and Linux. Customers can also bring their customized build environments as Docker containers. CodeBuild also integrates with open source tools such as Jenkins and Spinnaker.

CodeBuild can also create reports for unit, functional, or integration tests. These reports provide a visual view of how many tests cases were run and how many passed or failed. The build process can also be run inside an Amazon Virtual Private Cloud (Amazon VPC) which can be helpful if your integration services or databases are deployed inside a VPC.

AWS CodeArtifact

AWS CodeArtifact is a fully managed artifact repository service that can be used by organizations to securely store, publish, and share software packages used in their software development process. CodeArtifact can be configured to automatically fetch software packages and dependencies from public artifact repositories so developers have access to the latest versions.

Software development teams increasingly rely on open-source packages to perform common tasks in their application package. It has become critical for software development teams to maintain

control on a particular version of the open-source software to ensure the software is free of vulnerabilities. With CodeArtifact, you can set up controls to enforce this.

CodeArtifact works with commonly used package managers and build tools such as Maven, Gradle, npm, yarn, twine, and pip, making it easy to integrate into existing development workflows.

Continuous delivery

Continuous delivery (CD) is a software development practice where code changes are automatically prepared for a release to production. A pillar of modern application development, continuous delivery expands upon continuous integration by deploying all code changes to a testing environment and/or a production environment after the build stage. When properly implemented, developers will always have a deployment-ready build artifact that has passed through a standardized test process.

Continuous delivery lets developers automate testing beyond just unit tests so they can verify application updates across multiple dimensions before deploying to customers.

These tests might include UI testing, load testing, integration testing, API reliability testing, and more. This helps developers more thoroughly validate updates and preemptively discover issues. Using the cloud, it is easy and cost-effective to automate the creation and replication of multiple environments for testing, which was previously difficult to do on-premises.

AWS offers the following services for continuous delivery:

- AWS CodeBuild
- AWS CodeDeploy
- AWS CodePipeline

Topics

AWS CodeDeploy

AWS CodeDeploy is a fully managed deployment service that automates software deployments to a variety of compute services such as Amazon Elastic Compute Cloud (Amazon EC2), AWS Fargate, AWS Lambda, and your on-premises servers. AWS CodeDeploy makes it easier for you to rapidly release new features, helps you avoid downtime during application deployment, and handles the complexity of updating your applications. You can use CodeDeploy to automate software deployments, reducing the need for error-prone manual operations. The service scales to match your deployment needs.

CodeDeploy has several benefits that align with the DevOps principle of continuous deployment:

- **Automated deployments** — CodeDeploy fully automates software deployments, allowing you to deploy reliably and rapidly.
- **Centralized control** — CodeDeploy enables you to easily launch and track the status of your application deployments through the AWS Management Console or the AWS CLI. CodeDeploy gives you a detailed report enabling you to view when and to where each application revision was deployed. You can also create push notifications to receive live updates about your deployments.
- **Minimize downtime** — CodeDeploy helps maximize your application availability during the software deployment process. It introduces changes incrementally and tracks application health according to configurable rules. Software deployments can easily be stopped and rolled back if there are errors.
- **Easy to adopt** — CodeDeploy works with any application, and provides the same experience across different platforms and languages. You can easily reuse your existing setup code. CodeDeploy can also integrate with your existing software release process or continuous delivery toolchain (for example, AWS CodePipeline, GitHub, Jenkins).

AWS CodeDeploy supports multiple deployment options. For more information, refer to the *Deployment strategies* section of this document.

AWS CodePipeline

AWS CodePipeline is a continuous delivery service that you can use to model, visualize, and automate the steps required to release your software. With AWS CodePipeline, you model the full release process for building your code, deploying to pre-production environments, testing your application, and releasing it to production. AWS CodePipeline then builds, tests, and deploys your application according to the defined workflow every time there is a code change. You can integrate partner tools and your own custom tools into any stage of the release process to form an end-to-end continuous delivery solution.

AWS CodePipeline has several benefits that align with the DevOps principle of continuous deployment:

- **Rapid delivery** — AWS CodePipeline automates your software release process, allowing you to rapidly release new features to your users. With CodePipeline, you can quickly iterate on feedback and get new features to your users faster.

- **Improved quality** — By automating your build, test, and release processes, AWS CodePipeline enables you to increase the speed and quality of your software updates by running all new changes through a consistent set of quality checks.
- **Easy to integrate** — AWS CodePipeline can easily be extended to adapt to your specific needs. You can use the pre-built plugins or your own custom plugins in any step of your release process. For example, you can pull your source code from GitHub, use your on-premises Jenkins build server, run load tests using a third-party service, or pass on deployment information to your custom operations dashboard.
- **Configurable workflow** — AWS CodePipeline enables you to model the different stages of your software release process using the console interface, the AWS CLI, AWS CloudFormation, or the AWS SDKs. You can easily specify the tests to run and customize the steps to deploy your application and its dependencies.

Deployment strategies

Deployment strategies define how you want to deliver your software. Organizations follow different deployment strategies based on their business model. Some choose to deliver software that is fully tested, and others might want their users to provide feedback and let their users evaluate under development features (such as Beta releases). The following section discusses various deployment strategies.

In-place deployments

In this strategy, the previous version of the application on each compute resource is stopped, the latest application is installed, and the new version of the application is started and validated. This allows application deployments to proceed with minimal disturbance to underlying infrastructure. With an in-place deployment, you can deploy your application without creating new infrastructure; however, the availability of your application can be affected during these deployments. This approach also minimizes infrastructure costs and management overhead associated with creating new resources. You can use a load balancer so that each instance is deregistered during its deployment and then restored to service after the deployment is complete. In-place deployments can be all-at-once, assuming a service outage, or done as a rolling update. AWS CodeDeploy and AWS Elastic Beanstalkoffer deployment configurations for one-at-a-time, half-at-a-time, and all-at-once.

Blue/green deployment

Blue/green deployment, sometimes referred to as red/black deployment, is a technique for releasing applications by shifting traffic between two identical environments running differing versions of the application. Blue/green deployment helps you minimize downtime during application updates, mitigating risks surrounding downtime and rollback functionality.

Blue/green deployments enable you to launch a new version (green) of your application alongside the old version (blue), and monitor and test the new version before you reroute traffic to it, rolling back on issue detection.

Canary deployment

The purpose of a canary deployment is to reduce the risk of deploying a new version that impacts the workload. The method will incrementally deploy the new version, making it visible to new

users in a slow fashion. As you gain confidence in the deployment, you will deploy it to replace the current version in its entirety.

Linear deployment

Linear deployment means traffic is shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment.

All-at-once deployment

All-at-once deployment means all traffic is shifted from the original environment to the replacement environment all at once.

Deployment strategies matrix

The following matrix lists the supported deployment strategies for Amazon Elastic Container Service (Amazon ECS), AWS Lambda, and Amazon EC2/on-premises.

- Amazon ECS is a fully managed orchestration service.
- AWS Lambda lets you run code without provisioning or managing servers.
- Amazon EC2 enables you to run secure, resizable compute capacity in the cloud.

Deployment strategy	Amazon ECS	AWS Lambda	Amazon EC2/ on-premises
In-place	✓	✓	✓
Blue/green	✓	✓	✓*
Canary	✓	✓	X
Linear	✓	✓	X
All-at-once	✓	✓	X

Note

Blue/green deployment with EC2/on-premises works only with EC2 instances.

AWS Elastic Beanstalk deployment strategies

AWS Elastic Beanstalk supports the following type of deployment strategies:

- **All-at-once** Performs in place deployment on all instances.
- **Rolling** Splits the instances into batches and deploys to one batch at a time.
- **Rolling with additional batch** Splits the deployments into batches but for the first batch creates new EC2 instances instead of deploying on the existing EC2 instances.

- **Immutable** If you need to deploy with a new instance instead of using an existing instance.
- **Traffic splitting** Performs immutable deployment and then forwards percentage of traffic to the new instances for a pre-determined duration of time. If the instances stay healthy, then forward all traffic to new instances and shut down old instances.

Infrastructure as code

A fundamental principle of DevOps is to treat infrastructure the same way developers treat code. Application code has a defined format and syntax. If the code is not written according to the rules of the programming language, applications cannot be created. Code is stored in a version management or source control system that logs a history of code development, changes, and bug fixes. When code is compiled or built into applications, we expect a consistent application to be created, and the build is repeatable and reliable.

Practicing *infrastructure as code* means applying the same rigor of application code development to infrastructure provisioning. All configurations should be defined in a declarative way and stored in a source control system such as AWS CodeCommit, the same as application code. Infrastructure provisioning, orchestration, and deployment should also support the use of the infrastructure as code.

Infrastructure was traditionally provisioned using a combination of scripts and manual processes. Sometimes these scripts were stored in version control systems or documented step by step in text files or run-books. Often the person writing the run books is not the same person executing these scripts or following through the run-books. If these scripts or runbooks are not updated frequently, they can potentially become a show-stopper in deployments. This results in the creation of new environments not always being repeatable, reliable, or consistent.

In contrast, AWS provides a DevOps-focused way of creating and maintaining infrastructure. Similar to the way software developers write application code, AWS provides services that enable the creation, deployment and maintenance of infrastructure in a programmatic, descriptive, and declarative way. These services provide rigor, clarity, and reliability. The AWS services discussed in this paper are core to a DevOps methodology and form the underpinnings of numerous higher-level AWS DevOps principles and practices.

AWS offers the following services to define infrastructure as code.

Services

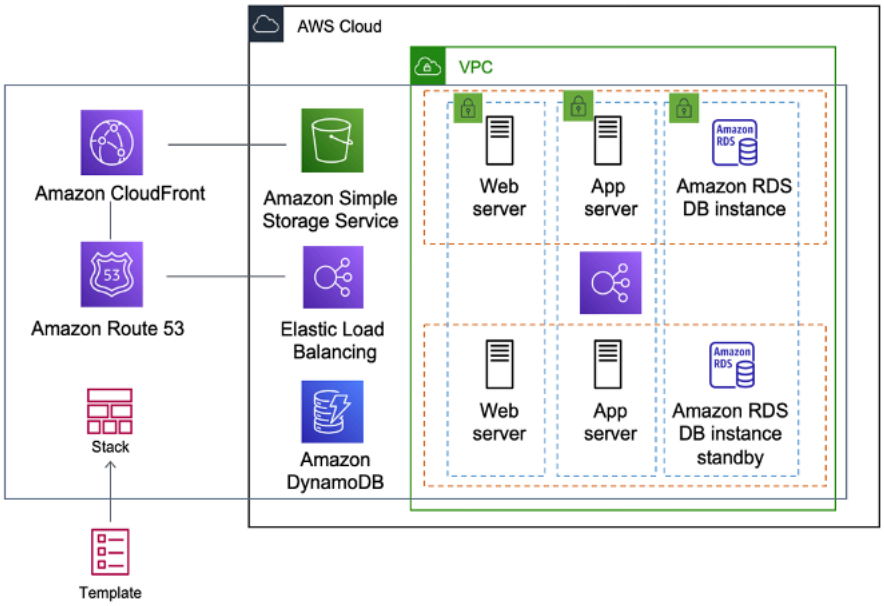
- AWS CloudFormation
- AWS Serverless Application Model
- AWS Cloud Development Kit (AWS CDK)
- AWS Cloud Development Kit for Kubernetes

- AWS Cloud Development Kit for Terraform
- AWS Cloud Control API

AWS CloudFormation

AWS CloudFormation is a service that enables developers to create AWS resources in an orderly and predictable fashion. Resources are written in text files using JSON or YAML format. The templates require a specific syntax and structure that depends on the types of resources being created and managed. You author your resources in JSON or YAML with any code editor such as AWS Cloud9, check it into a version control system, and then CloudFormation builds the specified services in safe, repeatable manner.

A CloudFormation template is deployed into the AWS environment as a stack. You can manage stacks through the AWS Management Console, AWS Command Line Interface, or AWS CloudFormation APIs. If you need to make changes to the running resources in a stack you update the stack. Before making changes to your resources, you can generate a change set, which is a summary of your proposed changes. Change sets enable you to see how your changes might impact your running resources, especially for critical resources, before implementing them.



AWS CloudFormation creating an entire environment (stack) from one template

You can use a single template to create and update an entire environment, or separate templates to manage multiple layers within an environment. This enables templates to be modularized, and also provides a layer of governance that is important to many organizations.

When you create or update a stack in the CloudFormation console, events are displayed, showing the status of the configuration. If an error occurs, by default the stack is rolled back to its previous state. Amazon SNS provides notifications on events. For example, you can use Amazon SNS to track stack creation and deletion progress using email and integrate with other processes programmatically.

AWS CloudFormation makes it easy to organize and deploy a collection of AWS resources, and lets you describe any dependencies or pass in special parameters when the stack is configured.

With CloudFormation templates, you can work with a broad set of AWS services, such as Amazon S3, Auto Scaling, Amazon CloudFront, Amazon DynamoDB, Amazon EC2, Amazon ElastiCache, AWS Elastic Beanstalk, Elastic Load Balancing, IAM, AWS OpsWorks, and Amazon VPC. For the most recent list of supported resources, refer to AWS resource and property types reference.

AWS Serverless Application Model

The AWS Serverless Application Model (AWS SAM) is an open-source framework that you can use to build serverless applications on AWS.

AWS SAM integrates with other AWS services, so creating serverless applications with AWS SAM provides the following benefits:

- **Single-deployment configuration** — AWS SAM makes it easy to organize related components and resources, and operate on a single stack. You can use AWS SAM to share configuration (such as memory and timeouts) between resources, and deploy all related resources together as a single, versioned entity.
- **Extension of AWS CloudFormation** — Because AWS SAM is an extension of AWS CloudFormation, you get the reliable deployment capabilities of AWS CloudFormation. You can define resources by using AWS CloudFormation in your AWS SAM template.
- **Built-in best practices** — You can use AWS SAM to define and deploy your IaC. This makes it possible for you to use and enforce best practices such as code reviews.

AWS Cloud Development Kit (AWS CDK)

The AWS Cloud Development Kit (AWS CDK) is an open source software development framework to model and provision your cloud application resources using familiar programming languages. AWS CDK enables you to model application infrastructure using TypeScript, Python, Java, and .NET.

Developers can leverage their existing Integrated Development Environment (IDE), using tools such as autocomplete and in-line documentation to accelerate development of infrastructure.

AWS CDK utilizes AWS CloudFormation in the background to provision resources in a safe, repeatable manner. Constructs are the basic building blocks of CDK code. A construct represents a cloud component and encapsulates everything AWS CloudFormation needs to create the component. The AWS CDK includes the AWS Construct Library, containing constructs representing many AWS services. By combining constructs together, you can quickly and easily create complex architectures for deployment in AWS.

AWS Cloud Development Kit for Kubernetes

AWS Cloud Development Kit for Kubernetes is an open-source software development framework for defining Kubernetes applications using general-purpose programming languages.

Once you have defined your application in a programming language (as of the date of this publication, only Python and TypeScript are supported), cdk8s will convert your application description in to pre-Kubernetes YAML. This YAML file can then be consumed by any Kubernetes cluster running anywhere. Because the structure is defined in a programming language, you can use the rich features provided by the programming language. You can use the abstraction feature of the programming language to create your own boilerplate code, and reuse it across all of the deployments.

AWS Cloud Development Kit for Terraform

Built on top of the open source JSII library, CDK for Terraform (CDKTF) allows you to write Terraform configurations in your choice of C#, Python, TypeScript, Java, or Go and still benefit from the full ecosystem of Terraform providers and modules. You can import any existing provider or module from the Terraform Registry into your application, and CDKTF will generate resource classes for you to interact with in your target programming language.

With CDKTF, developers can set up their IaC without context switching from their familiar programming language, using the same tooling and syntax to provision infrastructure resources similar to the application business logic. Teams can collaborate in familiar syntax, while still using the power of the Terraform ecosystem and deploying their infrastructure configurations via established Terraform deployment pipelines.

AWS Cloud Control API

AWS Cloud Control API is a new AWS capability that introduces a common set of Create, Read, Update, Delete, and List (CRUDL) APIs to help developers manage their cloud infrastructure in an easy and consistent way. The Cloud Control API common APIs allow developers to uniformly manage the lifecycle of AWS and third-party services.

As a developer, you might prefer to simplify the way you manage the lifecycle of all your resources. You can use Cloud Control API's uniform resource configuration model with a pre-defined format to standardize your cloud resource configuration. In addition, you will benefit from uniform API behavior (response elements and errors) while managing your resources.

For example, you will find it simple to debug errors during CRUDL operations through uniform error codes surfaced by Cloud Control API that are independent of the resources you operate on. Using Cloud Control API, you will also find it simple to configure cross-resource dependencies. You will also no longer require to author and maintain custom code across multiple vendor tools and APIs to use AWS and third-party resources together.

Automation and tooling

Another core philosophy and practice of DevOps is *automation*. Automation focuses on the setup, configuration, deployment, and support of infrastructure and the applications that run on it. By using automation, you can set up environments more rapidly in a standardized and repeatable manner. The removal of manual processes is key to a successful DevOps strategy. Historically, server configuration and application deployment have been predominantly a manual process. Environments become non-standard, and reproducing an environment when issues arise is difficult.

The use of automation is critical to realizing the full benefits of the cloud. Internally, AWS relies heavily on automation to provide the core features of elasticity and scalability.

Manual processes are error prone, unreliable, and inadequate to support an agile business. Frequently, an organization may tie up highly skilled resources to provide manual configuration, when time could be better spent supporting other, more critical, and higher value activities within the business.

Modern operating environments commonly rely on full automation to eliminate manual intervention or access to production environments. This includes all software releasing, machine configuration, operating system patching, troubleshooting, or bug fixing. Many levels of automation practices can be used together to provide a higher level end-to-end automated process.

Automation has the following key benefits:

- Rapid changes
- Improved productivity
- Repeatable configurations
- Reproducible environments
- Elasticity
- Automatic scaling
- Automated testing

Automation is a cornerstone with AWS services and is internally supported in all services, features, and offerings.

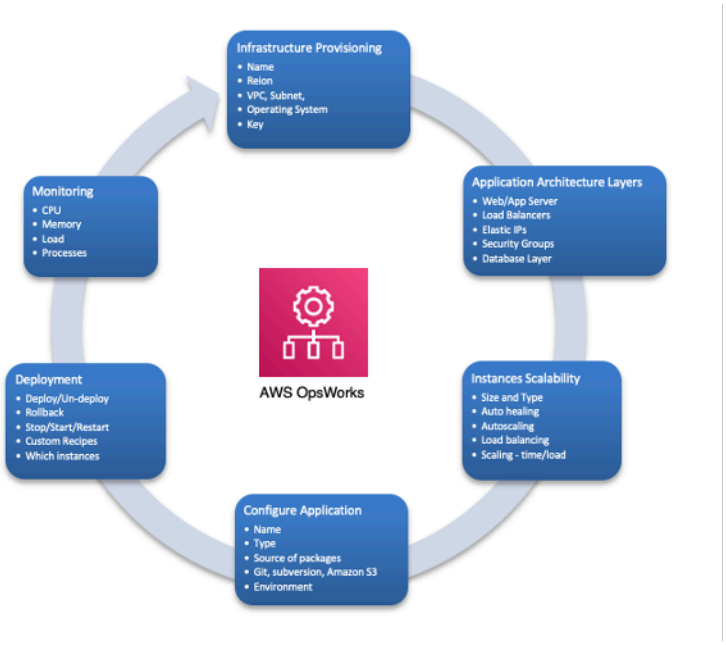
Topics

AWS OpsWorks

AWS OpsWorks takes the principles of DevOps even further than AWS Elastic Beanstalk. It can be considered an application management service rather than simply an application container. AWS OpsWorks provides even more levels of automation, with additional features such as integration with configuration management software (Chef) and application lifecycle management. You can use application lifecycle management to define when resources are set up, configured, deployed, un-deployed, or ended.

For added flexibility AWS OpsWorks has you define your application in configurable stacks. You can also select predefined application stacks. Application stacks contain all the provisioning for AWS resources that your application requires, including application servers, web servers, databases, and load balancers.

Application stacks are organized into architectural layers so that stacks can be maintained independently. Example layers could include web tier, application tier, and database tier. Out of the box, AWS OpsWorks also simplifies setting up AWS Auto Scaling groups and Elastic Load Balancing (ELB) load balancers, further illustrating the DevOps principle of automation. Just like AWS Elastic Beanstalk, AWS OpsWorks supports application versioning, continuous deployment, and infrastructure configuration management



AWS OpsWorks showing DevOps features and architecture

AWS OpsWorks also supports the DevOps practices of monitoring and logging (covered in the next section). Monitoring support is provided by Amazon CloudWatch. All lifecycle events are logged, and a separate Chef log documents any Chef recipes that are run, along with any exceptions.

AWS Elastic Beanstalk

AWS Elastic Beanstalk is a service to rapidly deploy and scale web applications developed with Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker on familiar servers such as Apache, NGINX, Passenger, and IIS.

Elastic Beanstalk is an abstraction on top of Amazon EC2, Auto Scaling, and simplifies the deployment by giving additional features such as cloning, blue/green deployments, Elastic Beanstalk Command Line Interface (EB CLI) and integration with AWS Toolkit for Visual Studio, Visual Studio Code, Eclipse, and IntelliJ for increase developer productivity.

EC2 Image Builder

EC2 Image Builderis a fully managed AWS service that helps you to automate the creation, maintenance, validation, sharing, and deployment of customized, secure, and up-to-date Linux or Windows custom AMI. EC2 Image Builder can also be used to create container images. You can use the AWS Management Console, the AWS CLI, or APIs to create custom images in your AWS account.

EC2 Image Builder significantly reduces the effort of keeping images up-to-date and secure by providing a simple graphical interface, built-in automation, and AWS-provided security settings. With EC2 Image Builder, there are no manual steps for updating an image nor do you have to build your own automation pipeline.

AWS Proton

AWS Proton enables platform teams to connect and coordinate all the different tools your development teams need for infrastructure provisioning, code deployments, monitoring, and updates. AWS Proton enables automated infrastructure as code provisioning and deployment of serverless and container-based applications.

AWS Proton enables platform teams to define their infrastructure and deployment tools, while providing developers with a self-service experience to get infrastructure and deploy code. Through AWS Proton, platform teams provision shared resources and define application stacks, including CI/CD pipelines and observability tools. You can then manage which infrastructure and deployment features are available for developers.

AWS Service Catalog

AWS Service Catalog enables organizations to create and manage catalogs of IT services that are approved for AWS. These IT services can include everything from virtual machine images, servers, software, databases, and more to complete multi-tier application architectures. AWS Service Catalog lets you centrally manage deployed IT services, applications, resources, and metadata to achieve consistent governance of your IaC templates.

With AWS Service Catalog, you can meet your compliance requirements while making sure your customers can quickly deploy the approved IT services they need. End users can quickly deploy only the approved IT services they need, following the constraints set by your organization.

AWS Cloud9

AWS Cloud9 is a cloud-based IDE that lets you write, run, and debug your code with just a browser. It includes a code editor, debugger, and terminal. AWS Cloud9 comes prepackaged with essential tools for popular programming languages, including JavaScript, Python, PHP, and more, so you don't need to install files or configure your development machine to start new projects. Because your AWS Cloud9 IDE is cloud-based, you can work on your projects from your office, home, or anywhere using an internet-connected machine.

AWS CloudShell

AWS CloudShell is a browser-based shell that makes it easier to securely manage, explore, and interact with your AWS resources. AWS CloudShell is pre-authenticated with your console credentials. Common development and operations tools are pre-installed, so there's no need to install or configure software on your local machine.

Amazon CodeGuru

Amazon CodeGuru is a developer tool that provides intelligent recommendations to improve code quality and identify an application's most expensive lines of code. Integrate CodeGuru into your existing software development workflow to automate code reviews during application development and continuously monitor application's performance in production and provide recommendations and visual clues on how to improve code quality, application performance, and reduce overall cost. CodeGuru has two components:

- **Amazon CodeGuru Reviewer** — Amazon CodeGuru Reviewer is an automated code review service that identifies critical defects and deviation from coding best practices for Java and Python code. It scans the lines of code within a pull request and provides intelligent recommendations based on standards learned from major open-source projects as well as Amazon codebase.
- **Amazon CodeGuru Profiler** — Amazon CodeGuru Profiler analyzes the application runtime profile and provides intelligent recommendations and visualizations that guide developers on how to improve the performance of the most relevant parts of their code.

Monitoring and observability

Communication and collaboration are fundamental in a DevOps philosophy. To facilitate this, feedback is critical. This feedback is provided by our suite of monitoring and observability services.

AWS provides the following services for monitoring and logging:

Topics

- Amazon CloudWatch metrics
- Amazon CloudWatch Alarms
- Amazon CloudWatch Logs
- Amazon CloudWatch Logs Insights
- Amazon CloudWatch Events
- Amazon EventBridge
- AWS CloudTrail
- Amazon DevOps Guru
- AWS X-Ray
- Amazon Managed Service for Prometheus
- Amazon Managed Grafana

Amazon CloudWatch metrics

Amazon CloudWatch metrics automatically collect data from AWS services such as Amazon EC2 instances, Amazon EBS volumes, and Amazon RDS database (DB) instances. These metrics can then be organized as dashboards and alarms or events can be created to trigger events or perform Auto Scaling actions.

Amazon CloudWatch Alarms

You can set up alarms using Amazon CloudWatch alarms based on the metrics collected by Amazon CloudWatch metrics. The alarm can then send a notification to Amazon SNS topic, or initiate Auto Scaling actions. An alarm requires period (length of the time to evaluate a metric), evaluation

period (number of the most recent data points), and datapoints to alarm (number of data points within the evaluation period).

Amazon CloudWatch Logs

Amazon CloudWatch Logs is a log aggregation and monitoring service. AWS CodeBuild, CodeCommit, CodeDeploy and CodePipeline provide integrations with CloudWatch logs so that all of the logs can be centrally monitored. In addition, the previously mentioned services various other AWS services provide direct integration with CloudWatch.

With CloudWatch Logs you can:

- Query your log data
- Monitor logs from Amazon EC2 instances
- Monitor AWS CloudTrail logged events
- Define log retention policy

Amazon CloudWatch Logs Insights

Amazon CloudWatch Logs Insights scans your logs and enables you to perform interactive queries and visualizations. It understands various log formats and auto-discovers fields from JSON logs.

Amazon CloudWatch Events

Amazon CloudWatch Events delivers a near real-time stream of system events that describe changes in AWS resources. Using simple rules that you can quickly set up, you can match events and route them to one or more target functions or streams.

CloudWatch Events becomes aware of operational changes as they occur. CloudWatch Events responds to these operational changes and takes corrective action as necessary, by sending messages to respond to the environment, activating functions, making changes, and capturing state information.

You can configure rules in Amazon CloudWatch Events to alert you to changes in AWS services and integrate these events with other third-party systems using Amazon EventBridge. The following are the AWS DevOps related services that have integration with CloudWatch Events.

- Application Auto Scaling Events

- CodeBuild Events
- CodeCommit Events
- CodeDeploy Events
- CodePipeline Events

Amazon EventBridge

> **Note**
>
> Amazon CloudWatch Events and EventBridge are the same underlying service and API, however, EventBridge provides more features.

Amazon EventBridgeis a serverless event bus that enables integrations between AWS services, Software as a services (SaaS), and your applications. In addition to build event driven applications, EventBridge can be used to notify about the events from the services such as CodeBuild, CodeDeploy, CodePipeline, and CodeCommit.

AWS CloudTrail

To embrace the DevOps principles of collaboration, communication, and transparency, it's important to understand who is making modifications to your infrastructure. In AWS, this transparency is provided by AWS CloudTrail. All AWS interactions are handled through AWS API calls that are monitored and logged by AWS CloudTrail. All generated log files are stored in an Amazon S3 bucket that you define. Log files are encrypted using Amazon S3 server-side encryption (SSE). All API calls are logged whether they come directly from a user or on behalf of a user by an AWS service. Numerous groups can benefit from CloudTrail logs, including operations teams for support, security teams for governance, and finance teams for billing.

Amazon DevOps Guru

Amazon DevOps Guru is a service powered by machine learning (ML) that is designed to make it easy to improve an application's operational performance and availability. DevOps Guru helps detect behaviors that deviate from normal operating patterns, so you can identify operational issues long before they impact your customers.

DevOps Guru uses ML models informed by years of Amazon.com and AWS operational excellence to help identify anomalous application behavior (for example, increased latency, error rates, resource constraints, and others) and surface critical issues that could cause potential outages or service disruptions.

When DevOps Guru identifies a critical issue, it saves debugging time by fetching relevant and specific information from a large number of data sources and automatically sends an alert and provides a summary of related anomalies, and context for when and where the issue occurred.

AWS X-Ray

AWS X-Ray helps developers analyze and debug production, distributed applications, such as those built using a microservices architecture. With X-Ray, you can understand how your application and its underlying services are performing to identify and troubleshoot the root cause of performance issues and errors. X-Ray provides an end-to-end view of requests as they travel through your application, and shows a map of your application's underlying components. X-Ray makes it easy for you to:

- **Create a service map** – By tracking requests made to your applications, X-Ray can create a map of services used by your application. This provides you with a view of connections among services in your application, and enables you to create a dependency tree, detect latency or errors when working across AWS Availability Zones or Regions, zero in on services not operating as expected, and so on.
- **Identify errors and bugs** – X-Ray can automatically highlight bugs or errors in your application code by analyzing the response code for each request made to your application. This enables easy debugging of application code without requiring you to reproduce the bug or error.
- **Build your own analysis and visualization apps** – X-Ray provides a set of query APIs you can use to build your own analysis and visualizations apps that use the data that X-Ray records.

Amazon Managed Service for Prometheus

Amazon Managed Service for Prometheus is a serverless monitoring service for metrics compatible with open-source Prometheus, making it easier for you to securely monitor and alert on container environments. Amazon Managed Service for Prometheus reduces the heavy lifting required to get started with monitoring applications across Amazon Elastic Kubernetes Service, Amazon Elastic Container Service, and AWS Fargate, as well as self-managed Kubernetes clusters.

Amazon Managed Grafana

Amazon Managed Grafana is a fully managed service with rich, interactive data visualizations to help customers analyze, monitor, and alarm on metrics, logs, and traces across multiple data sources. You can create interactive dashboards and share them with anyone in your organization with an automatically scaled, highly available, and enterprise-secure service.

Communication and Collaboration

Whether you are adopting DevOps Culture in your organization or going through a DevOps cultural transformation, communication and collaboration are an important part of your approach. At Amazon, we have realized that there was a need to bring a change to the mindset of our teams and thus adopted the concept of *Two-Pizza Teams*.

"We try to create teams that are no larger than can be fed by two pizzas," said Bezos. "We call that the two-pizza team rule."

The smaller the team, the better the collaboration. Collaboration is very important, as software releases are moving faster than ever. And a team's ability to deliver the software can be a differentiating factor for your organization against your competition. Imagine a situation in which a new product feature needs to be released or a bug needs to be fixed. You want this to happen as quickly as possible, so you can have a smaller go-to-market timed. You don't want the transformation to be a slow-moving process; you want an agile approach where waves of changes start to make an impact.

Communication between teams is also important as you move toward the shared responsibility model and start moving out of the siloed development approach. This brings the concept of ownership to the team, and shifts their perspective to look at the process as an end-to-end venture. Your team should not think about your production environments as black boxes where they have no visibility.

Cultural transformation is also important, because you might be building a common DevOps team or have a DevOps focused member in your team. Both of these approaches introduce Shared Responsibility in to the team.

Security

Whether you are going through a DevOps transformation or implementing DevOps principles for the first time, you should think about Security as integrated in your DevOps processes. This should be cross cutting concern across your build, test deployment stages.

Before exploring security in DevOps on AWS, this paper looks at the AWS Shared Responsibility Model.

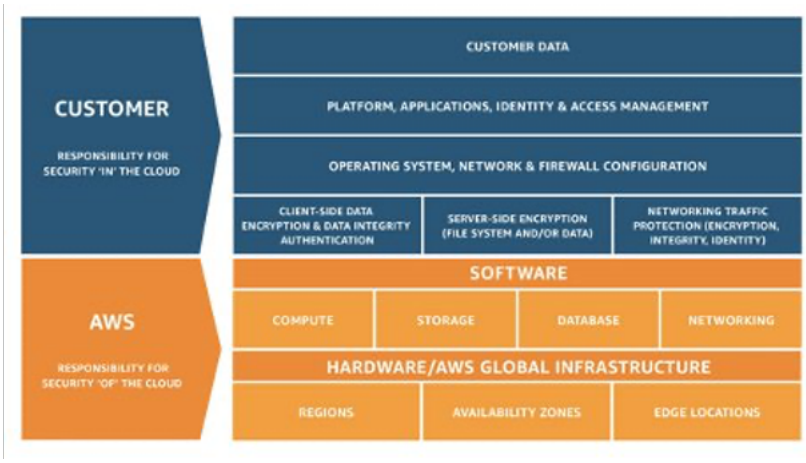
Topics

AWS Shared Responsibility Model

Security is a shared responsibility between AWS and the customer. The different parts of the Shared Responsibility Model are:

- **AWS responsibility "Security of the Cloud"** - AWS is responsible for protecting the infrastructure that runs all of the services offered in the AWS Cloud. This infrastructure is composed of the hardware, software, networking, and facilities that run AWS Cloud services.
- **Customer responsibility "Security in the Cloud"** – Customer responsibility is determined by the AWS Cloud services that a customer selects. This determines the amount of configuration work the customer must perform as part of their security responsibilities.

This shared model can help relieve the customer's operational burden as AWS operates, manages, and controls the components from the host operating system and virtualization layer down to the physical security of the facilities in which the service operates. This is critical in the cases where customer want to understand the security of their build environments.



AWS Shared Responsibility Model

For DevOps, assign permissions based on the least-privilege permissions model. This model states that “a user (or service)should have the exact access rights necessary to complete their role's responsibilities—no more, no less. ”.

Permissions are maintained in IAM. You can use IAM to control who is authenticated (signed in) and authorized (has permissions) to use resources.

Identity and Access Management

AWS Identity and Access Management (IAM) defines the controls and polices that are used to manage access to AWS resources. Using IAM you can create users and groups and define permissions to various DevOps services.

In addition to the users, various services may also need access to AWS resources. For example, your CodeBuild project might need access to store Docker images in Amazon Elastic Container Registry (Amazon ECR) and need permissions to write to Amazon ECR. These types of permissions are defined by a special type role know as service role.

IAM is one component of the AWS security infrastructure. With IAM, you can centrally manage groups, users, service roles and security credentials such as passwords, access keys, and permissions policies that control which AWS services and resources users can access. IAM Policy lets you define the set of permissions. This policy can then be attached to either a role, user, or a service to define their permission.

You can also use IAM to create roles that are used widely within your desired DevOps strategy. In some cases, it can make perfect sense to programmatically AssumeRole instead of directly getting

the permissions. When a service or user assumes roles, they are given temporary credentials to access a service that they normally don't have access to.

Conclusion

To make the journey to the cloud smooth, efficient, and effective, technology companies should embrace DevOps principles and practices. These principles are embedded in AWS, and form the cornerstone of numerous AWS services, especially those in the deployment and monitoring offerings.

Begin by defining your infrastructure as code using the service AWS CloudFormation or AWS CDK. Next, define the way in which your applications are going to use continuous deployment with the help of services like AWS CodeBuild, AWS CodeDeploy, AWS CodePipeline, and AWS CodeCommit. At the application level, use containers such as AWS Elastic Beanstalk, Amazon ECS, or Amazon Elastic Kubernetes Service (Amazon EKS). Use AWS OpsWorks to simplify the configuration of common architectures. Using these services also makes it easy to include other important services such as Auto Scaling and Elastic Load Balancing.

Finally, use the DevOps strategy of monitoring such as Amazon CloudWatch, and solid security practices such as IAM.

With AWS as your partner, your DevOps principles bring agility to your business and IT organization and accelerate your journey to the cloud.

Document Revisions

To be notified about updates to this whitepaper, subscribe to the RSS feed.

Change	Description	Date
Updated	Updated	April 7, 2023
Updated sections to include new services	Updated sections to include new services	October 16, 2020
Initial publication	Whitepaper first published	December 1, 2014

Contributors

Contributors to this document include:

- Abhra Sinha, Solutions Architect
- Anil Nadiminti, Solutions Architect
- Muhammad Mansoor, Solutions Architect
- Ajit Zadgaonkar, World Wide Tech Leader, Modernization
- Juan Lamadrid, Solutions Architect
- Darren Ball, Solutions Architect
- Rajeswari Malladi, Solutions Architect
- Pallavi Nargund, Solutions Architect
- Bert Zahniser, Solutions Architect
- Abdullahi Olaoye, Cloud Solutions Architect
- Mohamed Kiswani, Software Development Manager
- Tara McCann, Manager, Solutions Architect

Notices

Customers are responsible for making their own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) represents current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. AWS products or services are provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.